

INFORMATION AND COMMUNICATION TECHNOLOGY

S4

STUDENT BOOK

Copyright Page

© 2026 Rwanda Basic Education Board

All rights reserved.

This book is the property of the government of Rwanda.
Credit must be provided to REB when the content is quoted.

FOREWORD

Dear Student,

Rwanda Basic Education Board is honored to present to you this ICT Student's Book for Senior Four which serves as a guide to competence-based teaching and learning to ensure consistency and coherence in the learning of ICT. The Rwandan educational philosophy is to ensure that you achieve full potential at every level of education which will prepare you to be well integrated in society and exploit employment opportunities.

The government of Rwanda emphasizes the importance of aligning teaching and learning materials with the syllabus to facilitate your learning process. Many factors influence what you learn, how well you learn and the competences you acquire. Those factors include the instructional materials available among others. In this book, special attention was paid to the activities that facilitate the learning process in which you can develop your ideas and make new discoveries during concrete activities carried out individually or with peers.

In competence-based curriculum, learning is considered as a process of active building by the learner where concepts are mainly introduced by an activity, a situation or a scenario that helps the learner to construct knowledge, develop skills and acquire positive attitudes and values.

For effective use of this textbook, your role is to:

- Work on given activities which lead to the development of knowledge and skills;
- Share relevant information with other learners through presentations, discussions, group work and other active learning techniques such as field visit, laboratory work, investigation, role play, case studies and research in the library, from the Internet or from your community;
- Participate and take responsibility for your own learning;
- Draw conclusions based on the findings from the learning activities.

I wish to sincerely extend my appreciation to REB staff who organized the editing process of this book. Special gratitude goes to the lecturers, teachers, illustrators and designers who diligently worked to successful completion of this book. Any comment or contribution would be welcome for the improvement of this textbook for the next edition.



Dr. MBARUSHIMANA Nelson
Director General, REB

ACKNOWLEDGEMENT

I wish to express my appreciation to all the people who played a major role in editing process of ICT Student's Book for Senior Four. It would not have been successful without their active participation.

Special thanks are given to those who gave their time to read and refine this textbook to meet the needs of competence-based curriculum. I owe gratitude to different Universities and secondary schools in Rwanda that allowed their staff to work with REB to edit this book. I therefore, wish to extend my sincere gratitude to lecturers, teachers, illustrators, designers and all other individuals whose efforts in one way or the other contributed to the completion of this edition.

Finally, my word of gratitude goes to the Rwanda Basic Education Board staff particularly those from Curriculum, Teaching and Learning Resources Department who were involved in the whole process of editorial work.



Joan MURUNGI,

Head of Department of Curriculum, Teaching and Learning Resources

Contents

FOREWORD	i
ACKNOWLEDGEMENT	ii
UNIT 1: GOOGLE DOCS	11
Key unit competency	11
Learning objectives	11
Knowledge and understanding	11
Skills	11
Attitudes and values	12
1.0 Introduction.....	12
Introductory Activity.....	13
1.1. Introduction to Google Doc	13
1.1.1. Meaning and importance.....	13
1.1.2. Accessing Google Docs	14
1.1.3. Navigating the interface.....	14
1.1.4. Creating a new document.....	16
1.2. Text formatting basics.....	17
1.2.1. Changing font style, size, and color.....	18
1.2.2. Bold, italics, underline, and highlighting.....	18
1.2.3. Alignments	19
1.2.4. Line spacing and paragraph formatting	19
1.3. Working with lists and tables	20
1.3.1. Bulleted and numbered lists.....	20
1.3.2. Creating and formatting tables.....	21
1.3.3. Adjusting rows and columns.....	22
1.4. Inserting elements	23
1.4.1. Images, drawings, and shapes	23

1.4.2. Links and bookmarks	25
1.4.3. Headers, footers, and page numbers	26
1.5. Sharing and collaboration	28
1.5.1. Sharing settings	29
1.5.2. Working on the same document	29
1.5.3. Using comments and suggestions	29
1.6 End of unit assessment	31
UNIT 2: DIGITAL LOGIC FUNDAMENTAL	33
Key unit competence	33
Learning objectives	33
Knowledge and understanding	33
Skills	33
Attitudes and values	34
2.0 Introduction:	34
Introductory activity	34
2.1 Number systems	34
2.1.1 Types of number systems	35
2.1.2 Numbers conversion	38
2.2 Boolean algebra and logic gates	45
2.2.1 Boolean logic and its operators	45
2.3 Logic circuit design	48
2.3.1. Logic simulation tools	49
2.3.2 Interconnecting logic gates	52
2.4 Constructing logical expressions	55
2.4.1. Boolean expressions	56
2.4.2. Truth table	57
2.5 Logic circuits optimization	58

2.5.1. Boolean rules	58
2.5.2. Simplification of boolean expressions	60
2.6 End of unit assessment.....	61
UNIT 3: PROGRAMMING ALGORITHM	63
Key Unit Competence	63
Knowledge and understanding	63
Skills	63
Attitudes and values.....	63
3.0 Introduction.....	64
Introductory activity.....	64
3.1 Introduction to algorithms.....	64
3.1.1 Definition of algorithms.....	64
3.1.2 Characteristics of algorithm.....	65
3.1.3 Importance of algorithms.....	66
3.2 Variables and operations	67
3.2.1 Definition of variable.....	67
3.2.2 Data type	69
3.2.3 Declaration.....	70
3.2.4 Operators.....	71
3.3. Input and output.....	73
3.3.1 Initialization.....	74
3.3.2 Read and write statement	77
3.4 Algorithm representation	80
3.4.1 Concept of flowchart.....	80
3.4.2 Flowchart symbols	81
3.4.3 Create basic flowcharts from given algorithms or pseudocode ..	82
3.5 Conditional statement.....	83

3.5.1 If statement.....	84
3.5.2 If else statement	85
3.6 Nested if conditional statements	87
3.6.1 Nested if statement social	87
3.7 Loop statement.....	90
3.7.1 For loop.....	91
Pseudocode	92
Output	92
SUM = 15.....	92
3.7.2 While loop.....	92
Pseudocode	93
Output	93
3.8 End of unit assessment.....	94
UNIT 4: PYTHON PROGRAMMING I	96
Key Unit Competence	96
Knowledge and understanding	96
Skills	96
Attitudes and values	96
4.0 Introduction	96
Introductory activity	97
4.1. Introduction to python.....	97
4.1.1 Python overview	98
4.1.2 Python installation	99
4.2 Variables operation.....	102
4.2.1 Data types.....	103
4.2.2 Variable	104
4.2.3 Operators.....	104

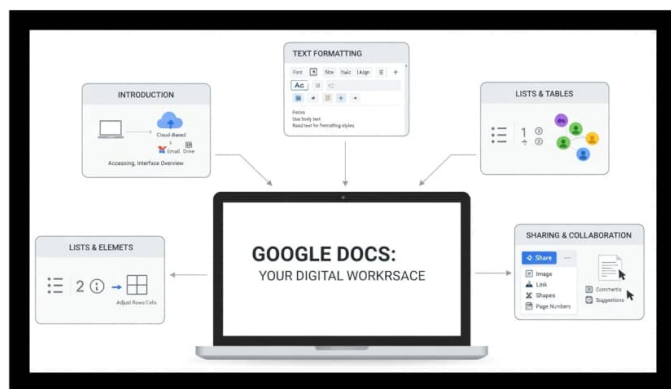
2) Answer true or false	106
4.3 Input and output functions	106
4.3.1 Input function.....	107
4.3.2 Output function	108
4.4 Conditional statements.....	108
4.5 Nested conditions.....	110
4.5.1 Nested if	111
4.6 While loops statements	112
4.6.1 While loop.....	112
4.7 For loop statements	114
4.7.1 For loop.....	114
4.8 End of unit assessment.....	115
4.8.1. Project Activity	115
4.8.2 Assessment questions.....	116
UNIT 5: HTML-BASED WEB DEVELOPMENT	119
Key unit competence.....	119
Knowledge and understanding	119
Skills	119
Attitudes and values.....	119
5.0 Introduction.....	119
Introductory activity.....	120
5.1 Introduction to HTML	120
5.1.1 Definition of HTML	120
5.1.2 Role of HTML	121
5.1.3 HTML page structure.....	121
5.2 Page structure tags	122
5.3 Text formatting tags	124

5.3.1 Headings tag.....	125
5.3.2 Paragraphs tag.....	125
5.3.3 Emphasis tags.....	126
5.3.4 Marquee tag	126
5.4 List tags.....	127
5.4.1 Unordered list, ordered list, list item tags	127
5.5 Media tags.....	129
5.5.1 Embed images tag	129
5.5.2 Audio tag	129
5.5.3 Video tag	130
5.6 Table tags	131
5.6.1 Structure tabular data tags.....	131
5.7 Form elements tags	132
5.7.1 Handle user input tags.....	133
5.8 Link and navigation tags.....	135
5.8.1 Hyperlinks.....	135
5.9 End of unit assessment.....	137
5.9.1 Project	137
5.9.2 Assessment questions.....	137
UNIT 6: CASCADING STYLE SHEET.....	138
Key unit competence	139
Knowledge and understanding	139
Skills	139
Attitudes and values	139
6.0 Introduction	139
Introductory activity	140
6.1 Text formatting.....	140

6.1.1 Font styling	141
6.1.2 Text appearance.....	146
6.1.3 Spacing and layout.....	150
6.1.4 Effects	153
6.2 Layout styling	157
6.2.1 Borders and padding elements	158
6.2.2 Background of elements	161
6.3 Styles with internal CSS	167
6.4 Styles with external CSS.....	172
6.5 Styles with inline CSS	176
6.6 End of unit assessment.....	180
6.6.1 Project activity	180
6.6.2 End of unit assessment.....	181
UNIT 7: DATABASE DESIGN.....	184
Key unit competence.....	184
Knowledge and understanding.....	184
Skills	184
Attitudes and values.....	184
7.0 Introduction.....	184
Introductory activity.....	185
7.1 Database concepts.....	185
7.1.2 Physical independence	188
7.1.3 Relational database model	190
7.1.4 Primary key	192
7.1.5 Foreign key	193
7.2 Types of relationship.....	195
7.2.1 One to one relationship	196

7.2.2 One to many relationship	197
7.2.3 Many to many relationship	198
7.3 Entity relationship diagram.....	200
7.3.1 Entities	200
7.3.2 Attributes.....	202
7.3.3 Relationship	204
7.3.4 Representation of E-R-D.....	204
7.4 Relational model	206
7.4.1 Tables	207
7.4.2 Columns	207
7.4.3 Rows	208
7.4.4 Primary keys	209
7.4.5 Foreign keys.....	210
7.4.6 Data types.....	212
7.5 Keys in relational model	214
7.5.1 Tables creation	215
7.5.2 Relation model representation	218
7.6 Normalization	222
7.6.1 Normalization definition.....	223
7.6.2 Normalization role	224
7.7 Types of normal forms	225
7.7.1 First normal form	226
7.7.2 Second normal form.....	227
7.7.3 Third normal form.....	229
7.8 End of unit assessment.....	231
7.8.1 Project activity	231
7.8.2 Assessment questions.....	232

UNIT 1: GOOGLE DOCS



Key unit competency:

Learners should be able to create documents using Google Docs to enhance communication, organization, and teamwork in a working environment effectively.

Learning objectives:

At the end of this unit, you will be able to:

Knowledge and understanding

- Explain the use of Google Docs.
- Describe how to access Google Docs through Gmail or Google Drive.
- Identify parts of the Google Docs interface.
- State the steps for creating a new document.
- Explain basic text formatting features and their functions.
- Describe how to create and format lists and tables.
- Identify the process of inserting elements.
- Explain sharing and collaboration features in Google Docs.

Skills:

- Create a new document.
- Navigate the Google Docs interface.
- Apply the basic text formatting.
- Insert images, shapes, links, and page elements.
- Collaborate with others by sharing documents.
- Navigate through menus and toolbars.

Attitudes and values:

- Appreciate the importance of collaboration and teamwork in digital environments.
- Develop a sense of responsibility when sharing and editing documents online.
- Demonstrate creativity and organization when formatting documents.
- Value accuracy and neatness in digital document preparation.
- Show respect for others' contributions when working collaboratively.

1.0 Introduction

The purpose of this unit is to help students understand the basic concepts and functionalities of Google Docs. It enables students to recognize the importance of Google Docs, access it via Gmail or Google Drive, and navigate its interface including the menu, toolbar, and workspace. Students will also be able to create new documents and apply text formatting such as changing font styles, sizes, colors, using bold, italics, underline, highlighting, alignment, and paragraph spacing.

The unit further introduces students to working with lists and tables, including creating bulleted and numbered lists, formatting tables, and adjusting rows and columns. Students will also explore how to insert elements like images, drawings, shapes, links, bookmarks, headers, footers, and page numbers. Finally, the unit covers sharing and collaboration, enabling students to set sharing permissions, work on the same document in real time, and use comments and suggestions effectively.

Introductory Activity:

- 1) If you use a notebook, how many people can write at the exact same time?
- 2) If you use Google Docs, how does the Auto-save feature prevent you from losing your work if the power goes out?
- 3) List two advantages of having your school assignments saved online instead of on a flash drive.

1.1. Introduction to Google Doc

Learning activity 1.1

- 1) Rukundo wants to create and edit a class assignment online with her classmates using one document. Which application can help her do this, and why?
- 2) Define Google Docs in your own words.
- 3) a) State any three importance of Google Docs.
b) Identify two gateways used to access Google Docs.

1.1.1. Meaning and importance

Google Docs is a Web-based application in which documents can be created, edited and stored online. It allows you to create, edit, format, and share documents using the internet. Google Docs is a component of the Google Workspace suite. It functions similarly to Microsoft Word but operates entirely within a web browser.

Importance of Google Docs

- **Collaboration:** Multiple users can edit the same document simultaneously.
- **Cloud Storage:** Documents are saved to Google Drive, reducing the risk of file loss.
- **Cost:** It is free for anyone with a Google account.
- **Accessibility:** Work from any device (laptop, tablet, or smartphone).

1.1.2. Accessing Google Docs

To access Google Docs there are two primary gateways to reach Google Docs:

- **Via Gmail:** Open your email, click the Apps Grid, and select Docs.
- **Via Google Drive:** Open Drive, click the "+ New" button, and select Google Docs.

ACCESSING GOOGLE DOCS: TWO GATEWAYS

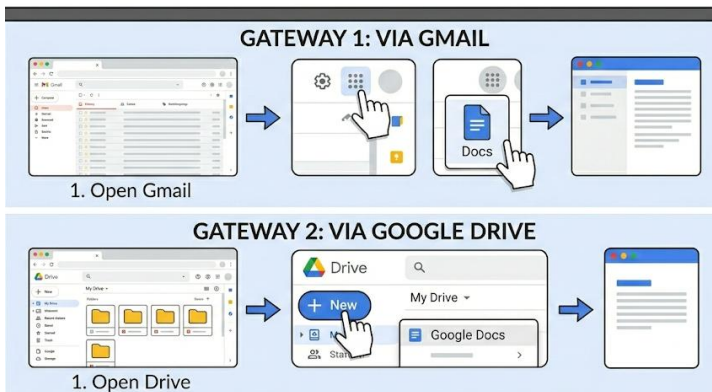


Figure 1. 1. Two gateways of Google Docs

1.1.3. Navigating the interface

Once you open a document, you will see three main sections:

-The menu bar: This is where all the deep settings and tools are hidden including File ,Edit ,View ,Insert ,Format ,etc.

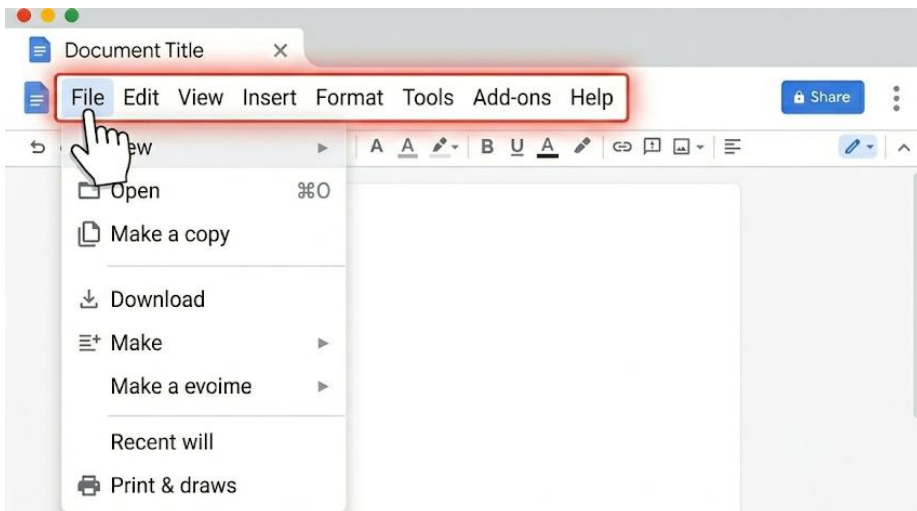


Figure 1. 2. The menu

-The shortcut toolbar: This sits right below the menu. It contains icons for common tasks like Bold, Italics, Font Size, and Alignment.

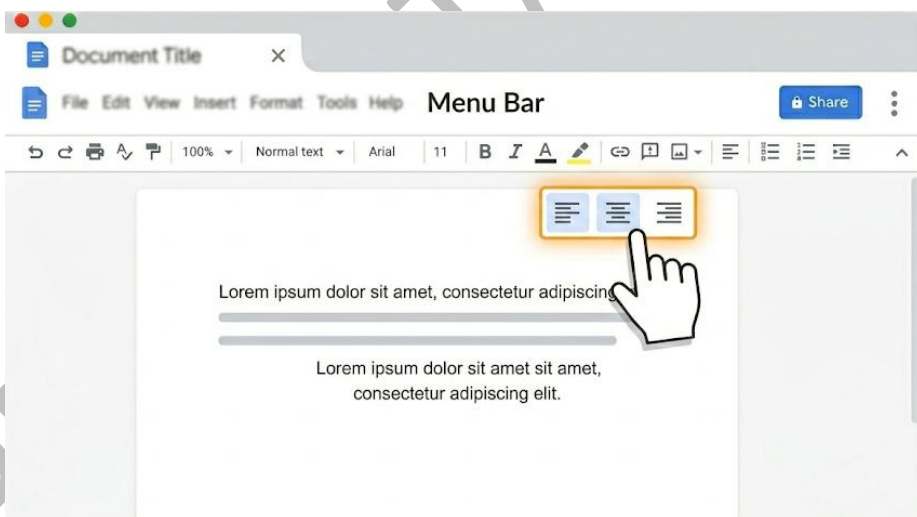


Figure 1. 3. Shortcut Toolbar

-The workspace: This is the large white area that looks like a sheet of paper. This is where you type your content.

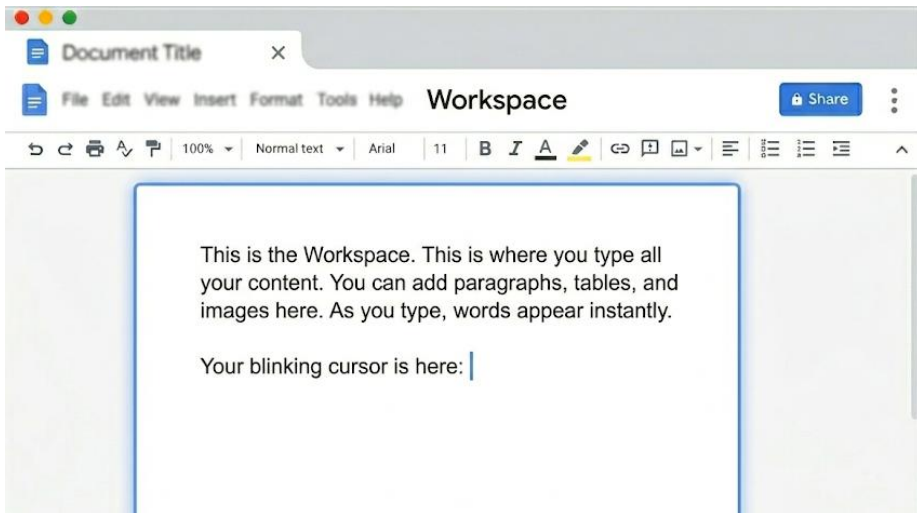


Figure 1. 4. The Workspace

1.1.4. Creating a new document

Creating a new document means starting a fresh page where you can type your work, such as notes, assignments, or reports.

- **Blank document:** click the large “+” to start with a clean slate.
- **Templates:** Choose from pre-made designs like “Resume”, “Report”, or “project proposal” to save time on formatting.
- **Naming:** Always click on "Untitled Document" in the top-left corner immediately after creating a file to give it a specific name.

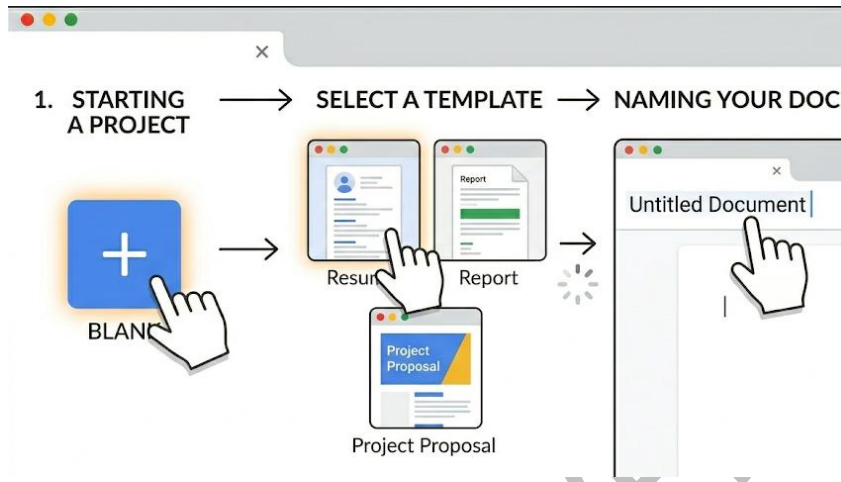


Figure 1. 5. Creating a New Document

Application activity 1.1

1) Access Google Docs using the Google Drive method.

- Create a Blank document.
- Rename the document as Lesson1_Intro_Practice.
- Type three sentences describing your favorite school subject.
- Locate the Toolbar and change your text to Bold and Blue.
- Close the browser tab. Re-open it and verify if your changes were saved automatically.

2) What are the requirements to access Google Docs?

1.2. Text formatting basics

Learning activity 1.2

- Open a new, blank Google Doc.
- Type your full name at the top of the page.
- Hover your mouse over the icons in the Shortcut Toolbar without clicking.
- Can you find the icon that looks like a Paint Bucket or a Highlighter? What is the official name that pops up in the tooltip?

5) Find the 100% dropdown menu. What happens to your workspace when you change it to 150%?

1.2.1. Changing font style, size, and color

To change how your words look, you must first select the text you want to change.

- **Font style:** Click the font name to choose a new typeface like Times New Roman or Impact.
- **Font size:** Use the + or - buttons to make text larger for headings or smaller for footnotes.
- **Font color:** Click the "A" with a colored bar under it to pick a new color for your text.



Figure 1. 6. Applying Font Style, Size, and Color

1.2.2. Bold, italics, underline, and highlighting

These tools help specific words stand out from the rest of the paragraph:

- **Bold (B):** Makes text thicker. Used for headings or key terms.
- **Italic (I):** Slants the text. Used for book titles or emphasis.
- **Underline (U):** Adds a line below text.
- **Highlight Color:** Looks like a felt-tip pen icon. It adds a splash of color *behind* the text.

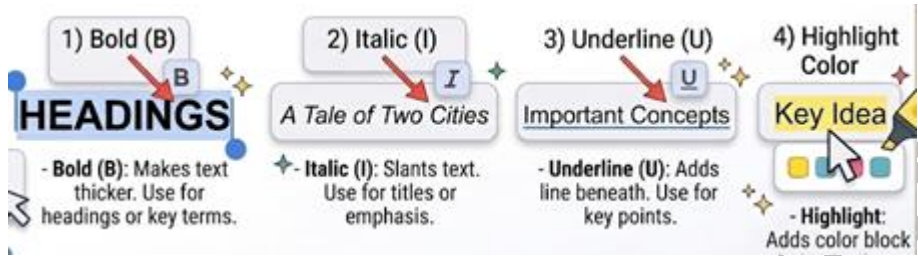


Figure 1. 7. Applying Bold, Italics, Underline, and Highlighting

1.2.3. Alignments

Alignment determines how your text sits between the left and right margins of the page:

- **Left align:** The default style ,text starts at the left margin.
- **Center align:** Text is positioned in the middle, means it is ideal for titles.
- **Right align:** Text sits against the right margin means it is used for dates.
- **Justify:** Text is spread evenly between both margins for a clean block look.

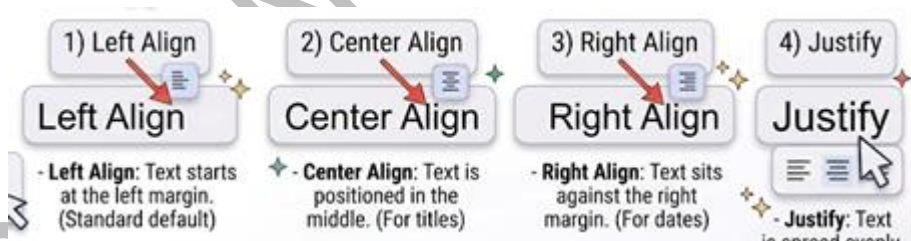


Figure 1. 8. Applying text alignments

1.2.4. Line spacing and paragraph formatting

Line spacing is the amount of white space between lines of text.

- **Single space:** Lines are close together.
- **Double space (2.0):** Adds a full blank line between each row of text.
- **Paragraph spacing:** Adds extra space before or after a paragraph to separate ideas clearly.

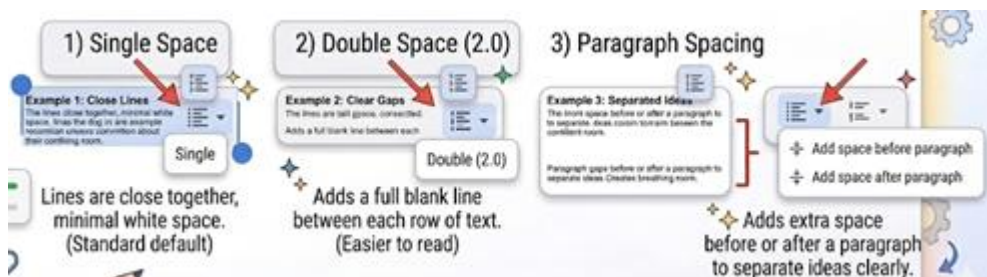


Figure 1. 9. Applying Line Spacing and Paragraph Formatting

Application activity 1.2

- 1) Open your "Lesson1_Intro_Practice" document from the previous lesson.
- 2) Type "My Favorite Subject" at the very top. Center align it and change the Font size to 18.
- 3) Pick one key word in your sentences and make it Bold and Italic.
- 4) Highlight your favorite school subject's name in Yellow.
- 5) Highlight all your text and change the Line spacing to double.

1.3. Working with lists and tables

Learning activity 1.3

- 1) Open a new, blank Google Doc.
- 2) Type three random grocery items on different lines.
- 3) Highlight all three items.
- 4) Locate the toolbar icon that has numbers and click it. What happened to your list?

Text is not always written in paragraphs. Sometimes, information needs to be structured in a specific order or categorized into data cells. This is where lists and tables become essential.

1.3.1. Bulleted and numbered lists

Use lists to separate items from the main paragraph text. There are two main types of lists found in the shortcut toolbar:

- **Numbering:** Use this list type when the order matters. This is ideal for steps in a recipe, instructions, or rankings.
- **Bulleted List:** Use this list type when you have a set of items but the order does not matter, such as a generic grocery list or a list of ideas.

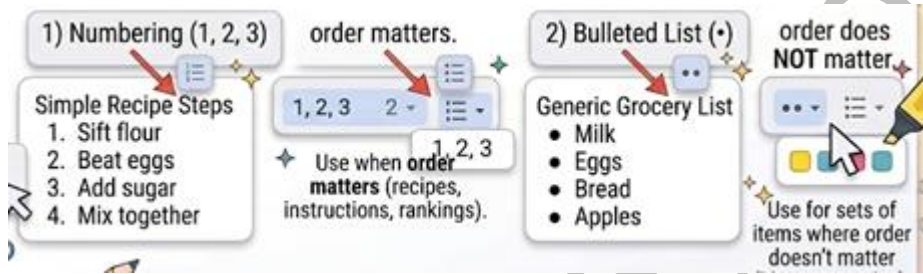


Figure 1.10. Applying Bulleted and Numbered Lists

Note: You can change the appearance of the numbers or the bullets by clicking the small downward-pointing arrow next to the list icon.

1.3.2. Creating and formatting tables

A table allows you to structure information in a clean grid made of rows and columns.

a. Creating a table:

- Navigate to the main Menu bar and select Insert.
- Hover your mouse over Table.
- A grid visual will appear then drag your mouse to select how many columns and rows you need (e.g., 3 x 4) and click.

b. Formatting a table:

Formatting a table: means changing the appearance and layout of a table so that it looks clear, organized, and professional .

- **Highlight cells** within the table to use standard tools like Bold, Center align, or change Font Color.

- **Table Properties:** If you right-click inside a cell, you can find the Table Properties menu, where you can change cell background colors, border thickness, or border colors.

1.3.3. Adjusting rows and columns

The initial size of your table is almost never perfect. You will frequently need to adjust the structure after you've created it:

- To resize:** If you need a column wider, hover your mouse cursor directly over the border line until it changes into a two-sided arrow. Click and drag the line to make the column or row larger or smaller.
- **To add/remove:** You are not stuck with the original grid size. If you left-click inside a cell and then right-click, a menu will appear allowing you to select one of these options: Insert row above or below, Insert column left or right and Delete row/column.

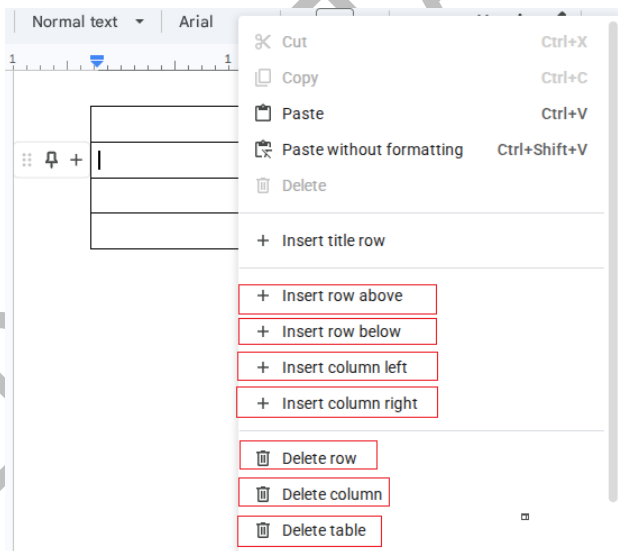


Figure 1. 11. Inserting and deleting table, row and column

Application activity 1.3

- 1) Access Google Docs and create a blank document.
- 2) Rename the document as "Lesson1_Lists_and_Tables_Practice".
- 3) Type "Shopping List" and press Enter. Underneath, use a bulleted list to create a list of at least five items.
- 4) Move to a new area on your page. Insert a 2 x 3 table.
- 5) In the first column, add "Subject" as the title. In the second column, add "Teacher" as the title. Fill the other rows with your actual class subjects and teachers.
- 6) Center Align both of the header titles ("Subject" and "Teacher") and make them Bold. Drag the column border line so that the Subject column is much wider than the Teacher column.

1.4. Inserting elements

Learning activity 1.4

- 1) Open your Lesson1_Lists_and_Tables_Practice document from the last activity.
- 2) Find the Insert menu in the Menu Bar and click it.
- 3) Look for the word Image in the list. Without clicking, move your mouse over it. A side menu will appear.
- 4) Which of the following options is the easiest way to add a picture directly from the internet in Google Docs?
 - a) Upload from computer b) Search the web
 - c) Drive d) Photos

A professional document often includes more than just words. You may need to add visual explanations, connect to external resources, or add professional page numbering. This is where the Insert menu becomes essential.

1.4.1. Images, drawings, and shapes

In Google Docs, images, drawings, and shapes are important elements that help make your document more attractive, clear, and easy to understand.

Add an image in this way:

- Go to the menu bar and select insert.
- Hover over image.
- Choose your source: You can choose to upload from computer, Search the web or Drive.

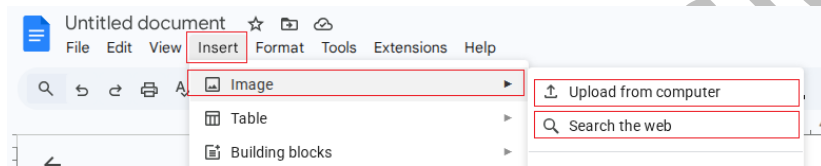


Figure 1. 12. Inserting images

Drawings and shapes: If you need a specific diagram or flow chart, use insert > Drawing. This opens a mini-canvas where you can:

- Use the Line tool to make custom lines or connectors.
- Use the Shape tool to draw squares, circles, or stars .
- Once finished, click Save and Close to put your creation into the document.

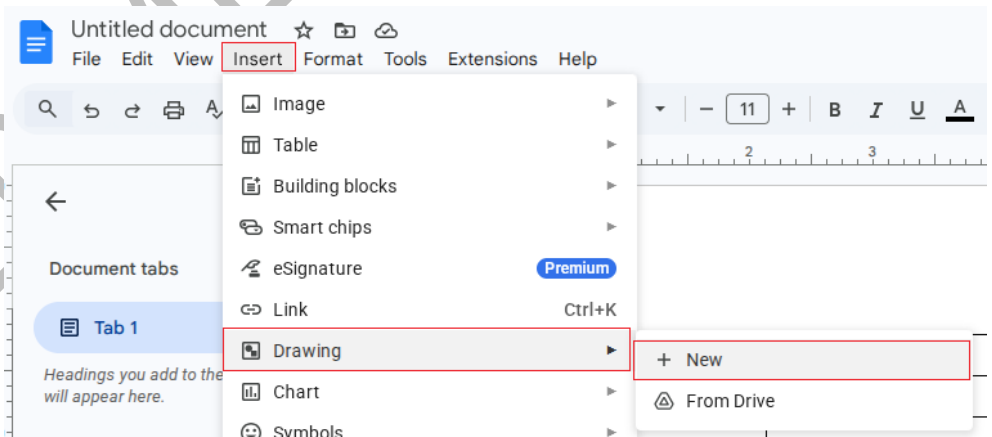


Figure 1. 13. Adding new drawings and shapes

The picture below shows the Drawing tool in Google Docs, specifically the Shapes menu. This tool allows users to create and insert drawings, shapes, arrows, callouts, and other objects into a document. Drawings and shapes help make documents more attractive, organized, and easier to understand by presenting information visually.

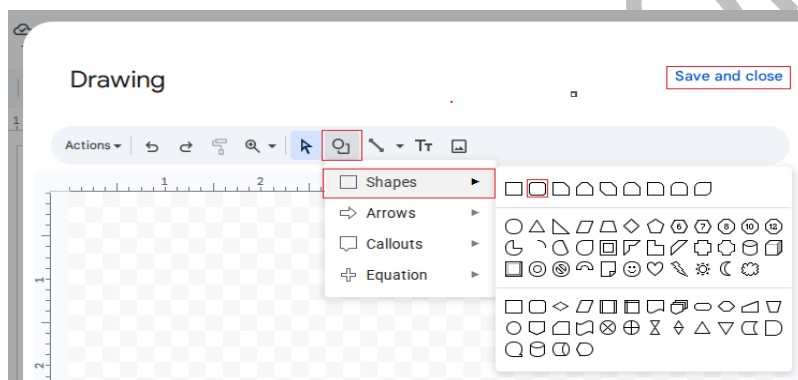


Figure 1. 14. Adding shape

1.4.2. Links and bookmarks

In Google Docs, links and bookmarks help users organize documents and quickly move to important information.

a) **Links:** A link is a clickable text or object that directs users to a webpage, file, email address, or another location in a document. Links (Hyperlinks) connects your text to an external website.

- Highlight (select) the text you want to turn into a link.
- Click Insert > Link, you can use the shortcut Ctrl+K for window or Cmd+K for mac
- Type or paste the URL into the box that appears and click Apply. The text will now be blue and underlined.

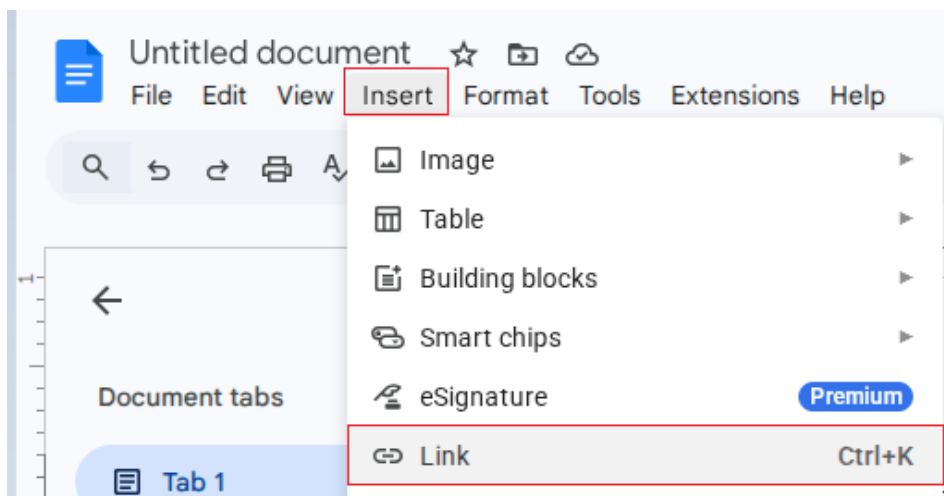


Figure 1. 15. Applying links

b) **Bookmark:** A bookmark is a marker placed at a specific point in a document to help users quickly return to or jump to that location. Bookmarks connects text to another spot within the same document.

- Click where you want your bookmark to be (like a chapter heading).
- Select Insert > Bookmark. A small flag icon will appear.
- To link *to* that bookmark, select different text and follow the Insert > Link steps, but this time select Bookmarks as the destination. (Ideal for complex, multi-page reports)

1.4.3. Headers, footers, and page numbers

These tools add repeated, professional information at the very top or bottom of every page.

- **Header:** The area at the very top margin of the page. It's perfect for adding your name, the date, or the document title.
- **Footer:** The area at the very bottom margin. Often used for contact information or footnotes.

Go to Insert > Page element> Headers & footers.

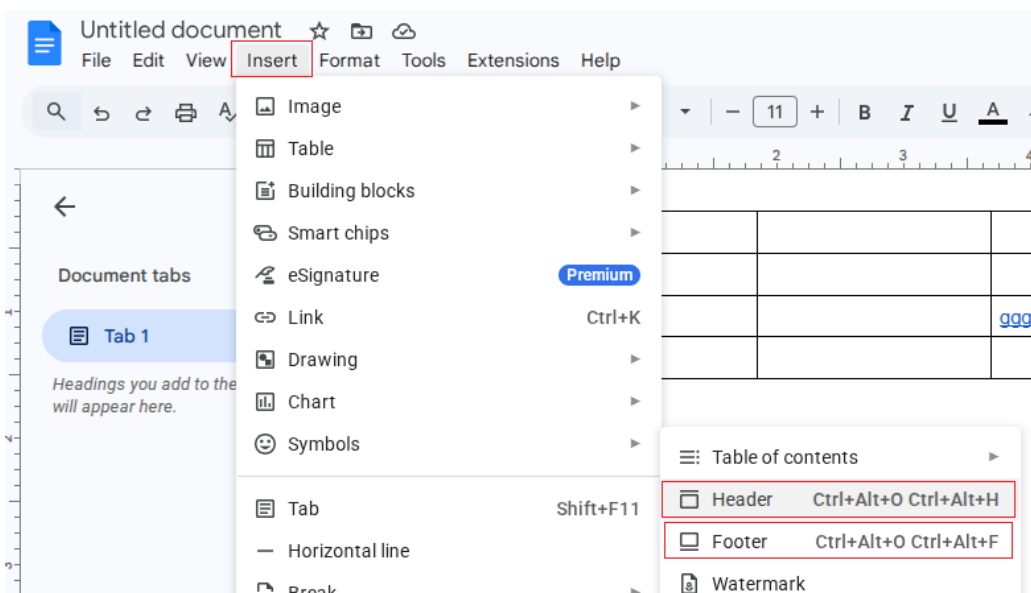


Figure 1. 16. Inserting headers, footers

- **Page numbers:** You should never type page numbers manually, as they won't update when you add more text. Let Google Docs do it automatically.
 - 1) Go to Insert > Page elements > Page numbers.
 - 2) Choose where you want the numbers (top or bottom) and whether you want them to start on the first or second page.

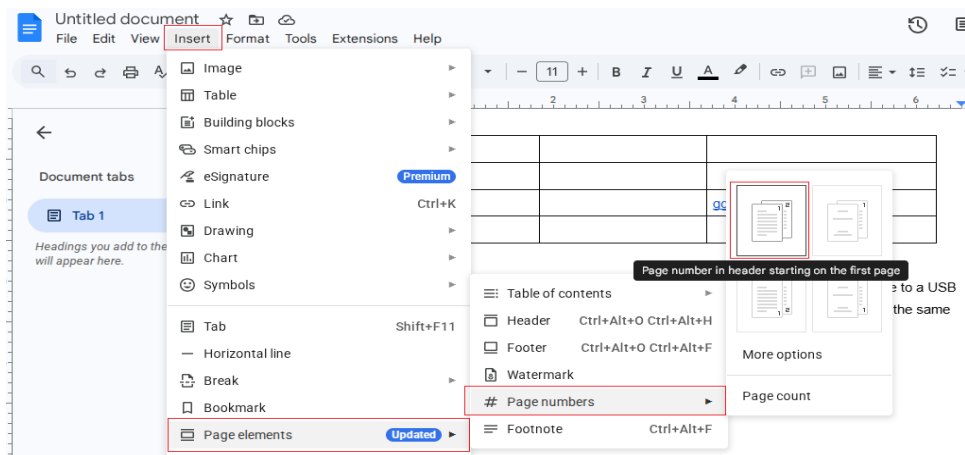


Figure 1. 17. Inserting page numbers

Application Activity 1.4

- 1) Access Google Docs and open your "Lesson1_Lists_and_Tables_Practice" document.
- 2) Type your full name and the current date inside the header space.
- 3) Insert a Search the web image of anything school-related (e.g., a "classroom" or "backpack"). Resize it by dragging the blue corners to make it about 3 inches wide.
- 4) Insert a Link to "www.google.com". Verify that the word turns blue.
- 5) Choose the option to put the number in the bottom-right corner of *every* page.

1.5. Sharing and collaboration

Learning activity 1.5

- 1) Open your "Lesson1_Intro_Practice" document.
- 2) Look at the top-right corner for the large Share button.
- 3) Does it have a Padlock icon on it? If so, hover your mouse over it. What does the tooltip say?

4) After clicking the Share button in Google Docs, you see the General access section in the pop-up window. What is its default setting?

- | | |
|------------------|--------------------|
| a) Public access | b) Anyone can edit |
| c) Restricted | d) Comment only |

1.5.1. Sharing settings

When you click Share, you decide exactly how much power to give others. There are three main access levels:

- **Viewer:** The person can read the document but cannot change anything.
- **Commenter:** The person can read and leave comments on the side, but cannot change your actual typing.
- **Editor:** The person has full power to type, delete, and format.

1.5.2. Working on the same document

When multiple people are in a document at once, their profile icons appear at the top-right.

- **Cursors:** You will see colorful cursors moving across the page, each labeled with the person's name.
- **Instant Updates:** Every letter they type appears on your screen instantly.

1.5.3. Using comments and suggestions

a) Comments

A comment is a note added to provide feedback or ask questions. The following steps show how you can add a comment to a selected part of a document.

Step1: Highlight the text you want to discuss.

Step2: Click the Add comment icon (a blue speech bubble with a plus sign) on the right margin.

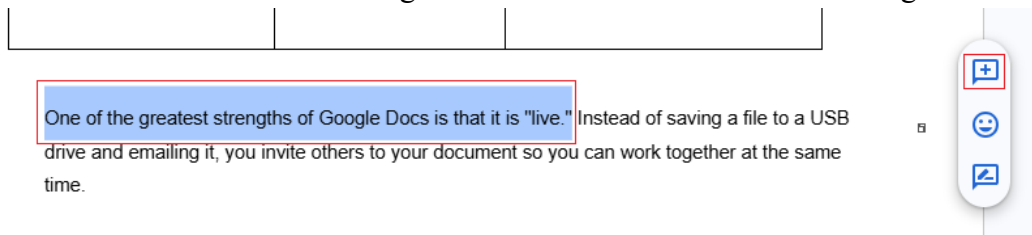


Figure 1. 18. Adding comment

Step3: Type your message and click Comment.

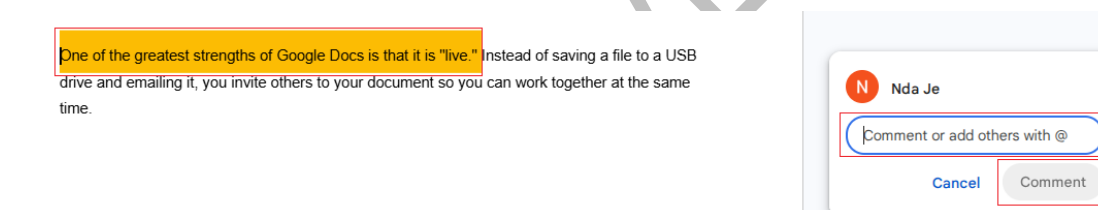


Figure 1. 19. Adding comment

b) Suggestion

Suggestion is a proposed edit that can be accepted or rejected later. These features help users collaborate and work together effectively on documents.

Switch from editing to suggesting mode in the top-right corner.

- Any text you delete will be crossed out in green.
- The owner can click the Checkmark as to accept or the X as to reject on your suggestion.

Application activity 1.5

- 1) Type the email address of a classmate or your teacher.
- 2) Change their role to Commenter and click Send.
- 3) Click the Pencil icon and select Suggesting.

- 4) Delete a word in one of your sentences. Notice how it gets a green line through it instead of disappearing.
- 5) Click the Checkmark to Accept your own suggestion and see it apply to the document.
- 6) Explain why it is important to ask permission or follow instructions before editing someone else's work in a shared Google Docs document.

1.6 End of unit assessment

- 1) Explain what Google Docs is and describe the difference between saving a file to a flash drive versus saving it to the cloud.
- 2) List four features of google docs
- 3) Match the following interface parts with their correct description:

a) Menu Bar	i. The large white area where content is typed
b) Shortcut toolbar	ii. The strip containing icons for Bold, Italics, and Alignment.
c) Workspace	iii. The area containing File, Edit, View, and Insert menu.

- 4) a) Why is it important to log out of your Google account when using a shared school computer?
- b) How does Auto-save encourage a sense of responsibility in your work?
- 5) When using the Search the web feature to add images to your document, why did the unit content remind you to respect copyright?
- 6) Open a new Google Doc and perform the following:
 - a) Rename the document as Unit1_Assessment_Test.
 - b) Insert a Header with your Full Name and the Date.

- c) Type a title: The Importance of Technology. Center align it and change the font color to Blue.
- 7) Create a new document and type the sentence: Click here to visit the school website.
 - a) Turn the word "here" into a Hyperlink leading to www.google.com.
 - b) Insert Page numbers at the bottom right of the page.

UNIT 2: DIGITAL LOGIC FUNDAMENTAL



Key unit competence: Design an optimized digital logic circuit implementing Boolean logic

Learning objectives:

At the end of this unit, you will be able to:

Knowledge and understanding:

- Explain the relationship between decimal, binary, octal, and hexadecimal systems.
- Describe Boolean logic and its fundamental operators (AND, OR, NOT).
- Explain how logic gates are interconnected to form logic circuits.
- Discuss the process of simplifying Boolean expressions.

Skills:

- Convert numbers from one system to another
- Represent digital data using appropriate number systems.
- Construct Boolean expressions from logical statements or circuit diagrams.
- Simulate logic circuits using appropriate software tools.

- Generate truth tables to verify Boolean expressions.
- Apply optimization techniques to simplify logic circuits.

Attitudes and values:

- Appreciate the importance of number systems and logic in computer operations.
- Demonstrate accuracy and precision in performing conversions and simplifications.
- Show creativity in experimenting with logic simulation tools.

2.0 Introduction:

The purpose of this unit is to help you understand the basic concepts of number systems and digital logic used in computing. It enables you to identify different types of number systems such as decimal, binary, octal, and hexadecimal, and perform conversions between them. You will also understand Boolean algebra and its operators and how they are used in logic gates. The unit further introduces you to logic circuit design, including the use of simulation tools and interconnection of logic gates. It also covers constructing logical expressions using Boolean expressions and truth tables, as well as optimizing logic circuits through Boolean rules and simplification, enabling you to develop problem-solving and analytical skills in digital systems.

Introductory activity:

A student types the number 25 on a computer keyboard. The computer does not understand numbers the same way humans do. Instead, it processes information using only 0 and 1.

- 1) Do computers use the same number system that humans use? Explain.
- 2) Do computers use only 0 and 1 to represent information?
- 3) Which number systems that computers might use besides decimal?

2.1 Number systems

Learning activity 2.1

1) Digital computers use a number system with a base of two, rather than base ten to represent any data. This is because it is much easier to engineer circuits that implement “binary number system.”

- a) Discuss the four types of number systems and classify them according to symbols used to represent any number.
 - b) Convert 15 and 20 in binary system.
- 2) Answer with True or False

- a) Binary numbers use digits from 0 to 9.
- b) Remainders in decimal to binary conversion are read from bottom to top.
- c) Hexadecimal digits include letters A to F.
- d) Binary to octal conversion uses groups of 4 bits.
- e) Adding zeros in binary grouping helps complete the groups correctly.

3) Which of the following is the octal equivalent of 111001

- a) 61 b) 71 c) 51 d) 41

Number systems are determined by the base representing valid digits used to represent a number. The four types of number systems used in computing are decimal (base 10), binary (base 2), octal (base 8), and hexadecimal (base 16) number systems.

2.1.1 Types of number systems

a) Decimal number system

Decimal number system consists of ten digits 0-9 most of us are familiar with. Because the decimal number system has ten digits, it is also known as a base 10 or denary number system. In computing, counting of decimal numbers start from 0.

Significance of decimal digits

Significance of a digit refers to its weight that is determined by its absolute and place value. In a decimal number system, the most significant digit (MSD) is the leftmost, while the least significant digit (LSD) is the rightmost digit. For a

number like 7085, the following table shows that 5 is the least significant having a place of 5 while 7 is the most significant with place value of 7000.

Place value	$10^3 = 1000$	$10^2 = 100$	$10^1 = 10$	$10^0 = 1$
Decimal digit	7	0	8	5
Weight	7000	0	80	5

Table 2. 1. LSD and MSD in decimal numbers

b) Binary number system

Binary numbers consist of two digits – 0 and 1 referred to as binary digits, in short' bits. In binary base system, the positional value of a number increases by powers of two. When dealing with different number systems, always remember to indicate the base of a binary number such as 1011_2 .

Significance of binary digits

The most significant digit in a binary number is the leftmost digit, while the least significant digit is the rightmost digit. For example, the following table shows that in binary number like 1011_2 the LSD on the right has weight of 1 that is (1×2^0) , while the MSD has a weight of 8.

Place value	$2^3 = 8$	$2^2 = 4$	$2^1 = 2$	$2^0 = 1$
Binary digit	1	0	1	1
decimal value	8	0	2	1

Table 2. 2. LSD and MSD in binary numbers

NB: The total weight of the binary number 1011_2 represents 11 in decimal numbers obtained by adding: $8+0+2+1 = 11_{10}$

c) Octal number system

The octal number system has eight digits ranging from 0 – 7 that are used to represent any number. This means that a number like 785 cannot be a valid octal number because 8 is between 7 and 5 is not within 0 to 7 digits.

Significance of octal digits

In octal number system, the MSD is the leftmost digit, while LSD is on the right. For example, the following table illustrates an octal number 7245_8 with 7 being the most significant digit with decimal weight of 3584_{10} .

Place values	$8^3=512$	$8^2=64$	$8^1=8$	$8^0=1$
Octal digit	7	2	4	5
Base 10 value	3584	128	32	5

Table 2. 3. LSD and MSD in octal numbers

To get the decimal number equivalent to 7245 we add: $3584 + 128 + 32 + 5 = 3749$ Thus; $7245_8 = 3749_{10}$.

d) Hexadecimal number system

Hexadecimal is a base 16 number system consisting of 16 digits that range from 0 to 9, and A to F. The letters A to F are used to represent numbers 10 to 15 as shown in the following table. Always remember to indicate the base of a hexadecimal number using the subscript 16 e.g. $4F9_{16}$.

Base 10	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Base 16	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F

Table 2. 4. Hexadecimal digits

Significance of hexadecimal digits

In hexadecimal number system, significance of digits increases from right to left in multiples of 16. For example, the following table shows in 946_{16} , 6 is the LSD while 9 is the MSD with decimal place value of 2304_{10} .

The decimal equivalent of 946_{16} is obtained by adding: $2304 + 64 + 6 = 2374$

Thus $946_{16} = 2374_{10}$

Place value	$16^2=256$	$16^1=16$	$16^0=1$
Hexadecimal Digit	9	4	6
Base 10 value	2304	64	6

Table 2. 5. Significance of hexadecimal numbers

The table below shows a summary of the four number systems classified according to their base values:

System	Base	Valid digits	Example
Binary	2	01	1001_2
Octal	8	01234567	5640_8
Decimal	10	0123456789	5640_{10}
Hexadecimal	16	0123456789 ABCDEF	$56AF_{16}$

Table 2. 6. Summary of number systems

2.1.2 Numbers conversion

a) Converting decimal to other base Systems

Mathematically, it is possible to convert a number from one base system to another. In the following section, we demonstrate how to convert decimal numbers to other base systems..

- **Decimal to binary number conversion**

To convert a decimal number to binary, there are two possible methods, the division-remainder, and positional-value methods.

Division-by-base method

In division-by-base method, a decimal number is repeatedly divided by the base until the dividend is indivisible by 2. In every division, write down the remainder on the right of the dividend. Read the sequence of 0s and 1s bottom-ups that represents the binary number. For example, to convert 45_{10} to binary form, proceed as follow:

$$\begin{array}{r|l}
 2 & 45 \\
 \hline
 2 & 22 \text{ R } 1 \\
 2 & 11 \text{ R } 0 \\
 2 & 5 \text{ R } 1 \\
 2 & 2 \text{ R } 1 \\
 2 & 1 \text{ R } 0 \\
 2 & 0 \text{ R } 1^*
 \end{array}
 \quad \text{Thus: } 45_{10} = 101101_2$$

Explanation

Divide 45 by 2. We get 22 remainder 1.

Next divide 22 by 2. We get 11 remainder 0.

Continue dividing until the number is indivisible by 2. In this case, 1 is not divisible hence we write 0 remainder 1. Read the remainder digits as 0s and 1s bottom up.

NB: The remainder in the last division marked with asterisk is 1 because 1 is not perfectly divisible by 2 in the previous step.

The following example demonstrates how to convert 107_{10} to binary form:

$$\begin{array}{r|l}
 2 & 107 \\
 \hline
 2 & 53 \text{ R } 1 \\
 2 & 26 \text{ R } 1 \\
 2 & 13 \text{ R } 0 \\
 2 & 6 \text{ R } 1 \\
 2 & 3 \text{ R } 0
 \end{array}$$

$$107_{10} = 1101011_2$$

Explanation

1. Divide 107 by 2. We get 53 remainder 1.

2. Continue dividing until the quotient is not

perfectly divisible by 2.

2	1	R 1
	0	R 1

3. Read the remainders upwards.

- **Decimal to octal conversion**

To convert a decimal number to octal form, we repeatedly divide the dividend by the base value 8 until the quotient is indivisible by 8. The remainders consisting of digits between 0 and 7 are read upwards. For example, to convert 586_{10} to an octal number, proceed as follows:

8	586		
8	73	R 2	$\rightarrow (586 \div 8 = 73 \text{ rem } 2)$
8	9	R 1	$\rightarrow (73 \div 8 = 9 \text{ rem } 1)$
8	1	R 1	$\rightarrow (9 \div 8 = 1 \text{ rem } 1)$
8	0	R 1	$\rightarrow (1 \div 8 = 0 \text{ rem } 1)$

Thus: $586_{10} = 1112_8$

- **Decimal to hexadecimal conversion**

To convert a decimal number to hexadecimal form, repeatedly divide the quotient by 16 until the quotient is not divisible by the base value. The resulting remainders consisting of digits from 0-9, and A-F are read bottom-up. For example, to convert a decimal number 896 to hexadecimal form, proceed as follows: Continue dividing until the quotient is no longer divisible by 16.

Read the remainders from bottom to top.

Thus: $896_{10} = 380_{16}$

16	896		
16	56	R 0	$\rightarrow (896 \div 16 = 56 \text{ rem } 0)$
16	3	R 8	$\rightarrow (56 \div 16 = 3 \text{ rem } 8)$
	0	R 3	$\rightarrow (3 \div 16 = 0 \text{ rem } 3)$

Thus: $896_{10} = 380_{16}$

Explanation

Divide the number by 16 and write down the quotient and the remainder. Note the remainder can be a digit between 0 and F.

b) Binary to other base system conversion

Conversion of a binary number to other base systems is the reverse procedure to what we have covered in the previous section. In this section, we demonstrate how to convert binary numbers into decimal (base 10), octal (base 8) and hexadecimal (base 16) form.

- **Binary to decimal conversion**

To convert a binary number to decimal form, proceed as follows:

Write place values under which you place the bits from the least significant to the most significant. Binary to decimal conversion is done by assigning place values to each binary digit starting from the right side, where the rightmost bit has the value of 2^0 and each position to the left increases by a power of 2. Each binary digit is then multiplied by its corresponding place value, beginning with the leftmost digit which has the highest power. After multiplying all the digits, the results are added together to obtain the equivalent decimal number.

For example, in case of 101101, multiply the left most bit by 32.

Sum the partial products to get the decimal number. In our case we add $(1 \times 2^5) + (0 \times 2^4) + (1 \times 2^3) + (1 \times 2^2) + (0 \times 2^1) + (1 \times 2^0)$

This gives us:

$$32+0+8+4+0+1=45$$

Therefore, $101101_2 = 45_{10}$

Place value	2^5	2^4	2^3	2^2	2^1	2^0
Binary digits	1	0	1	1	0	1

Table 2. 7. Binary to decimal conversion

- **Binary to octal conversion**

To convert a binary to Octal system group the One's (1's) and zero's(0's) into sets of three bits starting from right to left. The reason for grouping into 3 bits is because the maximum octal digit (7) has a maximum of 3 digits as shown in following table:

Bits	000	001	010	011	100	101	110	111
Octal	0	1	2	3	4	5	6	7

Table 2. 8. Binary representation of octal digit

For example, to convert 11010001_2 to octal format, proceed as follows

Group the bits to sets of 3 starting from right.

Write down the octal digit represented by each set of bits as shown in following table:

Binary digits	011	010	001
Octal digits	3	2	1

Table 2. 9. Binary to Octal conversion

Thus: $011010001_2 = 321_8$

• Binary to hexadecimal conversion

Similar to the approach used with octal number system, a binary number can be converted to hexadecimal format by grouping the bits to a set of 4 bits. This is because the largest hexadecimal digit has 4 bits as shown in the following table:

Hexadecimal digit	Decimal	4-bits
00	00	0000
01	01	0001
02	02	0010
03	03	0011

04	04	0100
05	05	0101
06	06	0110
07	07	0111
08	08	1000
09	09	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

Table 2. 10. Binary representation of hexadecimal digits

For example, to convert 11010001_2 to hexadecimal form, group the bits into sets of 4 starting from right to left as follows: as shown in the following table:

Binary	1101	0001
Hexadecimal	D	1

Table 2. 11. Binary to hexadecimal conversion

Thus: $= 1101\ 0001_2 = D1$

If a binary number does not have an exact set of 4 bits after grouping such as 1100100001 , proceed as follows:

- Split the number into sets of 4 bits starting from right to left. In our case, we get three complete sets and one incomplete one:
- 11 0010 0001

- Because the leftmost set has two bits, add two zeros to it on the left to get: 0011 0010 0001
- Using the binary equivalents in following table, place each the equivalent hexadecimal digit under each of the set of bits.

Binary digits	0011	0010	0001
Hexadecimal	3	2	1

Table 2. 12. Hexadecimal digits

Thus: $0011010001_2 = 321_{16}$

Application activity 2.1

- What is the base used when converting decimal numbers to binary?
a) 8 b) 10 c) 2 d) 16
- When converting a decimal number to binary, the number is repeatedly divided by:
a) 10 b) 2 c) 8 d) 16
- Which of the following is the binary equivalent of 18_{10} ?
a) 10010 b) 10100 c) 11000 d) 10001
- What is the hexadecimal equivalent of 255_{10} ?
a) EE b) FF c) AF d) F0
- When converting binary to octal, digits are grouped in:
a) 2 bits b) 3 bits c) 4 bits d) 5 bits
- When converting binary to hexadecimal, digits are grouped in:
a) 2 bits b) 3 bits c) 4 bits d) 8 bits
- What is the decimal equivalent of 1010_2 ?
a) 8 b) 10 c) 12 d) 14
- Identify the base and valid digits used in each of the following number systems:
a) Binary b) Octal c) Decimal d) Hexadecimal
- State whether the following numbers are valid or invalid in the given number systems. Give reasons:
a) 1011_2 b) 785_8 c) 29_{10} d) $4G_{16}$
- Determine the Most Significant Digit (MSD) and Least Significant Digit (LSD) in the following numbers:
a) 5083_{10} b) 1101_2 c) 7245_8 d) $9A3_{16}$

11) Using the place value method, convert the following to binary number equivalent to:

(i) 145_{10} (ii) 1280_{10} (iii) 5204_{10}

12) Convert the following binary numbers to decimal form:

(a) 0101_2 (b) 1111_2 (c) 10101101110_2

2.2 Boolean algebra and logic gates

Learning activity 2.2

- 1) You pass the subject if you do coursework AND the final exam, can you pass if one is missing?
- 2) Explain the difference between the AND and OR operators. If you were designing a safety system for a machine that requires two different buttons to be pressed simultaneously to start, which operator would you use and why?
- 3) The text mentions that the NOT operator reverses the condition. Describe a real-world digital scenario where reversing a signal would be necessary

Boolean logic is a subset of algebra used for creating true/false statements in computing and search queries, based on binary variables (1/0 or true/false). Its core operators AND, OR, NOT define logical relationships: AND requires all conditions to be true, OR requires at least one, and NOT reverses the condition.

2.2.1 Boolean logic and its operators

Boolean logic is a form of algebra used to represent logical relationships using binary values. In Boolean logic, variables can have only two possible values: 1 (True) and 0 (False). Boolean logic is widely used in digital electronics and computer systems to design and analyze logic circuits. Boolean logic uses logical operators to combine or modify binary variables. The three basic operators are AND, OR and NOT.

a) AND operator

The AND operator produces an output of 1 only when all inputs are 1.
If any input is 0, the output becomes 0.

Symbol: $A \cdot B$, sometimes written as A AND B. The AND operation checks whether both conditions are true. A computer system allows access only if the username is correct AND the password is correct.

Truth table

A	B	$A \cdot B$
0	0	0
0	1	0
1	0	0
1	1	1

Table 2. 13.How AND operator works

The output is 1 only when both A and B are 1.

b) OR operator

The OR operator produces an output of 1 (True) when at least one input is 1.
The output becomes 0 only when all inputs are 0. $A + B$, sometimes written as A OR B, the OR operation checks whether at least one condition is true.

Example: A student can enter the computer lab if they have a student ID OR teacher permission.

Truth table

A	B	$A + B$	
0	0	0	
0	1	1	

1	0	1	
1	1	1	

Table 2. 14. How OR operator works

The output is 1 if at least one input is 1.

c) NOT operator

The NOT operator reverses the value of the input.

Symbol: A' , $\bar{A}$, NOT A

If the input is 1, the output becomes 0. If the input is 0, the output becomes 1.

Example: If a system checks whether a door is closed, the NOT operator will represent the condition “door NOT closed”, meaning the opposite state of the door being closed. Here is its truth table:

Truth table

A	A'
0	1
1	0

Table 2. 15. NOT operator

Application activity 2.2

1) A computer grants access to a system only if the user enters the correct password (A) and the user enters the correct username (B). Expression: $Y = A \cdot B$

What will be the output Y for each input?

A (Password correct)	B (Username correct)	Output Y
----------------------	----------------------	----------

0	0	?
0	1	?
1	0	?
1	1	?

2) A student can borrow a laptop from the lab if: They have a student ID (A) OR they have permission from the teacher (B). This can be written in the expression as $Y = A + B$

What will be the output Y for each input?

A (Student ID)	B (Teacher permission)	Output Y
0	0	?
0	1	?
1	0	?
1	1	?

3.) A security system activates when the door is NOT closed. This can be written in expression form as $Y = A'$

What will be the output Y for each input?

A (Door closed)	Output Y
1	?
0	?

2.3 Logic circuit design

Learning activity 2.3

- 1) Why are logic gates interconnected when designing digital circuits?
- 2) What advantages do logic simulation tools provide before building real circuits?
- 3) Give two examples of devices that use interconnected logic gates.
- 4) Which gate produces output 1 only when all inputs are 1?
a) OR b) AND c) NOT d) XOR

Logic circuit design is a part of digital electronics where circuits are created using logic gates to perform specific logical operations on binary inputs (0 and 1). These circuits are the foundation of computers, microcontrollers, and digital systems.

Key goals of logic circuit design are: Process binary data, perform operations like AND, OR, NOT, implement decision-making in hardware, build components such as adders, multiplexers, and registers.

Basic logic gates used:

Logic gates	Outputs
AND	1 only if all inputs are 1
OR	1 if at least one input is 1
NOT	Inverts the input
NAND	Derived or universal gates
NOR	Derived or universal gates
XOR	Derived or universal gates
XNOR	Derived or universal gates

Table 2. 16. Basic logic gates

2.3.1. Logic simulation tools

Logic simulation tools are software programs used to design, test, and verify digital circuits before building them physically. They allow engineers and

students to place logic gates, connect wires, and simulate outputs. Common logic simulation tools are Logisim and Proteus

a) Logisim

Logisim is a free, open-source tool specifically designed for education. It is used to teach the basic concepts of digital logic and how computer CPUs are designed. Logisim has a modern version which is called Logisim-evolution, and it includes improved features, better performance, and additional components that make it more suitable for designing and simulating complex digital circuits in computer science education. Logisim or Logisim-evolution is better for learning basic digital logic and computer architecture.

Key features:

- **Simplicity:** It uses a clean, "drag-and-drop" interface for gates (AND, OR, NOT).
- **Real-time interaction:** You can click on input switches while the simulation is running to see the outputs change instantly.
- **Color-coded wires:** Wires change color based on their state (e.g., bright green for 1, dark green for 0, and blue for an unconnected wire).
- **Hierarchical design:** You can create "Sub-circuits" (like a 1-bit adder) and then use them as a single block in a larger circuit (like an 8-bit adder).
- **Best for students learning the fundamentals of Boolean logic and those building virtual 8-bit or 16-bit computer processors.**

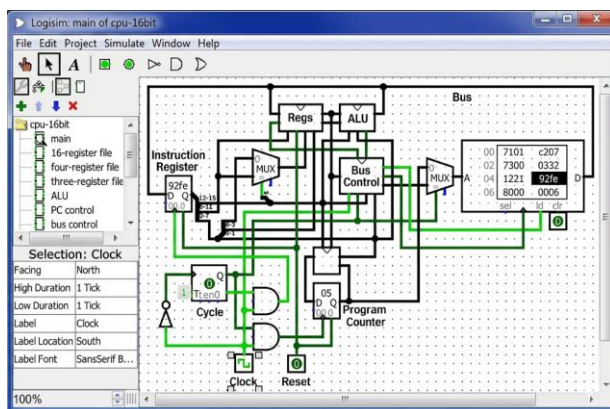


Figure 2. 1. Logisim interface- main of CPU

b) Proteus

Proteus design suite is a professional-grade software used for electronic design automation. It is much more powerful than Logisim because it simulates real-world electronic physics. Proteus is more powerful for real electronics and engineering applications.

Key features:

- **Mixed-mode simulation:** It can simulate both digital logic gates and analog components (resistors, capacitors, transistors).
- **Microcontroller simulation:** It allows you to upload code including C++ or Assembly to virtual microprocessors like Arduino or PIC to see how they interact with logic gates.
- **PCB layout:** Once the circuit works in the simulation, Proteus can help you design the physical Printed circuit board known as PCB to be manufactured.
- **Virtual instruments:** Includes virtual versions of real laboratory tools like oscilloscopes and voltmeters to measure signals.

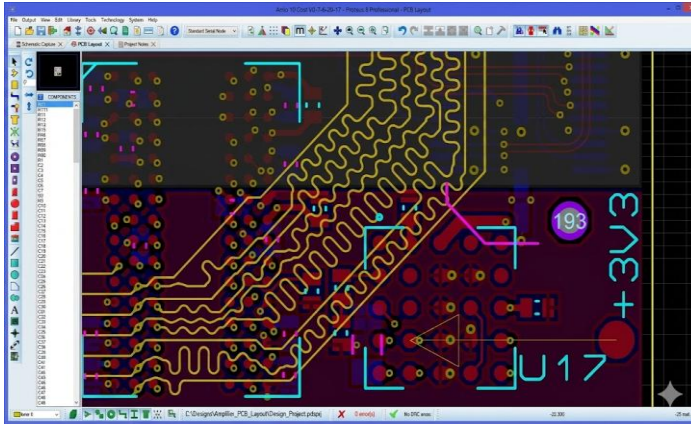


Figure 2. 2. Proteus (professional & hardware focus)

Benefits of simulation tools are: test circuits without hardware, detect errors early, visualize signal changes and save time and cost

2.3.2 Interconnecting logic gates

Interconnecting logic gates means connecting the output of one logic gate to the input of another logic gate in order to build a more complex digital circuit. Instead of using a single gate, several gates are combined to perform more complicated logical operations.

In digital electronics and computer systems, most circuits such as adders, comparators, and memory units are built by linking multiple logic gates together.

- **Basic idea of interconnection**

Each logic gate has inputs signals entering the gate usually 0 or 1 and output the result after the logic operation. When the output of the first gate becomes the input of the second gate, the gates are said to be interconnected.

- **Why logic gates are interconnected**

Logic gates are interconnected to: perform complex operations that one gate cannot do alone, build digital systems such as adders, multiplexers, memory circuits, CPUs.

Interconnecting logic gates means linking outputs of some gates to inputs of others, it allows the creation of complex digital circuits and common combinations include: AND → NOT, AND → OR, OR → NOT

These connections form the basis of modern computer system.

Example 1: AND gate connected to NOT gate

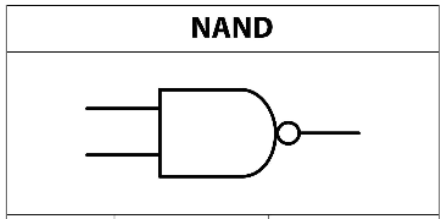


Figure 2. 3. AND gate connected to NOT gate

In this circuit: Inputs A and B enter an AND gate, the output of the AND gate goes into a NOT gate, The NOT gate inverts the result.

$$Y = \overline{A \cdot B}$$

Logical expression:

This circuit actually behaves like a NAND gate.

A	B	AND	NOT (Output Y)
0	0	0	1
0	1	0	1
1	0	0	1
1	1	1	0

Table 2. 17. NAND truth table

Example 2: AND gate connected to OR gate

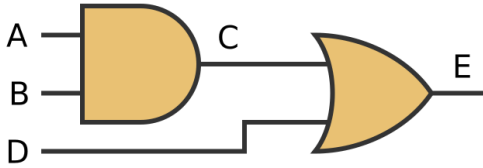


Figure 2. 4. AND gate connected to NOT gate

In this circuit: Inputs A and B enter an AND gate, the output of the AND gate becomes one input of an OR gate and another input C also enters the OR gate

Logical expression: $Y = (A \cdot B) + D$

A	B	C	$A \cdot B$	Output Y
0	0	0	0	0
0	1	0	0	0
1	1	0	1	1
0	0	1	0	1

Table 2. 18. Truth values

- **Control decision-making in computers:** For example, calculators, computers, and smartphones all contain millions or billions of interconnected logic gates.
- **Simple real-life example:** Consider a security alarm system: The alarm rings if the door is open AND the alarm is activated OR the window is broken. This requires interconnected AND and OR gates.

Application activity 2.3

1) Why is a NAND gate called a universal gate?

2) Consider two inputs A and B connected by an AND gate and the output goes into a NOT gate. The corresponding expression is $Y = (A \cdot B)'$.

Complete the following truth table:

A	B	$A \cdot B$	$Y = \text{NOT}(A \cdot B)$
0	0	?	?
0	1	?	?
1	0	?	?
1	1	?	?

3) Create truth table to the following expressions: $A \cdot B \cdot C$

4) Create truth table of the expressions $(A \cdot B) + (C \cdot D)$

2.4 Constructing logical expressions

Learning activity 2.4

A. Choose the correct answer

- Which operator gives an output of 1 only when both inputs are 1?
a) OR, b) NOT, c) AND, d) XOR
- The Boolean expression $Y = A + B$ represents which gate?
a) AND gate, b) OR gate, c) NOT gate, d) NAND gate
- What does the NOT operator do?
a) Adds values, b) Multiplies values, c) Reverses the input value, d) Combines inputs
- Truth table is used to:
a) Store data, b) Display hardware components,

c) Design a database
outputs

d) Show all possible input combinations and

B. Answer with True or False

5) Boolean variables can only have the values 0 or 1.

6) The expression $Y = A \cdot B$ represents an OR operation.

7) Truth tables help verify Boolean expressions.

8) The OR gate outputs 0 when at least one input is 1.

9) Boolean expressions are used to describe how logic circuits work.

In digital electronics and computing, logical expressions are used to represent how logic gates process inputs to produce an output.

These expressions are written using Boolean algebra, where variables can only have two values: 1 representing True or ON, 0 representing False or OFF. Logical expressions help describe how inputs are combined using logic operations such as AND, OR, and NOT.

2.4.1. Boolean expressions

A Boolean expression is a mathematical expression made up of boolean variables (A, B, C, etc.), logic operators (AND, OR, NOT) and parentheses (to control order).

Common Boolean operators

Operation	Symbol	Meaning	Example
AND	$\cdot$ or AB	True only if both inputs are true	$A \cdot B$
OR	+	True if at least one input is true	$A + B$
NOT	$\neg$ or '	Reverses the value	A'

Table 2. 19. Common Boolean operators

2.4.2. Truth table

A truth table is a table that shows all possible combinations of input values and the corresponding output of a logic expression or logic circuit. Truth tables help us to understand how a logic circuit works, verify Boolean expressions and design digital circuits.

Truth table for a combined expression $Y=(A \cdot B)+C$

A	B	C	$A \cdot B$	Y
0	0	0	0	0
0	0	1	0	1
0	1	0	0	0
0	1	1	0	1
1	0	0	0	0
1	0	1	0	1
1	1	0	1	1
1	1	1	1	1

Table 2. 20. Truth table for a combined expression

Explanation: First calculate $A \cdot B$, then apply OR with C

Application activity 2.4

1) A designer has proposed the expression $Y = A + (B \cdot C)$. To verify how the circuit will behave, complete the missing values in the truth table below.

A	B	C	$Y=A+(B \cdot C)$
0	0	0	?
0	0	1	?

0	1	0	?
0	1	1	?
1	0	0	?
1	0	1	?
1	1	0	?
1	1	1	?

2) If $A = 1$ and $B = 0$, determine the value of C where $C = A + B$.

3) A designer has proposed the expression $Y = A(B+C)$. To verify how the circuit will behave, create a truth table for the expression.

2.5 Logic circuits optimization

Learning activity 2.5

You are an engineer at a tech company in Kigali. You have two designs for a phone's power-saving circuit: design A: Uses 5 logic gates, design B: Uses 1 logic gate. Both designs produce the exact same output.

- 1) List three reasons why your manager would choose design B over design A.
- 2) How does using fewer gates affect the battery life and heat produced by the phone?

Logic circuit optimization means is reducing the complexity of a logic circuit without changing its output. The goal is to make circuits simpler, faster, and cheaper, use fewer logic gates. Optimization is usually done by applying Boolean rules to simplify Boolean expressions before building the circuit.

2.5.1. Boolean rules

Boolean rules are mathematical laws used to simplify logical expressions. They help reduce the number of variables and operations in a circuit.

Below are the most important Boolean rules:

Boolean Rules	Expression	Explanation	Expanded form
Identity	$A + 0 = A$	OR with 0 does not change the value	$Y = A + 0$,
	$A \cdot 1 = A$	AND with 1 does not change the value	Simplified: $Y = A$
Null	$A + 1 = 1$	OR with 1 always gives 1	$Y = A \cdot 0$,
	$A \cdot 0 = 0$	AND with 0 always gives 0	Simplified: $Y = 0$
Idempotent	$A + A = A$	OR of the same variable	$Y = A + A$,
	$A \cdot A = A$	AND of the same variable	Simplified: $Y = A$
Complement	$A + A' = 1$	Variable OR its complement	$Y = A \cdot A'$,
	$A \cdot A' = 0$	Variable AND its complement	Simplified: $Y = 0$
Commutative	$A + B = B + A$	Order does not matter.	$A + B = B + A$
	$A \cdot B = B \cdot A$	Order does not matter.	-
Associative	$(A + B) + C = A + (B + C)$	Grouping does not change the result.	-
	$(A \cdot B) \cdot C = A \cdot (B \cdot C)$	Grouping does not change the result.	

De Morgan's theorems	$(A \cdot B)' = A' + B'$	NOT changes AND to OR	-
	$(A + B)' = A' \cdot B'$	NOT changes OR to AND	-
Distributive	$A(B + C) = AB + AC$	$A(B+C)$, Expanded: $AB+AC$	$A(B+C)$, Expanded: $AB+AC$
	$A + BC = (A + B)(A + C)$		-

Table 2. 21. Boolean rules.

2.5.2. Simplification of boolean expressions

Simplification means reducing a Boolean expression to its simplest form using Boolean rules. Simplification leads to lower logic gates, simpler circuits, and lower cost hardware

Example 1: Simplification of the expression $Y = A(B + B')$

- Step 1: Apply Complement Law, $B + B' = 1$
- Step 2: $Y = A \cdot 1$
- Step 3: Apply identity law, $Y = A$

Final simplified expression: $Y = A$

Example 2: Using idempotent law, $Y = AB + AB$ becomes $Y = AB$

Simplifying logic circuits helps to reduce number of logic gates, reduce power consumption, increase circuit speed, reduce hardware cost, and make circuits easier to design.

Application activity 2.5

1) Analyze the two logic expressions: Circuit A is $Y = (A \cdot B) + (A \cdot B)$ and Circuit B is $Y = A \cdot B$.

a) Which Boolean law proves that Circuit A and circuit B are identical in output?

b) In a simulation tool like Logisim, which circuit would be easier to troubleshoot if there was a connection error? Why?

2) Solve the expression $Y = A + A' \cdot B$

3) Simplify Expression: $Y = (A + A) \cdot B$

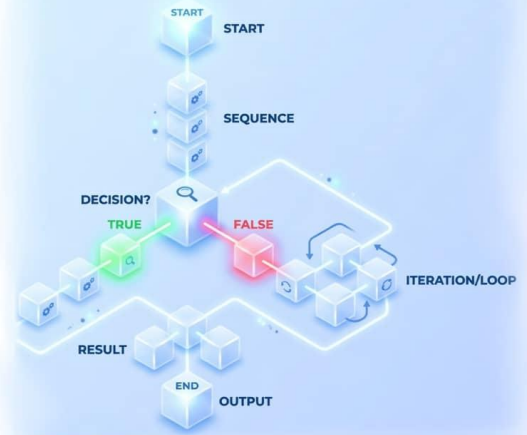
2.6 End of unit assessment

- 1) Explain why digital computers use the binary number system rather than the decimal system that humans prefer.
- 2) Describe the function of the AND, OR, and NOT operators. For each, provide a real-world scenario (e.g., a security system or school rule) where that logic would apply.
- 3) Compare Logisim and Proteus. Which tool is better suited for a student learning CPU design, and which is better for an engineer designing a physical Printed Circuit Board (PCB)?
- 4) Convert 147_{10} to binary using the division-by-base method.
- 5) Convert $56AF_{16}$ to decimal.
- 6) Convert 1002_8 to hexadecimal.
- 7) Generate a complete truth table for the following combined Boolean expression: $Y = (A \cdot B) + C'$
- 8) Draw a logic circuit diagram for the expression $Y = (A + B) \cdot C$. If you were to test this in Logisim, what color would the wire turn if the signal is 1 ?
- 9) In the context of binary-to-hexadecimal conversion, why is it critical to group bits starting from the right rather than the left? What happens to the data if this precision is ignored?
- 10) You are given a logic simulation tool and asked to build a smart home lighting system. Describe how you would use interconnected gates to ensure lights only turn on when it is dark AND someone is in the room.

11) How does learning Boolean simplification help you become a better problem-solver in other areas of Information Technology?

12) Why should an ICT professional value optimization of digital circuit? Consider the impact on a country's electronic waste and energy consumption.

UNIT 3: PROGRAMMING ALGORITHM



Key Unit Competence: Develop algorithmic solutions from problems using pseudo code and flowcharts.

At the end of this unit, learner should be able to:

Knowledge and understanding

- Explain the role, characteristics, and structure of an algorithm
- Explain how data is stored in computer memory
- Describe the importance of algorithms in problem-solving.
- Describe the input and output operations in algorithms.
- Explain the purpose and structure of conditional statements and loop statements.

Skills

- Write an algorithm that solves a given problem
- Use reading and writing functions in algorithm
- Write a Flowchart using symbols
- Convert pseudocode into flowcharts accurately and vice versa

Attitudes and values

- Demonstrate a willingness to follow algorithmic steps when solving computer problems.
- Develop logical thinking and attention to detail when designing algorithms.

- Value accuracy, organization, and clarity in algorithm representation.

3.0 Introduction

The purpose of this unit is to help you understand the basic concepts of algorithms and problem-solving in computing. It enables you to define algorithms, understand their characteristics, and recognize their importance in solving problems. Students will also use variables, data types, and operators, as well as how to handle input and output in programs. The unit further introduces algorithm representation using flowcharts and pseudocode, and covers decision-making and repetition through conditional statements and loops, enabling students to develop logical thinking and problem-solving skills.

Introductory activity

MUKASINE want to make a cup of tea at home. To prepare it, she must follow the steps in the correct order as follows: (1) boil water, (2) put tea leaves or a tea bag in a cup, (3) pour the hot water into the cup, (4) add sugar or milk if desired, (5) stir and drink.

- 1) What do you understand by step-by-step instructions when solving a problem?
- 2) Can you give another daily life example where steps must be followed in order?
- 3) Why is it important to follow the correct order of steps?

3.1 Introduction to algorithms

Learning activity 3.1

- 1) What do you understand by algorithm?
- 2) Why are algorithms important in computer programming?
- 3) Give two examples of algorithms used in daily life.
- 4) Explain why computers need algorithms in order to perform tasks.
- 5) Identify the three basic parts of a simple algorithm.

3.1.1 Definition of algorithms

An algorithm is a step-by-step set of instructions used to solve a problem or perform a task. It describes the exact sequence of actions that must be followed to obtain a desired result.

In computing, algorithms are very important because computers cannot think by themselves. They must follow clear and precise instructions written by programmers. An algorithm usually has a starting point, several processing steps, and an ending point.

This is an example of a simple algorithm that shows how a problem can be solved by following organized steps.

1. Start
2. Input the first number
3. Input the second number
4. Add the two numbers
5. Display the result
6. End

Real-life examples of algorithms

- Steps for cooking rice
- Instructions for using an ATM machine

3.1.2 Characteristics of algorithm

- A good algorithm must have certain important characteristics to work correctly.
- **Input:** An algorithm must receive zero or more inputs that are used to solve the problem. Example: numbers entered by a user to calculate their sum.
- **Output:** An algorithm must produce at least one output, which is the result after processing the inputs. Example: the sum or average of numbers.
- **Definiteness:** Each step of the algorithm must be clear, precise, and unambiguous so that it can be understood easily.

Example: instead of saying “calculate something”, the algorithm should say “add number1 and number2”.

- **Effectiveness:** Each step in the algorithm must be simple and possible to perform within a reasonable amount of time.
- **Correctness:** The algorithm must produce correct results for the problem it is designed to solve
- **Efficiency:** Good design results in an efficient program that consumes less processor time and memory space.
- **Simplicity:** Though a program may be complex, its simplicity is one of the most important factors that influence its user-friendliness and ease of maintenance.
- **Documentation:** It is good practice to provide documentation containing details of a program algorithms.

3.1.3 Importance of algorithms

Algorithms play a very important role in computer science and problem-solving. They provide a systematic approach to solving problems and help in developing effective and efficient solutions. The following are some of the importance of algorithms:

- **Provide clear steps to solve problems:** Algorithms break down complex problems into smaller and manageable steps, making them easier to understand and solve.
- **Help in program development:** Before writing a computer program, programmers usually design an algorithm first. This helps them organize the logic of the program.
- **Improve efficiency:** Algorithms help find the most efficient way to solve a problem, saving time and computing resources.
- **Facilitate communication:** Algorithms allow programmers and system designers to communicate solutions clearly before implementing them in a programming language.
- **Help detect errors:** By designing an algorithm first, it becomes easier to identify mistakes or missing steps before coding.
- **Used in many fields:** Algorithms are used in many areas such as Computer programming, Banking systems, Navigation systems (GPS), Artificial Intelligence, Search engines and Online shopping systems.

Application activity 3.1

1) You are designing a simple program for a school library system that calculates a student's fine for returning a book late. The system must follow these rule: If a book is returned late, the student pays 500 RWF per day.

Write an algorithm which will calculate the student's fine according to number of late days.

2) With examples, differentiate input and output in computer programming.

3) Write an algorithm for calculating the product of two numbers.

3.2 Variables and operations

Learning activity 3.2

A student is developing a simple computer program that calculates the total and average marks of three subjects. Before writing the program, the student must identify the variables, data types, and operators that will be used in the algorithm.

- 1) Define a variable in programming.
- 2) Why are variables important in algorithms and computer programs?
- 3) In the example $\text{age} = 18$, identify:

- a) The variable name
- b) The value stored in the variable

3.2.1 Definition of variable

A variable is a named storage location in a computer's memory that is used to store data that can change during program execution. Variables allow programmers to store values such as numbers, text, or logical results so that the program can use them in calculations and other operations. Each variable has a name, a data type and value.

Example: If a program needs to store the age of a student, a variable $\text{age} = 15$ can be created. Here age is the variable name while 15 is the value stored in the variable.

In a mathematical problem of calculating area of a circle, radius can take any value as shown in the following table. Therefore, radius is an input variable while area is an output variable.

No	Input: radius	Area
1	2	12.568
2	5	78.55
3	10	314.20
4	15	706.95
5	20	1256.80

Table 3. 1. Use of radius as a variable to calculate the area

If this problem is solved using a computer program, radius and area variables represents memory locations reserved to hold values that change during program execution as shown in the table.

Example : Write an algorithm to store and display a student's age:

1. Start
2. Declare variable age
3. Set age = 18
4. Display age
5. End
6. Output: 18

a) Rules of naming variables

The name given to variable is matter of choice by a programmer subject to the following rules:

- Choose meaningful variable names that tell the reader of the program what the variable represents. For example, use sum instead of just s.
- Each variable in the same algorithm should be identified using a unique name. For example, you cannot use balls represent input, and balls to represent output

- By convention, variable names should begin with a letter of the alphabet but may be followed by numbers. For example, use balls3 instead of 3balls.
- Avoid using variable names that may conflict with reserved or keywords used in most programming languages.
- Variable names made up of two or more words should not have space in between the words, instead combine the two words or use an underscore. For example, instead of using Basic Salary as variable name, use BasicSalary or Basic_salary.
- Uppercase characters are distinct from lowercase characters.
- Special characters are not allowed

3.2.2 Data type

A data type specifies the kind of data that a variable can store. Different types of data require different storage formats and operations.

Common data types

Data Type	Description	Example
Integer	Stores whole numbers	5, 20, -3
Float / Real	Stores decimal numbers	3.5, 7.25
Character	Stores a single letter or symbol	'A', 'B'
String	Stores a sequence of characters (text)	"Hello"
Boolean	Stores logical values	True or False

Table 3. 2. Common data types

In logic, the true value is represented by one while false is represented by zero. In addition to primary data types, most programming languages support composite data types. A composite data type such as array, record and linked list is obtained by combining several primary data types.

3.2.3 Declaration

Declaration of variable refers to identify and explicitly state input and output variables required to solve a problem. For example, suppose you are required to solve a problem of finding sum and average of three numbers. To identify and state input and output variable from the problem, proceed as follows:

Express the problem using natural language in order to identify input, processing and output requirements as shown below:

Begin

Accept user input for 3 numbers

Calculate sum - add the 3 numbers

Calculate average - divide sum by 3

Display the results sum and average

End

Identify a statement or statements that indicate input is required. In the above algorithm, input is implied in the statement Accept user input for 3 numbers. The statement implies that the user is expected to input numbers on the keyboard. Represent the input values as variables using symbolic names such as Num1, Num2, and Num3.

Identify a statement or statements that indicate the algorithm provides output. The algorithm indicates output using a statement Display the results. Deeper look at the algorithm points the result to calculated sum as average. Represent the output values as variables using symbolic names such as Sum, Average.

Rewrite the algorithm to indicate the input and output variables as shown below:
Begin

Input: Num1, Num2, Num3,

Output: Sum, Average

PRINT Enter three numbers on the keyboard

READ Num1, Num2, Num3

Sum = Num1 + Num2 + Num3

Average = Sum/3

PRINT Sum, Average

End

3.2.4 Operators

Operators are symbols used to perform operations on variables and values. There are different types of operators used in algorithms and programming.

a) Assignment operators

The assignment operators such as (=) or (:=) causes the operand on the left side of the operator to be replaced by the value on the right side.

For example, in the following expression, the value of x is replaced by the sum of a and b. $x = a + b$

b) Arithmetic operators

These operators perform mathematical calculations.

Operator	Meaning	Example
+	Addition	$5 + 3 = 8$
-	Subtraction	$9 - 4 = 5$
*	Multiplication	$4 * 3 = 12$
/	Division	$10 / 2 = 5$
%	Modulus (remainder)	$10 \% 3 = 1$

Table 3. 3. Arithmetic operators

Example: $a = 6$, $b = 3$, $sum = a + b$, Output: $sum = 9$

c) Comparison operators

Relational operators are used to compare two values. The result is either true or false.

Operator	Meaning	Example
==	Equal to	$5 == 5 \rightarrow \text{True}$
!=	Not equal to	$5 != 3 \rightarrow \text{True}$
>	Greater than	$8 > 4 \rightarrow \text{True}$
<	Less than	$2 < 5 \rightarrow \text{True}$
>=	Greater than or equal	$6 >= 6 \rightarrow \text{True}$
<=	Less than or equal	$4 <= 3 \rightarrow \text{False}$

Table 3. 4. Comparison (relational) Operators

Example: $a = 10, b = 5, a > b$

Output: True

d) Logical operators

Logical operators are used to combine multiple conditions.

Operator	Meaning	Example
AND	True if both conditions are true	$(5 > 3 \text{ AND } 7 > 2)$
OR	True if at least one condition is true	$(5 > 8 \text{ OR } 6 > 2)$
NOT	Reverses the condition	NOT(True)

Table 3. 5. Logical Operators

Example: $\text{age} = 20, \text{age} > 18 \text{ AND } \text{age} < 30$, Output: True

e) Increment and decrement operators

These operators are used to increase or decrease a variable value by 1.

Operator	Meaning
++	Increment (increase by 1)
--	Decrement (decrease by 1)

Table 3. 6. Increment and Decrement Operators

Example1: $x = 5$, $x++$, new value: $x = 6$

Application activity 3.2

1) A school system is being developed to calculate the total and average marks of a student in three subjects.

- Identify Input and output variables
- Write an algorithm to calculate the sum and average of three numbers.
- Write an algorithm to calculate area of square.

2) A library management system is being developed to calculate the total number of books borrowed by a student and determine whether the student has exceeded the borrowing limit.

- Identify the input and output variables.
- Write an algorithm to calculate the total number of books borrowed if books are borrowed from three categories: Science, Literature, and History.
- Write an algorithm to determine whether the student has exceeded the borrowing limit of 10 books using a comparison operator.

3.3. Input and output

Learning activity 3.3

- Which statement is used to receive data from the user?

a) WRITE b) DISPLAY c) READ d) SHOW

2) Which statement is used to display information to the user?

a) INPUT b) READ c) WRITE d) GET

3) which of the following is a constant?

a) studentAge b) $PI = 3.14$ c) totalMarks d) number

Answer with True / False

4) A variable must be initialized before it is used.

5) A constant value can change during program execution.

6) READ is used to display output.

7) WRITE is used to display results to the user.

In algorithms and programming, input and output operations are used to allow a program to receive data and display results.

- Input refers to the data entered into a program.
- Output refers to the results produced by the program after processing the data.

For example, when a user enters two numbers in a calculator and the calculator displays the sum, the numbers entered are inputs, and the result displayed is the output.

Input and output operations are important because they allow communication between the user and the computer program.

3.3.1 Initialization

a) Variable initialization

Once a variable is declared it does not have a defined value, hence it cannot be used until it is initialized by assigning it a value. Initializing a variable goes beyond declaration to assign an initial value to a variable. For example, in our previous algorithm, we can initialize variables Num1, Num2, Num3 with initial values as shown below:

Input: Num1= 3, Num2 =5, Num3 =7,

The statement assigns the values to the variables such that if the algorithm is implemented, the initial sum and average before any user input is calculated as:

Sum = 3+ 5 + 7; This returns 15

Average = 15/3; Returns 5

Note that in a real program, if a variable has been declared but not initialized, the memory location contains nothing, hence we say it holds a null until the user enters values to be assigned to the variable.

Swap_two_numbers

BEGIN

SET Variable_A, Temporary, Variable_B

SET Temporary=0;

PRINT "Please enter Variable_A" READ Variable_A;

PRINT "Please enter Variable_B"

READ Variable_B

Temporary = Variable_A;

Variable_A=Variable_B;

Variable_B=Temporary;

PRINT Variable_A, Variable_B;

END.

A variable is a named memory location used to store data that can change during program execution. Initialization is the process of assigning an initial value to a variable before it is used in the program.

Example: Integer number = 10

Explanation: number is the variable, 10 is the initial value assigned to the variable.

An example of an algorithm to store a number:

1. Start
2. Declare variable number
3. Initialize number = 10
4. Display number
5. End

Output: 10

Example with Two Variables

Integer a = 5

Integer b = 8

These variables can later be used in calculations.

b) Constant initialization

A constant is a value that cannot change during program execution. Unlike a variable which is an identifier for values that can change, a constant is a fixed value which cannot be changed.

In mathematics and physics, examples of constants include pi (π), speed of light, and gravity.

For example, referring back to the problem of calculating area of a circle discussed earlier, π is a constant whose value is 3.142. If implemented as a

program, the value of π can never be changed during the program execution. Once a constant is defined, its value remains the same throughout the program.

Example: Constant $PI = 3.14$

Explanation: PI is a constant. Its value cannot be modified later in the program.

Example: Algorithm to calculate the area of a circle:

1. *Start*
2. *Declare constant $PI = 3.14$*
3. *Input radius r*
4. *Compute $area = PI \times r \times r$*
5. *Display area*
6. *End*

If the radius is 5, the output will be: $Area = 3.14 \times 5 \times 5$, $Area = 78.5$

3.3.2 Read and write statement

In algorithms and pseudocode, read and write statements are used to handle input and output operations.

a) **Read statement**

The READ statement is used to receive data from the user. To represent read functions in an algorithm, we use keywords like READ, INPUT, and GET.

For example, the following statements demonstrate how to use read functions to get radius as input from keyword. It allows the program to store user input in variables.

READ radius;

INPUT radius;

GET radius;

Good practice in algorithm design requires the READ functions to be in uppercase while values to be read also known as parameters to be in lowercase.

For clarity, if a function is to read several parameters, parenthesis may be used to enclose the parameters as shown below:

READ (length, width)

INPUT (length, width);

GET (length, width);

b) Write statement

Like in read operations, we use keywords like WRITE, DISPLAY, and SHOW to represent functions that display information on the screen.

For example, the following statements demonstrate how to display area on the screen:

WRITE area;

DISPLAY area;

SHOW area;

For clarity, if a write function is to display several values, parenthesis may be used to enclose the parameters as shown below:

WRITE (area, perimeter)

DISPLAY (area, perimeter);

SHOW (area, perimeter);

The WRITE statement is used to display information or results to the user.

Example: WRITE "Welcome to the program"

Output: Welcome to the program

Example of an algorithm to calculate the sum of two numbers: Using Read and Write together

Start

Declare variables a , b , sum

Read a

Read b

Compute $sum = a + b$

Write "The sum is "

Write sum

End

Execution

Input: $a = 4$, $b = 6$

Output: The sum is 10

Application activity 3.3

1) A small shop system requires a program that reads the price of an item and the quantity purchased, then calculates and displays the total cost.

a) Identify Input and Output

b) Write an algorithm

2) Write an algorithm that performs the following tasks:

a) Read the length of a rectangle

b) Read the width of the rectangle

c) Calculate the area of the rectangle

d) Display the area

3) Develop an algorithm that:

a) Reads two numbers

b) Calculates their quotient

c) Displays both results

3.4 Algorithm representation

Learning activity 3.4

A. Choose the right answer

- 1) A flowchart is used to:
a) Write a program code b) Represent steps of an algorithm visually
c) Store data in a computer d) Replace programming languages
- 2) Which symbol represents input or output?
a) Rectangle b) Oval c) Parallelogram d) Diamond
- 3) Which symbol is used for decision making in a flowchart?
a) Diamond b) Rectangle c) Oval d) Arrow

B. Answer with True / False

- 4) Flowcharts use standardized symbols.
- 5) The rectangle symbol represents input/output operations.
- 6) Flowcharts should start and end with terminal symbols.
- 7) Decision symbols usually have two possible outputs.

3.4.1 Concept of flowchart

A flowchart is a diagrammatic or symbolic representation of step-by-step solution to a given problem. Flowcharts use standard symbols that help programmers visualize input, processing and output operations to be performed by a computer program.

Unlike natural languages and pseudocode, use of standardized symbols makes the flowcharts easier to interpret hence more universally acceptable.

3.4.2 Flowchart symbols

The following table gives a brief description of six standard symbols used to create flowcharts.

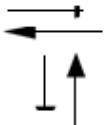
Symbol	Name/Meaning	Symbol	Meaning
	<u>Process</u> – Any type of internal operation: data transformation, data movement, etc.		<u>Connector</u> – connects sections of the flowchart, so that the diagram can maintain a smooth, linear flow.
	<u>Input/output</u> – input or output of data		<u>Terminal</u> – indicates start or end of the program or algorithm.
	<u>Decision</u> - evaluates a condition or statement and branches depending on whether the evaluation is true or false.		<u>Flow lines</u> - arrows that indicate the direction of the progression of the program.

Figure 3. 1.Flowchart symbols

The example shown in figure below shows a flowchart that takes radius as input to calculate and display area of a circle.

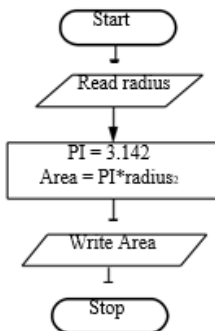


Figure 3. 2. Sample flowchart

The following are general rules that may be followed when expressing an algorithm using flowchart: Be sure to use the right symbol for the right purpose. For examples it is wrong to use a terminal symbol for input.

- All the symbols of a flowchart should be connected using arrows (flow lines) and not plain lines.
- The direction of flow should be from top to bottom, or sides depending on the page layout.
- The start and end of a flowchart must be indicated with (start/stop) terminal symbol.
- Flowchart should have only one entry point at the top and one exit point at the bottom or side.
- The decision symbol should have only two exit points for either true or false on the sides, or bottom and one side.
- If a flowchart does not fit one page or column, use connectors to indicate breaks in the flowchart.

3.4.3 Create basic flowcharts from given algorithms or pseudocode

Creating flowcharts from given algorithms or pseudocode helps you to understand program logic clearly and makes problem-solving easier before writing actual computer programs.

Given algorithm:

Start

Input two numbers A and B

Calculate Sum = A + B

Display Sum

End

Flowchart representation

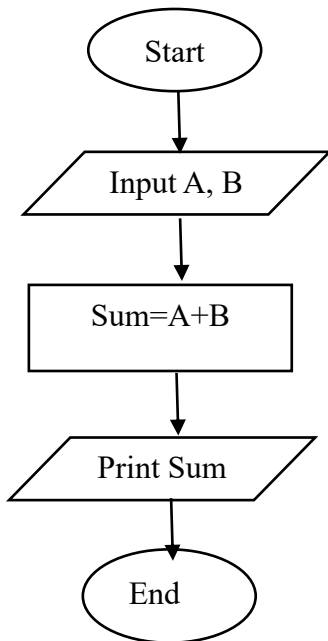


figure 3. 3 Flowchart performing sum

Application activity 3.4

- 1) Write an algorithm and draw a flowchart to calculate the area of a triangle .
- 2) Construct a flowchart that inputs two numbers, calculates their product, and displays the result.

3.5 Conditional statement

Learning activity 3.5

A. Choose the correct answer.

- 1) The IF statement is used to:
 - a) Repeat instructions
 - b) Store data
 - c) Test a condition
 - d) Draw flowcharts
- 2) In an IF...THEN statement, the instructions are executed when:
 - a) The condition is false
 - b) The condition is true
 - c) The program stops
 - d) The program restarts

- 3) Which statement is used when there are two possible outcomes?
- | | |
|---------------|------------------------|
| a) FOR loop | b) IF...ELSE statement |
| c) WHILE loop | d) READ statement |
- 4) In flowcharts, decisions are represented by:
- | | |
|--------------|------------------|
| a) Rectangle | b) Oval |
| c) Diamond | d) Parallelogram |

B. Answer with True or False.

- 5) An IF statement checks whether a condition is true or false.
- 6) IF...ELSE statements allow two possible actions.
- 7) The ELSE part runs when the condition is true.
- 8) Decision making in algorithms uses conditions.

3.5.1 If statement

In an algorithm the IF statement is a type of decision-making instruction used to check whether a condition is true or false before taking an action.

- If the condition is true, the statements inside the IF block are executed.
- If the condition is false, the algorithm does nothing and moves to the next step.

The general syntax of if...then is expressed as follows:

If condition is true, then

Do Task-A

For example, in the following statement, if...then condition tests whether mark is 80

and above. If the condition is true, the statement distinction is displayed on but this case, if the condition is false, nothing happens:

If mark $\geq$ 80 then

PRINT "distinction"

One important application of if...then selection is to validate user input. For example, a flowchart can be used with if ... then selection used to test whether a number entered is less than zero. If the number is negative, the algorithm displays invalid mark, which indicating that the value entered is not acceptable.

3.5.2 If else statement

If ... else selection is suitable when there are two available options. In general the format of if... else statement can be represented as:

```
IF <Boolean expression> THEN  
    Statement 1  
ELSE  
    Statement 2  
END IF
```

Explanation

The Boolean expression within If...then statement is first evaluated. If true, statement1 is evaluated otherwise statement 2 is evaluated if the condition returns false.

For example, the flowchart and pseudocode for checking voters eligibility depending on age. If a person is 18 years and above, the expression returns true and displays "Vote" else if a person is below the set age limit, the program displays "Do".

```

BEGIN
USE VARIABLE: AGE AS
INTEGER
WRITE "Enter person's age"
READ Age
IF Age >=18
WRITE "Vote"
ELSE
WRITE "Don't Vote"
END

```

Flowchart

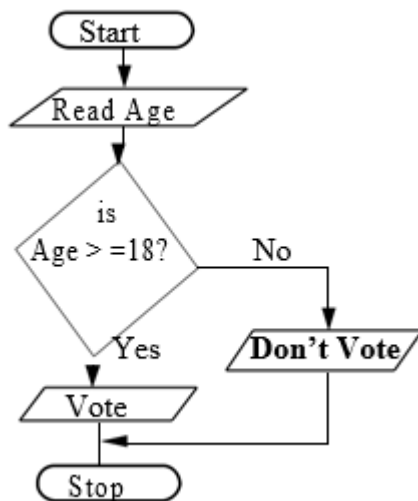


Figure 3. 4. If...Else

Application activity 3.5

- 1) Write an algorithm that reads a student's mark, displays "Pass" if the mark is 70 or above
- 2) Write an algorithm that reads a person's age, displays "Adult" if age is 18 or above, otherwise displays "Minor"
- 3) Write a pseudocode algorithm that reads a number from the user, checks if the number is positive or negative displays the result
- 4) Determine the output of the following pseudocode:

```
IF temperature > 30 THEN  
PRINT "Hot"  
ELSE  
PRINT "Cold"  
END IF
```

a) temperature = 35 b) temperature = 25

3.6 Nested if conditional statements

Learning activity 3.6

A. choose the correct answer

1) Nested IF statements are used when:

- | | |
|---------------------------------|--------------------------------------|
| a) Only one condition is tested | b) Several conditions must be tested |
| c) No condition is required | d) The program repeats instructions |

2) In nested IF statements, the ELSE part is executed when:

- | | |
|--------------------------------------|----------------------|
| a) The condition is true | b) The program stops |
| c) All previous conditions are false | d) Input is required |

3) In a flowchart, conditional tests are represented using:

- | | | | |
|--------------|------------------|------------|---------|
| a) Rectangle | b) Parallelogram | c) Diamond | d) Oval |
|--------------|------------------|------------|---------|

B. Answer with True or False

- 4) Nested IF statements allow multiple conditions to be checked.
- 5) The ELSE statement executes when all previous conditions are false.
- 6) Nested IF statements can only test two conditions.
- 7) In flowcharts, decisions are represented using diamonds.

3.6.1 Nested if statement social

Nested IF selection is used where several options have to be considered to make a selection. The general format of the Nested IF is:

```
IF <boolean expression> THEN
    statement 1
ELSE IF <boolean expression> THEN
    statement 2
ELSE IF <boolean expression> THEN
    statement 3
ELSE
    statement 4
END IF
```

Explanation

The statement first evaluates if the condition is true. If true, the statement is executed. If the first condition is false, the else if statement is evaluated. This continues until the else statement is encountered.

Note that in nested if selection, the last statement must be within else that executes the statement if the boolean expression returns false. When drawing a flowchart, if there are n options to select from, the number of diamonds representing IF should be n-1.

For example, the figure below shows an algorithm for a program that takes current date and date of birth. Depending on the current date, the algorithm computes age used to classify the people into categories shown in table on top-right side.

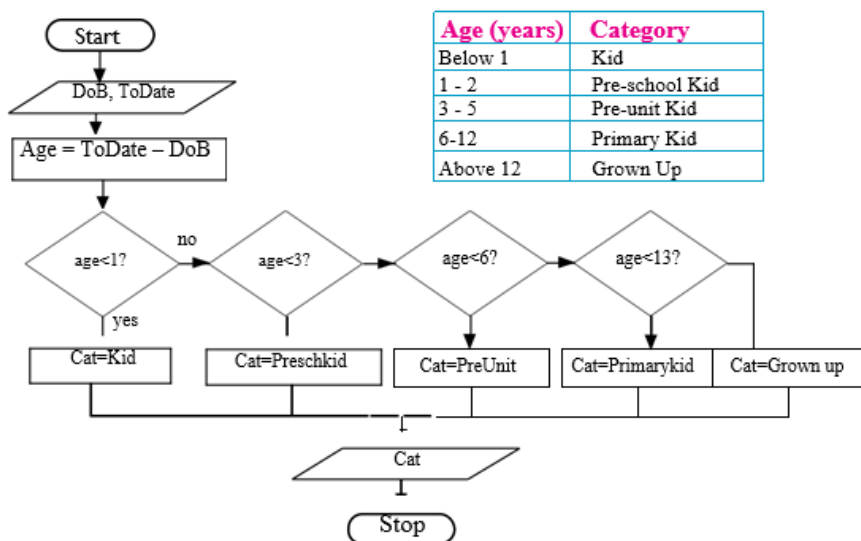


Figure 3. 5. Sample nested IF . ELSE selection

Explanation

The user first enters a person's date of birth (DoB) and the current date (ToDate) that are used to calculate age. Based on age, the algorithm uses nested if to assign the person to one of the categories (cat) defined in the table on the right.

For example, if age < 1 as indicated in the first decision symbol, cat is assigned to kid using the statement: cat = kid; the algorithm displays category of the person in the output symbol. For example, if cat is assigned to kid, the output symbol displays Grown up.

Application activity 3.6

1) Write an algorithm that classifies people based on age using the following conditions:

Age < 1 → Kid

Age < 13 → Child

Age < 20 → Teenager

Age ≥ 20 → Adult

2) Consider the following pseudocode:

```
READ age
IF age < 5 THEN
    PRINT "Small Child"
ELSE IF age < 13 THEN
    PRINT "Child"
ELSE IF age < 18 THEN
    PRINT "Teenager"
ELSE
    PRINT "Adult"
END IF
```

Fill in the correct output.

Age	Output
3	
10	
16	
25	

3.7 Loop statement

Learning activity 3.7

1) A loop statement is used to:

- a) Store data
- b) Repeat instructions
- c) Draw flowcharts
- d) Stop a program

2) Which loop is commonly used when the number of repetitions is known?

- a) While loop
- b) For loop
- c) If statement
- d) Nested loop

3) Which loop checks the condition before executing the statements?

- a) For loop
- b) While loop
- c) Both a and b
- d) None

4) Answer with True or False

- a) Loops allow a program to repeat instructions.
- b) A FOR loop is used when the number of repetitions is known.
- c) A WHILE loop always executes at least once.
- d) The WHILE loop checks the condition before executing the loop body.

The looping control structure, also referred to as iteration or repetition, causes the program to repeatedly execute statements within the loop until the condition is false.

For example, consider repetitive task that occurs during shopping represented by the following natural language algorithm.

Example:

```
WHILE shopping list is not empty DO
    pick an item and put in a shopping cart.
    Continue picking until the list is exhausted.
END WHILE
Proceed to checkout counter to make payment.
```

This algorithm describes a common practice of buying items in a retail outlet. The statements under the WHILE keyword indicate that a buyer continues picking items until the shopping list is exhausted. The keyword END WHILE shows that it is after picking all the items from the shopping list the buyer stops and proceeds to make payment at checkout counter.”

3.7.1 For loop

For loop is a looping statement used to evaluate a condition before executing statement in the body of the loop. The for loop can be represented using the following general syntax:

FOR variable = start_value TO end_value DO
statement(s)
END FOR

Example: Sum of numbers from 1 to 5

Pseudocode	Explanation	Output
<i>START</i> <i>SET SUM = 0</i> <i>FOR i = 1 TO 5 DO</i> <i>SUM = SUM +</i> <i>i</i> <i>END FOR</i> <i>PRINT SUM</i> <i>END</i>	$1 + 2 + 3 + 4 + 5 = 15$	SUM = 15

Table 3. 7. For loop example

3.7.2 While loop

Like the FOR loop, WHILE loop first evaluates the condition before executing the body of the loop. Therefore, while loop executes statements zero or more times. The general syntax of a while loop can be expressed using the following pseudocode and flowchart.

Characteristics of a For loop

- Used when number of repetitions is known.
- Includes initialization, condition, and update.
- Loop variable automatically increases each iteration.
- Commonly used in counting operations.

While loop Pseudocode	While loop Flowchart
-----------------------	----------------------

```

WHILE < boolean expression > DO
Statement(s)
update
END WHILE

```

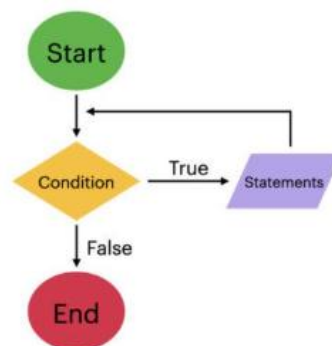


Figure 3. 6. While loop flowchart

Example: Print numbers from 1 to 5

Pseudocode	Explanation	Output
<pre> START SET i = 1 WHILE i <= 5 DO PRINT i i = i + 1 END WHILE END </pre>	Start with $i = 1$ Check condition $i \leq 5$ Print the value of i Increase i by 1 Repeat until $i > 5$	1 2 3 4 5

Table 3.8: While loop example

Characteristics for While loop

- It is a pre-test loop (condition checked first).
- It may execute zero or many times.
- It requires initialization and update.
- It is used when the number of repetitions is not exactly known.

Application activity 3.7

- 1) Write an algorithm that prints numbers from 1 to 10 using a FOR loop.
- 2) Determine the output of the following pseudocode:

```

SET SUM = 0
FOR i = 1 TO 4 DO
    SUM = SUM + i
END FOR
PRINT SUM

```


- 6) READ is used to display output.
- 7) WRITE is used to display results to the user.
- 8) A WHILE loop always executes at least once.

C. Open questions

- 1) Define an algorithm.
- 2) State two characteristics of a good algorithm.
- 3) Explain the importance of algorithms in problem solving.
- 4) List three common data types used in programming.
- 5) Name two flowchart symbols and their uses.
- 6) Why is it important to be careful when writing steps of an algorithm?

D. Practical questions

- 1) Write an algorithm that calculate area of annulus.
- 2) Draw a flowchart that reads temperature for each day in a week, in Celsius, converts the Celsius into Fahrenheit and then calculate the average weekly temperatures. The algorithm should display weekly average temperature in degrees Fahrenheit.
- 3) Design an algorithm for a program that would be used to compare three numbers x , y and z , and then display the least among the three.
- 4) Design an algorithm for a program that would be used to solve a quadratic equation: $y = ax^2 + bx + c$.

UNIT 4: PYTHON PROGRAMMING I



Key Unit Competence: Create Python programs that involve decision making and operations on data.

Knowledge and understanding

- Explain the importance of programming in solving real life problems.
- Explain how to install and run Python
- Identify different data types
- Explain the difference between variables and constants.
- Explain the use of operators, input and output functions and conditional statements
- Explain the role of loops in programming

Skills

- Install and run Python
- Write simple Python programs that declare variables and use appropriate data types.
- Use input and output functions to interact with users.
- Use different types of conditional statements in a program to make decisions.
- Write a program using different types of loops

Attitudes and values

- Value the use of programming in problem solving
- Appreciate accuracy, organization, and logic in coding.
- Demonstrate collaboration and knowledge sharing with peers.
- Value the role of programming in real-world problem-solving.

4.0 Introduction

The purpose of this unit is to help students understand the basic concepts of Python programming and how it is used to create simple computer programs. It enables students to gain an overview of Python, learn how to install it, and understand how programs are written using variables, data types, and operators. You will also become familiar with how programs interact with users through input and output functions.

Introductory activity

A group of students in a computer laboratory want to create a small program that helps the school record student information and display messages based on certain conditions.

- 1) What is a programming language, and why do we use programming languages like Python?
- 2) Why do you think a computer program needs variables to store information?
- 3) When writing a program that interacts with a user, why are input and output functions important?
- 4) In what situations might a program need to make decisions using conditions such as if statements?
- 5) Why are loops useful in programming instead of repeating the same instruction many times?

4.1. Introduction to python

Learning activity 4.1

- 1) What is a programming language, and why do we use programming languages?
- 2) Why do you think Python is considered a beginner-friendly programming language?
- 3) What types of tasks can be performed using Python?
- 4) Python is mainly used for:
 - a) Programming computers
 - b) Painting pictures
 - c) Playing music
 - d) Typing documents

- 5) Python programs are written using:
- a) Code
 - b) Images
 - c) Videos
 - d) Hardware
- 6) Which of the following is needed before running Python programs?
- a) Python installation
 - b) Printer installation
 - c) Keyboard installation
 - d) Scanner installation

4.1.1 Python overview

Python is a popular high-level programming language that is widely used to develop software, websites, automation tools, and data analysis applications. It is known for its simple and easy-to-read syntax, which makes it an excellent language for beginners who want to learn programming.

Python allows programmers to write instructions that a computer can understand and execute. These instructions help the computer perform tasks such as processing data, making calculations, making decisions, and repeating actions automatically. Because of its simplicity and powerful capabilities.

a) Some key features of Python include

Python has several key features that make it popular and easy to use. It is easy to learn and use, especially for beginners. It has a readable and simple syntax, which makes programs easier to understand and write.

Python is open source and free, meaning anyone can use and modify it without cost. It is also portable, as it can run on different operating systems such as Windows, Linux, and macOS.

b) Python is used in many areas such as:

Python is used in many fields including education, business, science, web development, and artificial intelligence. It is widely applied in web development for creating websites and web applications.

It is also used in data science and data analysis to process and interpret large amounts of data. In addition, Python plays an important role in

artificial intelligence and machine learning for building intelligent systems.

4.1.2 Python installation

The steps for installing Python in a computer:

Step 1: Open a web browser on the computer and go to the official Python website <https://www.python.org>

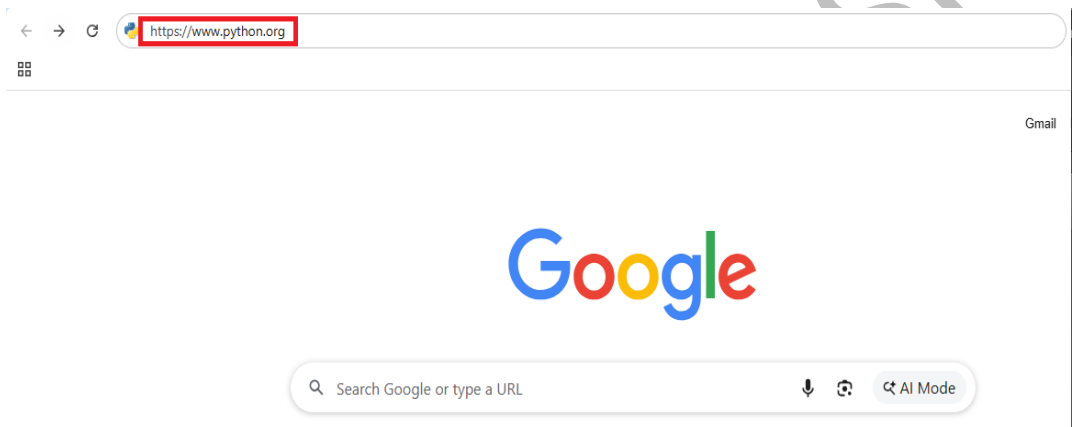


Figure 4. 1. Official Python website

Step 2: Click on **Downloads**.

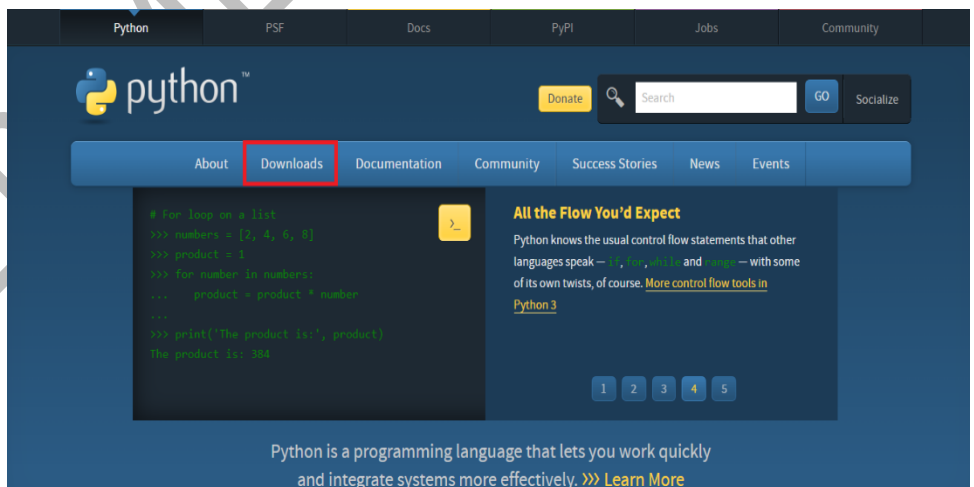


Figure 4. 2. Python download page

Step 3: Download the latest version of Python suitable for your operating system.

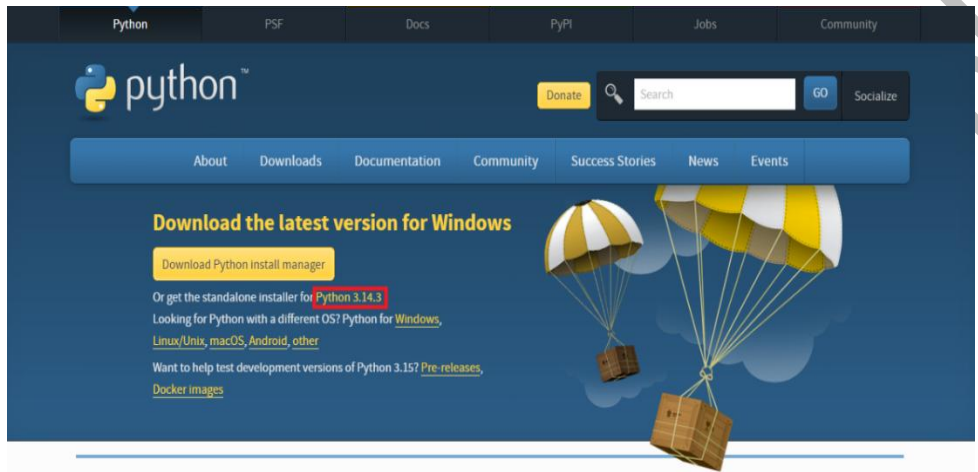


Figure 4. 3. Downloading python latest version

Step 4: Run a downloaded latest version of Python setup

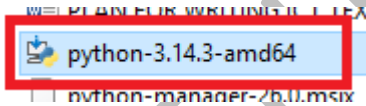


Figure 4. 4. Downloaded python setup file

Step 5: Tick Use admin privileges when installing py.exe and then after Click **Install Now**.

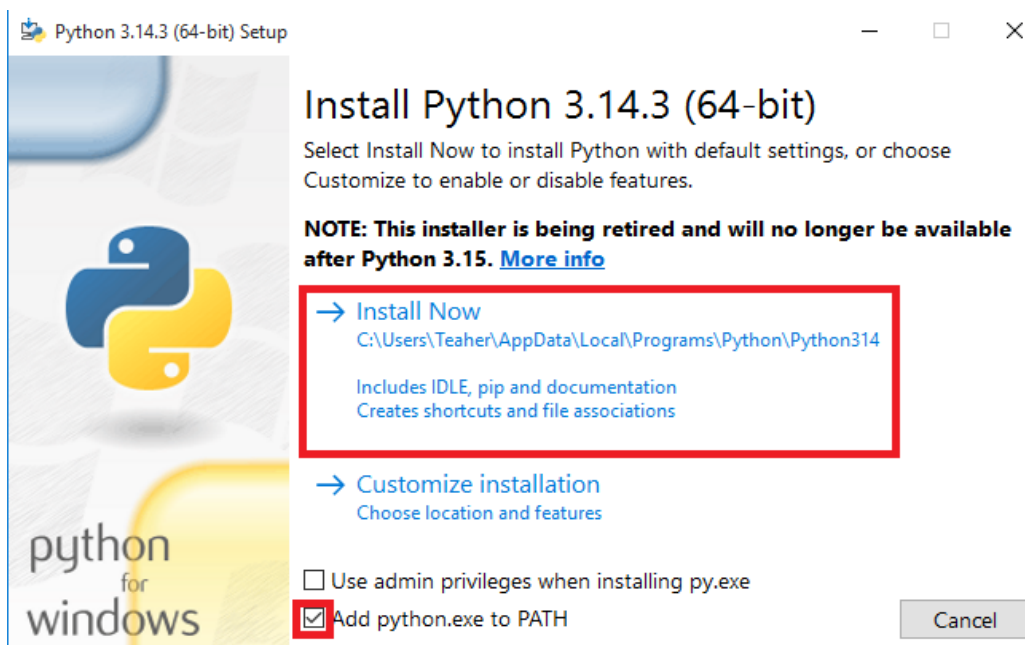


Figure 4. 5. Installation of Python

Step 6: Wait until the installation process finishes.

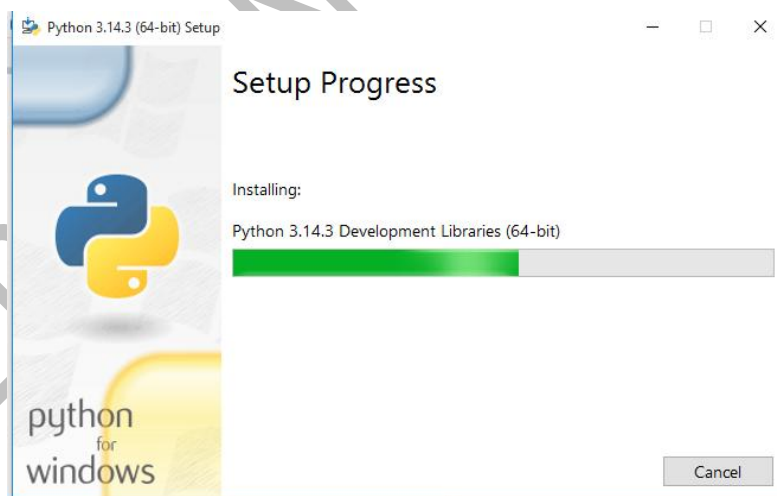


Figure 4. 6. Python setup progress

After installation, Python can be used through IDLE (Integrated Development and Learning Environment) or other code editors.

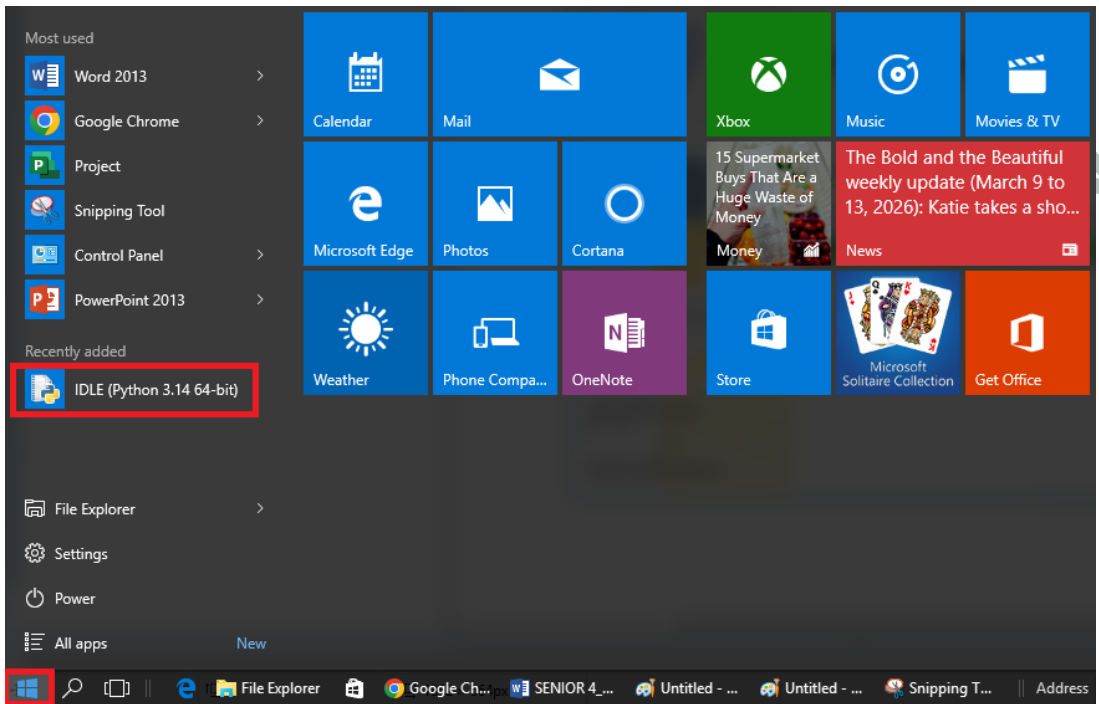


Figure 4. 7. Integrated Development and Learning Environment (IDLE)

Application activity 4.1

1) Installing and running Python

- a) Explain the steps required to install Python on a computer.
- b) Install Python on your computer and demonstrate how to verify that the installation was successful.
- c) Open Python IDLE or another Python editor.
- d) Write and run the following simple Python program: `print ("Hello, Python!")`

2) a)What message appears after running the program?

b) What command was used to display the message on the screen?

c) Why is Python installation necessary before writing Python programs?

4.2 Variables operation

Learning activity 4.2

- 1) What do you understand by the term variable in programming?
- 2) Why do you think programs need different types of data such as numbers and text?
- 3) Can you give examples of information that can be stored in variables?
- 4) Why are operators important when performing calculations in programs?
- 5) A variable is used to:
 - a) Store data
 - b) Delete programs
 - c) Turn off the computer
 - d) Print documents
- 6) Which of the following is a data type in Python?
 - a) int
 - b) monitor
 - c) keyboard
 - d) folder

In Python programming, variables are used to store information that can be used or changed during the execution of a program. Variables allow programmers to keep data such as numbers, text, or logical values so that the program can process and display them when needed.

When working with variables, programmers often perform different operations such as calculations, comparisons, or logical evaluations. These operations are carried out using operators. Understanding variables, data types, and operators is essential because they form the foundation for writing Python programs that can manipulate and process data effectively.

4.2.1 Data types

A data type defines the type of value that a variable can store. Python supports several types of data, but the most common ones include:

- **int (Integer):** Used to store whole numbers such as 5, 10, or 100.
- **float:** Used to store decimal numbers such as 3.5 or 7.25.
- **str (String):** Used to store text such as names or messages.
- **bool (Boolean):** Used to store logical values such as True or False.
- **datetime:** Used to store date and time values such as date of birth

Example:

```
age = 20
price = 15.50
name = "Alice"
is_student = True
DOB=16/03/2026
```

4.2.2 Variable

A variable is a named location in memory used to store data that can be used later in a program. In Python, variables are created when a value is assigned to them using the assignment operator (=).

Example:

```
x = 10
y = 5
total = x + y
print(total)
```

In this example x and y store numbers and “total” stores the result of the addition.

4.2.3 Operators

Operators are symbols used to perform operations on variables and value

a) Arithmetic operators

Operator	Meaning	Example
+	Addition	5 + 3
-	Subtraction	5 - 3
*	Multiplication	5 * 3
/	Division	6 / 2

Table 4. 1. Arithmetic operators

Example:

```

a = 10
b = 5
print(a + b)
print(a - b)
print(a * b)
print(a / b)

```

b) Comparison operators

Comparison operators are used in Python to compare two values or expressions. After comparison, they return a result that is always either True or False. These results are very important in programming because they help the computer make decisions, especially in conditions like if statements and loops. If we compare two numbers, Python checks whether one is greater than, less than, or equal to the other.

Here are the common comparison operators

Operator	Name	Example	Result
==	Equal to	5==5	True
!=	Not equal to	5!=3	True
>	Greater than	10>4	True
<	Less than	3<7	True
>=	Greater than or equal	5>=5	True
<=	Less than or equal	4<=7	True

Table 4.2 comparison operators

c) Logical operators

Logical operators are used in Python to combine two or more conditions and produce a single result that is either True or False. They are very important in decision-making because they allow a program to check multiple conditions at the same time.

For example, instead of checking one condition only, a program can check whether two conditions are both true, or if at least one condition is true.

Here are the logical operators in a table

Operator	Description	Example	Result
----------	-------------	---------	--------

AND	Returns True only if both conditions are true	(5 > 3) AND (10 > 2)	True
OR	Returns True if at least one condition is true	(5 > 10) OR (10 > 2)	True
NOT	Reverses the result of a condition	NOT (5 > 3)	True

Table 4.3 Logical operators

Examples: OR operator

```
marks = 45
attendance = 80
if marks >= 50 or attendance >= 75:
    print("Student passes")
```

Application activity 4.2

1) Which operator is used to assign a value to a variable?

- a) + b) = c) * d) >

2) Answer true or false

- a) Variables are used to store data in memory.
- b) A float data type stores decimal numbers.
- c) Python variables must be declared before use.
- d) The bool data type stores logical values.
- e) Comparison operators return True or False.
- f) Arithmetic operators are used for calculations.
- g) The operator > means less than.
- h) Logical operators can combine multiple conditions.

3) a) Open Python IDLE or another Python editor and write a Python program that asks the user to initialize (assign) two numbers, calculates the sum, difference, product, and division.

b) Which operators were used in the program?

c) What type of data was used when entering numbers?

4) Write a Python program that stores your name and age, then displays them.

4.3 Input and output functions

Learning activity 4.3

- 1) Why do computer programs need to receive information from users?
- 2) How do you think a program can display results to the user?
- 3) Can you give examples of situations where a program needs user input?
- 4) Why is it important for a program to display output clearly?
- 5) Which Python function is used to receive data from the user?
a) print() b) input() c) read() d) write()
- 6) Which Python function is used to display output on the screen?
a) input() b) display() c) print() d) show()
- 7) Input data from the user is usually entered using the:
a) mouse b) keyboard c) printer d) speaker

In Python programming, input and output functions are used to allow interaction between the user and the computer program. These functions help a program receive data from the user and display results or messages on the screen.

The input function allows the user to enter information such as numbers or text through the keyboard. The output function is used to display information or results to the user. These functions are very important in programming because they make programs interactive and useful in solving real-world problems.

4.3.1 Input function

The input() function is used in Python to receive information from the user through the keyboard. When the program reaches the input function, it waits for the user to type a value and press Enter. The value entered by the user can be stored in a variable for later use in the program.

Example:

```
name = input("Enter your name: ")  
print("Hello", name)
```

In this example, the program asks the user to enter a name and stores the entered value in the variable name. The program then uses the stored value to display a greeting message to the user. If the program needs numbers, the input value can be converted using int() or float().

Example:

```
age = int(input("Enter your age: "))
print("Your age is", age)
```

4.3.2 Output function

The print() function is used to display information or results on the screen. It allows a program to communicate results to the user.

Example:

```
print("Welcome to Python programming")
```

Output: Welcome to Python programming

The print function can also display the value stored in variables.

Example:

```
Fname = "KABAYIZA"
print("My first name is: ", Fname)
```

Output: My first name is: KABAYIZA

Application activity 4.3

Practical: Using Input and Output Functions

Open Python IDLE or another Python editor.

- 1) Write a Python program that asks the user to enter a name, age, and displays a message showing the entered information.
- 2) explain the use of input () and output function in python?

4.4 Conditional statements

Learning activity 4.4

- 1) What does it mean for a computer program to make a decision?

- 2) Can you give examples from daily life where decisions are made based on conditions?
- 3) Why do you think programs need conditional statements?
- 4) Which keyword is used to test a condition in Python?
a) if b) for c) print d) input
- 5) Which statement allows a program to choose between two options?
a) if b) if...else c) for d) while

Conditional statements allow a program to make decisions based on certain conditions. They control the flow of a program by executing specific instructions only when a given condition is true. In Python, conditional statements help programs perform different actions depending on the situation.

Python uses keywords such as if, if...else, and if...elif...else to implement decision-making in programs.

a) If statement

The if statement is used to execute a block of code only if a condition is true.

Syntax:

```
if condition:  
statement(s)
```

Example:

```
age = 18  
if age >= 18:  
    print("You are an adult")
```

b) If...Else Statement

The if...else statement allows the program to choose between two possible actions.

Syntax:

```
if condition:  
statement(s)  
else:  
statement(s)
```

Example:

```
age = 16  
if age >= 18:  
    print("You can vote")  
else:  
    print("You cannot vote")
```

c) If...Elseif...Else Statement

This statement is used when there are multiple conditions to check.

Syntax:

```
if condition1:  
    statement(s)  
elif condition2:  
    statement(s)  
elseif condition3:  
    statement(s)  
else:  
    statement(s)
```

Example:

```
marks = 75  
  
if marks >= 80:  
    print ("Grade A")  
elif marks >= 60:  
    print("Grade B")  
else:  
    print("Grade C")
```

Application activity 4.4

- 1) Using conditional statement, write a python program that solves quadratic equation of the following format: $ax^2+bx+c=0$.
- 2) Write a Python program which prompts a user to input 4 numbers and then displays the biggest number and the smallest number.
- 3) Write a Python program that ask a user to enter a number and then informs whether it is odd or even.
- 4) Write a Python program that ask a user to enter a number and then informs whether it is negative or positive.

4.5 Nested conditions

Learning activity 4.5

- 1) What do you understand by the term nested condition?
- 2) Why might a program need to check more than one condition?
- 3) A nested condition means:
 - a) A loop inside a loop
 - b) An if statement inside another if statement
 - c) A print inside a loop

d) A variable inside a program

A nested condition occurs when one if statement is placed inside another if statement. This allows a program to check multiple related conditions.

4.5.1 Nested if

A nested if statement is used when a decision depends on another condition being true first. Nested if statements help programmers to make decisions step by step, check multiple conditions and create more detailed programs.

Syntax

```
if condition1:  
    # code executed if condition1 is true  
if condition2:  
    # code executed if condition2 is also true
```

The second if is called a nested if because it is inside the first one.

Example1: Positive even number

```
number = 8  
if number > 0:  
    if number % 2 == 0:  
        print("Positive even number")
```

Example2:

```
num = 10  
if num > 0:  
    if num % 2 == 0:  
        print("The number is positive and even")
```

Example 3:

```
temperature = 30  
if temperature > 20:  
    if temperature > 35:  
        print("Very Hot")  
    else:  
        print("Warm")
```

The program means: If the temperature is above 20, check again, If it is above 35 > display "Very Hot", otherwise > display "Warm".

Application activity 4.5

1) Write a Python program that asks the user to enter age, If the age is 18 or above, ask if the person is a citizen, display whether the person is eligible to vote.

2) Study the following Python program and explain its output.

```
marks = 70
if marks >= 50:
    if marks >= 80:
        print("Grade A")
    else:
        print("You passed")
else:
    print("You failed")
```

4.6 While loops statements

Learning activity 4.6

- 1) Why are loops important in programming?
- 2) What might happen if a loop never stops?
- 3) Which statements are used for repeating instructions while a condition is true?
 - a) if
 - b) for
 - c) while
 - d) input

4.6.1 While loop

A while loop is used to repeat a block of code as long as a condition is true. It is useful when the number of repetitions is not known in advance. It is called a loop because the statements keep running again and again until the condition becomes false.

- Python checks the condition
- If the condition is True, the code inside the loop runs
- After running, Python checks the condition again

- The loop stops when the condition becomes False

Syntax:

while condition:
statement(s)

Example 1:

```
count = 1
while count <= 5:
    print(count)
    count += 1
```

Output

1
2
3
4
5

This program prints numbers from 1 to 5 using a while loop. It keeps repeating until the condition becomes false.

Example 2: Countdown program**Output:**

```
number = 5
while number > 0:
    print(number)
    number = number - 1

print("Done")
```

5
4
3
2
1
Done

This program counts backwards from 5 to 1, then prints "Done" at the end.

In this example, the variable number is assigned the value 5, which means the countdown starts from 5. The program then repeatedly executes the instructions while checking the condition. The countdown continues by reducing the value step by step until the condition becomes false, at which point the loop stops executing.

Application activity 4.6

- 1) Write a Python program that uses a while loop to display numbers from 1 to 10.
- 2) Write a Python program that displays the odd numbers within the number from 15 up to 1, using While loop.

4.7 For loop statements

Learning activity 4.7

1. What is the difference between a for loop and a while loop?
2. When is it better to use a for loop instead of a while loop?
3. Which loop is commonly used when the number of repetitions is known?
a) if b) while c) for d) input

4.7.1 For loop

A for loop in Python is used to repeat a block of code a fixed number of times or to go through items in a sequence (like a list, string, or range of numbers).

Syntax:

for variable in
sequence:
statement(s)

Example1:

*For i in range (5):
Print(i)*

Output

0
1
2
3
4

Example2:

*for i in range(2, 11, 2):
print(i)*

Output

2
4

6

8

10

Application activity 4.7

- 1) Write a Python program that uses a for loop to display the multiplication table of 5.
- 2) Write Python program that calculate the sum of numbers from 1 up to 5

4.8 End of unit assessment

4.8.1. Project Activity

Project Title: Student Marks Calculator Program

Guidance:

You are responsible for calculating and recording student performance using a simple Python program.

- Create a Python program that asks the user to enter a student's name and marks for 3 subjects (Math, English, ICT).
- Use variables to store all input values.
- Use arithmetic operators to calculate total marks and average marks.
- Use conditional statements to assign grades based on the average: A (80–100), B (60–79), C (below 60)
- Use a while loop to allow the teacher to enter more than one student without restarting the program.
- Display a clear report for each student including name, total marks, average, and grade.
- The program should stop only when the teacher decides to exit.

Questions on the Project:

- 1) Which variables were used in your program and what was their purpose?
- 2) Which operators and data types were used in the program?
- 3) How did you use conditional statements to assign grades?
- 4) Why is a while loop important in this program?

4.8.2 Assessment questions

1) Match the Python term in Column A with the correct description in Column B.

Column A	Column B
a) Variable	i Repeats a block of code several times
b) Data type	ii Stores information in memory
c) Loop	iii Determines the type of value stored
d) Operator	iv Symbol used to perform calculations

2) State any three advantages of Python programming language.

3) Identify whether the following statements are True or False.

- a) Python programs are written using English-like syntax.
- b) A variable must always store text values only.
- c) The print() function is used to show output to the user.
- d) Loops help reduce repetition of instructions in a program.

4) Explain why data types are important in Python programs.

5) Explain how conditional statements help a program make decisions.

6) Describe two situations in real life where programming logic could be used.

7) Study the following Python program and answer the questions below.

```
x = 8
y = 4
result = x * y
print ("The result is:", result)
```

- a) What values are stored in variables x and y?
- b) What operation is performed in the program?
- c) What will be the output of the program?

8) Consider the following program:

```
num = 7
if num > 5:
    print("Number is greater than five")
else:
    print("Number is five or less")
```

a) What condition is tested in the program?

b) What output will appear when the program runs?

9) Write a Python program that asks the user to enter the length and width of a rectangle and displays the area of the rectangle.

10) Write a Python program that asks the user to enter two numbers and displays the largest number using conditional statements.

11) Write a Python program that uses a for loop to display numbers from 10 down to 1.

12) Write a Python program that asks the user to enter a number and displays the square of that number.

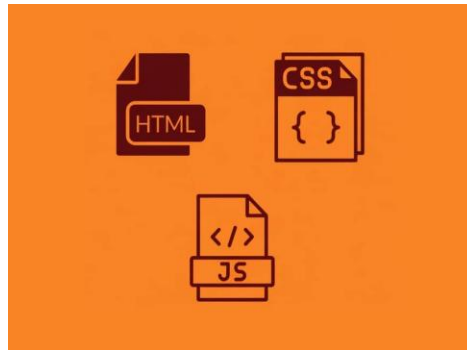
13) A shop gives a discount based on the amount of money spent. If the amount is 100 or more, the customer gets a 10% discount and If the amount is less than 100, there is no discount. Write a Python program that asks the user to enter the purchase amount and displays the final amount after applying the discount.

14) Explain two reasons why learning programming skills is important for students today.

15) Write a Python program that asks the user to enter five numbers and calculates their total using a loop.

EXPERIMENTAL VERSION

UNIT 5: HTML-BASED WEB DEVELOPMENT



Key unit competence: Develop well-structured web page using HTML tags and web development elements.

At the end of this unit you should be able to:

Knowledge and understanding:

- Explain the importance of HTML tags in web development
- Identify different types of tags used to develop a web page
- Describe the structure of an HTML document and a web page
- Explain the importance of using tables and forms on a web page

Skills

- Create webpages using HTML tags with appropriate text formats.
- Construct tables and forms using HTML tags
- Create hyperlinks and basic navigation links between web pages.

Attitudes and values

- Value the importance of HTML as the foundation of web development.
- Show creativity in designing webpages.
- Demonstrate patience and persistence when solving coding errors.

5.0 Introduction

The purpose of this unit is to help you understand the basic concepts of HTML as a foundation for creating web pages. Through this unit, you will become familiar with creating websites using HTML tags. These websites can then be hosted online so that they become available online. To get these skills you will learn different HTML tags so that you can create a web page containing forms, lists, buttons, etc.

Introductory activity

A group of students in a computer laboratory are interested in creating their own web page to display school information such as announcements, student lists, images, and videos.

1. What is HTML, and why is it important in creating web pages?
2. What role does HTML play in the structure of a web page?
3. Explain the common HTML tags ?

5.1 Introduction to HTML

Learning activity 5.1:

- 1) What do you understand by HTML?
- 2) Explain the use of <html>, <head>, and <body> tags in a simple HTML page.

5.1.1 Definition of HTML

HTML or HyperText Markup Language is the standard markup language used to create web pages. It uses tags to structure content so browsers can display headings, paragraphs, lists, images, links, and forms correctly.

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>HTML Example</title>
</head>
<body>
  <p>This is a simple HTML page.</p>
```

```
</body>
</html>
```

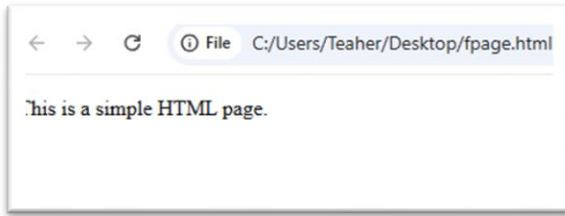


Figure 5. 1. Simple HTML page

5.1.2 Role of HTML

HTML is responsible for structuring content on a web page. It defines headings, paragraphs, and lists, embeds images, audio, and video and adds links and interactive elements.

Example: The following code uses the `<p>` tag to create a paragraph.

`<p>`HTML organizes headings, paragraphs, images, and links so that browsers display them properly. `</p>`

5.1.3 HTML page structure

- **Head section:** Contains metadata like `<title>` and `<meta>`. Content here is not visible.
- **Body section:** Contains all content visible on the page such as text, images, links, tables and forms.

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>School Page</title>
</head>
<body>
  <h1>Welcome to Our School Website</h1>
  <p>This content is visible to users.</p>
</body>
</html>
```

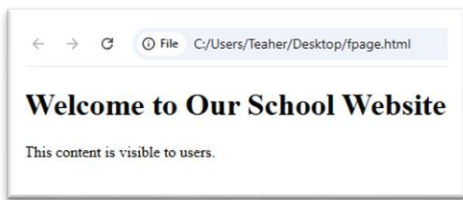


Figure 5. 2. HTML page structure

Application activity 5.1

- 1) Create a page with a head section containing a title of your choice, body section containing a heading and a paragraph from your choice.
- 2) Create a page with a head section containing a title called “Teacher’s website”, body section containing a heading called “All courses” and two paragraphs from your choice.

5.2 Page structure tags

Learning activity 5.2

- 1) Discuss why metadata is placed in the head and not in the body.
- 2) What is the purpose of page structure tags in HTML?
 - a) To add animations b) To organize and divide a webpage into sections
 - c) To create databases d) To install software
- 3) Which HTML tag is the root element of a webpage?
 - a) <body> b) <head> c) <html> d) <title>
- 4) What is the purpose of the <meta> tag?
 - a) To insert images b) To provide information about the webpage
 - c) To create paragraphs d) To display headings

Page structure tags in HTML are special elements used to organize and divide a web page into meaningful sections. They help make the page easier to read, understand, and manage for both developers and browsers.

Page structure tags are semantic HTML tags that describe the role of different parts of a webpage. Instead of just using <div>, these tags clearly show what each section contains.

These are important tags used in the <head> section of an HTML document. They help define information *about* the webpage rather than displaying content directly.

- <html>: The root element that contains all HTML content.
- <head>: Contains metadata such as title and linked CSS files.
- <body>: Contains all visible content like text, images, lists, tables, and forms.

Example:

```
<html>
<head>
  <title>ICT Class</title>
</head>
<body>
  <p> HTML allows users to organize content.</p>
</body>
</html>
```

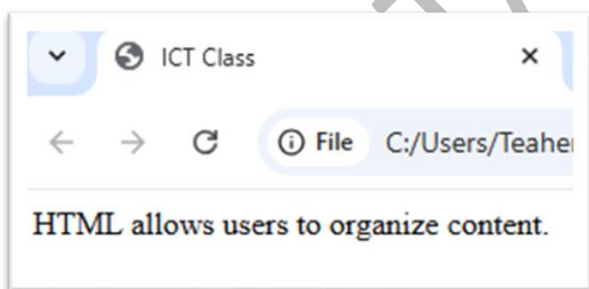


Figure 5. 3. Page structure (a)

- <title>: Used to set the page title displayed in the browser tab.
- <meta>: Used to provide information such as character set or description.
- <link>: Used to link external files like CSS stylesheets.
- <!DOCTYPE>: Used to declare the HTML version to the browser.

Example:

```

<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>School Website</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <p>Welcome to our school website!</p>
</body>
</html>

```

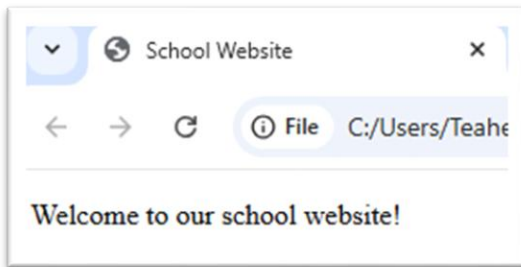


Figure 5. 4. Page structure (b)

Application activity 5.2

- 1) Create a page with a title, meta charset="UTF-8", a link to a CSS file, and <!DOCTYPE html>.
- 2) Which declaration tells the browser the HTML version being used?
 - a) <html>
 - b) <meta>
 - c) <!DOCTYPE html>
 - d) <head>
- 3) What is the difference between <head> and <body>?
- 4) What information does the <meta charset="UTF-8"> tag provide?

5.3 Text formatting tags

Learning activity 5.3

- 1) Discuss why headings are important for organizing content.
- 2) Which HTML tag is used for the smallest heading?
 - a) <h1>
 - b) <h3>
 - c) <h6>
 - d) <head>
- 3) Paragraphs are mainly used to:
 - a) Insert videos
 - b) Organize sentences into readable blocks
 - c) Create headings
 - d) Draw tables

Text formatting tags are HTML elements used to change the appearance, emphasis, or meaning of text on a webpage. They help make content clearer, more attractive, and easier to read.

5.3.1 Headings tag

Heading tags are used in HTML to create titles and subtitles on a webpage. They help organize content by defining the structure and hierarchy of information, making the webpage easier to read and understand. Headings also help users quickly identify important sections of a page and improve the appearance and organization of content. HTML provides six levels of heading tags, from `<h1>` to `<h6>`. Headings define the structure and hierarchy of content:

- `<h1>`: Main title, largest and most important
- `<h2>` to `<h6>`: Subheadings with decreasing importance/size

Example:

```
<h1>School Website</h1>
<h2>About Us</h2>
<h3>Our Mission</h3>
```

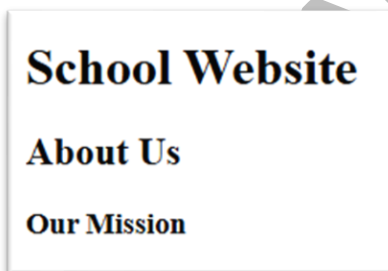


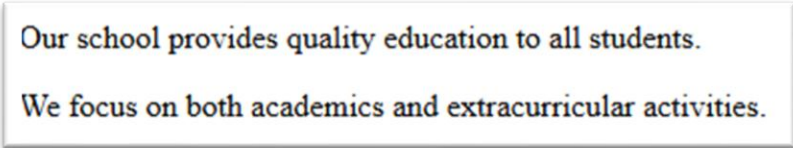
Figure 5. 5. Text formatting

5.3.2 Paragraphs tag

The paragraph tag in HTML is used to define and organize blocks of text into paragraphs on a web page. It helps separate content into readable sections and automatically adds space before and after each paragraph. This makes the webpage clear, and easier for users to read. Paragraphs group sentences into blocks of text for readability.

Example:

```
<p>Our school provides quality education to all students.</p>  
<p>We focus on both academics and extracurricular activities.</p>
```



Our school provides quality education to all students.

We focus on both academics and extracurricular activities.

Figure 5. 6. Paragraphs text organization

5.3.3 Emphasis tags

Emphasis tags are HTML elements used to give special importance or stress to certain words in a text. They not only change how text looks, but also add meaning, which is important for browsers, search engines, and accessibility tools.

- `<b>`: Makes text bold for strong emphasis.
- `<i>`: Italicizes text to highlight terms or foreign words.
- `<u>`: Underlines text for importance or attention.

Example1:

```
<p><b>Bold</b>, <i>Italic</i>, <u>Underline</u></p>
```



Bold, *Italic*, Underline

Figure 5. 7. Bolding, Italic and underling text

5.3.4 Marquee tag

Marquee tag creates scrolling text across the page. This tag is deprecated in modern HTML but still useful for simple practice.

Example:

```
<marquee>Welcome to our school website!</marquee>
```

Welcome to our school website!

Figure 5. 8. Marquee text

Application activity 5.3

- 1) Create a page with one <h1> heading for the page title, one <h2> subheading, two paragraphs introducing the school, at least one word in each paragraph using , <i>, or <u> and scrolling welcome message using <marquee> tag.
- 2) Create a webpage containing: a heading “Welcome to ICT”, a paragraph about HTML and then bold, italic, and underlined text.
- 3) Write HTML code to display "School Website" as a Heading 1, "About School" as a Heading 2, and "Mission" as a Heading 3 on a web page.

5.4 List tags

Learning activity 5.4

- 1) Identify ordered and unordered lists on a sample webpage.
- 2) Discuss when to use numbered lists and when to use bullet points.

List tags are used to organize information into ordered, unordered, or descriptive lists. They make content easier to read and understand.

5.4.1 Unordered list, ordered list, list item tags

Displays items in a numbered format and are used when the order of items is important.

- : Creates a bullet-point list.
- : Creates a numbered list.
- : Each item in the list.

Example1:

```

<h3>Subjects Offered</h3>
<ul>
  <li>Mathematics</li>
  <li>English</li>
  <li>Science</li>
</ul>
<h3>Steps to Register</h3>
<ol>
  <li>Fill the form</li>
  <li>Submit documents</li>
  <li>Receive confirmation</li>
</ol>

```

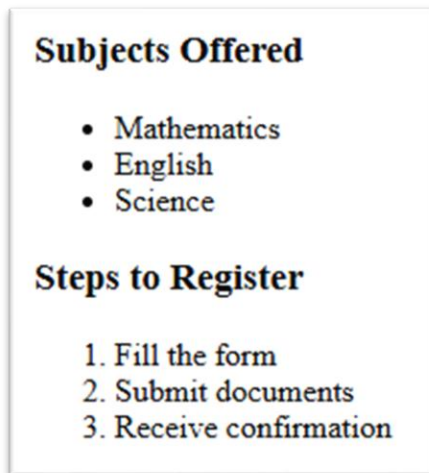
Output

Figure 5. 9. Ordered and unordered lists

Example2: An ordered list uses numbers (1, 2, 3...) to organize items.

```

<ol>
  <li>Books</li>
  <li>Pens</li>

  <li>Exercise books</li>
</ol>

```

Output

1. Books
2. Pens
3. Exercise books

Example3:

```

<ol>
  <li>Books</li>
  <li>Pens</li>
  <li>Exercise books</li>
</ol>

```

Output

- Books
- Pens
- Exercise books

Application activity 5.4:

- 1) Create a page with an unordered list of at least 5 school subjects, an ordered list of at least 3 steps to join a school club.
- 2) Create an unordered list of three fruits
- 3) Create an ordered list of three subjects.

5.5 Media tags

Learning activity 5.5

- 1) Identify image, audio, and video elements on a sample page.
- 2) Discuss why multimedia is important for web pages.

5.5.1 Embed images tag

An embedded image in HTML is an image that is directly displayed on a webpage using the HTML `<img>` tag. The image is is ML is an image that is directly displayed on a webpage using the HTMLser can show it automatically when the page loads.

Embed images tag displays images using the `src` attribute. `alt` provides alternative text if the image cannot load.

Example:

```

```

5.5.2 Audio tag

Audio tag plays audio files. The `controls` attribute adds play, pause, and volume options.

Example:

```
<audio controls>  
  <source src="welcome.mp3" type="audio/mpeg">  
  Your browser does not support the audio element.  
</audio>
```

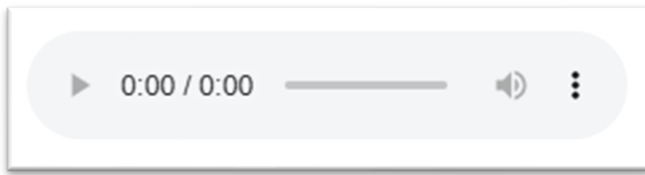


Figure 5. 10. Added audio

5.5.3 Video tag

Video tag plays video files. Attributes like controls, width, and height make the video interactive.

Example:

```
<video width="320" height="240" controls>  
  <source src="ict.mp4" type="video/mp4">  
</video>
```

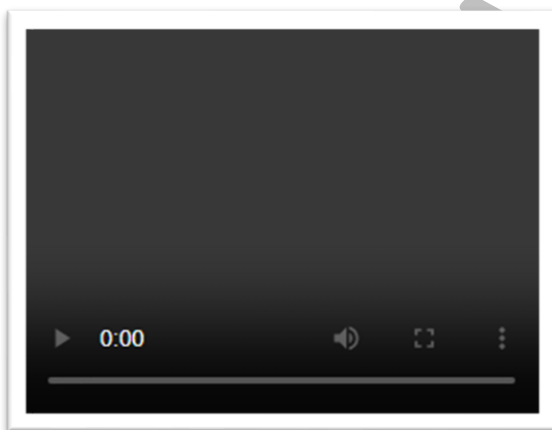


Figure 5. 11. Added video

Application activity 5.5

- 1) Create a page with one image of your choice, one audio file that can be played on the page and one video with controls.
- 2) What is the role of the <source> tag inside the audio element?
- 3) Name two common attributes used in the video tag.
- 4) Write an example of a video tag.

5.6 Table tags

Learning activity 5.6

- 1) Define table headers, rows, and columns .
- 2) Discuss situations where tables are useful.
- 3) What does the border attribute do in a table?

Table tags in HTML are used to organize data in a tabular format using rows and columns. Tables are especially useful for displaying structured information like schedules, scores, or comparisons.

5.6.1 Structure tabular data tags

The structure of tabular data tags in HTML refers to the set of tags used to create and organize tables on a webpage. These tags include `<table>`, `<tr>` (table row), `<th>` (table header), and `<td>` (table data). They work together to arrange information into rows and columns for easy understanding and presentation.

- `<table>`: Creates a table container
- `<tr>`: Table row
- `<td>`: Table data cell
- `<th>`: Table header (bolded by default)

Example1:

```
<table border="1">
  <tr>
    <th>Name</th>
    <th>Age</th>
    <th>Class</th>
  </tr>
  <tr>
    <td>John</td>
    <td>14</td>
    <td>8A</td>
  </tr>
  <tr>
    <td>Mary</td>
    <td>13</td>
    <td>7B</td>
  </tr>
</table>
```

Output is:

Name	Age	Class
John	14	8A
Mary	13	7B

Table 5. 1. Tabular data

Example2: Dropdown list with five countries

```
<h1>This is a list of countries of East Africa </h1>
<select>
  <option>Rwanda</option>
```

```

<option>Kenya</option>
<option>Uganda</option>
<option>Burundi</option>
<option>Tanzania</option>
</select>

```

This is a list of countries of East Africa

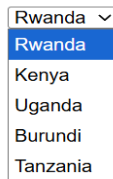


Figure 5. 12 List

Application activity 5.6

1) Create a table with at least 10 rows of students, columns for Name, Age, Address and Class

2) Answer with True or False

- The `<img>` tag is used to insert images in HTML.
- The controls attribute is used in audio and video tags.
- `<source>` is used only inside tables.
- The `<th>` tag creates bold table headings by default.
- Tables are useful for organizing structured data.
- The `<video>` tag cannot play videos.
- The src attribute specifies the media file location

3) Write HTML code to display an image named student.jpg.

4) Create a table showing three students and their marks.

5.7 Form elements tags

Learning activity 5.7

- 1) Describe the main elements found in an HTML form and explain the function of each element.
- 2) Discuss why forms are important for collecting user data.
- 3) Which tag is used as a container for all form elements?
a) <table> b) <form> c) <body> d) <div>
- 4) Which tag is used to create a text box in HTML?
a) <textarea> b) <button> c) <input> d) <select>
- 5) Which input type is used for entering normal text?
a) radio b) checkbox c) text d) submit

Form elements tags are used to create **interactive forms** that allow users to enter and submit data on a webpage.

5.7.1 Handle user input tags

- <form>: Container for all input elements
- <input>: Text box, radio button, checkbox
- <textarea>: Multi-line input
- <button>: Submit or reset button
- <select> + <option>: Dropdown list

Example1:

```
<form>
Name: <input type="text" name="name"><br><br>
Gender:
<select>
  <option>Male</option>
  <option>Female</option>
</select><br><br>
Comment: <textarea rows="4" cols="30"></textarea><br><br>
<button type="submit">Submit</button>
</form>
```

Figure 5. 13 Form elements

Example2:

```
<form>
Student Name:
<input type="text" name="studentname"><br><br>

Gender:
<select>
  <option>Male</option>
  <option>Female</option>
</select><br><br>

Feedback:<br>
<textarea rows="4" cols="30"></textarea><br><br>

<button type="submit">Submit</button>
</form>
```

Application activity 5.7

- 1) Create a form with name input, gender selection dropdown, comments text area and submit button
- 2) Which tag is used to create a multi-line text area?
 - a) <input>
 - b) <textarea>
 - c) <option>
 - d) <select>
- 3) Which tag creates a clickable button?
 - a) <click>
 - b) <submit>
 - c) <button>
 - d) <inputbox>

- 4) Which tag is used to create a dropdown list?
a) <textarea> b) <select> c) <option> d) <list>
- 5) Which tag is used inside a dropdown list?
a) <input> b) <button> c) <option> d) <textarea>
- 6) Fill in the Blanks

- a) The <_____> tag is used to create a form.
b) The <_____> tag is used to create a text box.
c) The <_____> tag is used for multi-line input.
d) The <_____> tag creates a dropdown list.
e) The <_____> tag creates items inside a dropdown list.
f) The <_____> tag creates buttons.
g) The type="_____" attribute creates a submit button.

5.8 Link and navigation tags

Learning activity 5.8

- 1) What is link and describe how you can identify links on a web page.
- 2) Differentiate internal and external links.

Link and navigation tags are used to connect web pages and help users move from one page to another within a website or to external sites.

5.8.1 Hyperlinks

Hyperlinks are connections that allow users to move from one webpage to another page, website, document, or section of a webpage. Hyperlinks make websites interactive and easy to navigate.

HTML uses the <a> tag, also called the anchor tag, to create hyperlinks.

Syntax: Link Text

a) Href attribute

An attribute in HTML is a special part added inside an HTML tag that provides extra information or instructions about an element. Attributes are written inside the opening tag and usually come in a name = "value" format. Href Attribute: Specifies the link destination

Eg: `<a href="about.html">About Page</a>`

b) Target in html

The target attribute is used in HTML links (inside the `<a>` tag) to specify where the linked page should open when a user clicks the link. It controls the window or tab behavior of the link.

`target="_blank"`: Opens link in a new tab

Example:

```
<p>Visit our <a href="about.html">About Page</a></p>
<p>Go to <a href="https://www.google.com" target="_blank">Google</a></p>
```

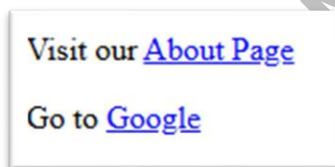


Figure 5. 14 Link and navigation

UP TO HERE

Application activity 5.8

- 1) Create a page with one link to an internal page and one link to an external website that opens in a new tab.
- 2) Create a hyperlink to a page called home.html.
- 3) Create a hyperlink that opens Google in a new tab.
- 4) Create an email hyperlink for student@gmail.com.

5.9 End of unit assessment

5.9.1 Project

You are required to design a simple web page for your school using HTML. The page should present basic information such as school name, subjects, student data, media, and a contact form.

Project description

You are required to design a simple web page for your school using HTML. The page should present basic information such as school name, subjects, student data, media, and a contact form.

Apply the following

- Use `<!DOCTYPE html>`
- Include `<html>`, `<head>`, and `<body>`
- Add a title of your choice
- Include `<meta charset="UTF-8">`
- Create one main heading `<h1>` (School name)
- Create one subheading `<h2>`
- Create at least two paragraphs describing the school
- Format your text using Bold, Italic and underline on each heading
- Add a scrolling message of your choice
- One unordered list of at least 5 school subjects
- One ordered list showing steps to join a school club
- Add one image (`<img>`), one audio (`<audio controls>`), one video (`<video controls>`)
- Create a table with, columns: name, age, class and at least 3 student records
- Create a simple form including: Name input, gender dropdown, comment box, submit button.
- Add one internal link and one external link

5.9.2 Assessment questions

- 1) HTML stands for:
 - a) Hyper Tool Markup Language
 - b) HyperText Markup Language

- c) HighText Machine Language d) Hyper Transfer Mark Language
- 2) Which tag is used to define the title of a webpage?
a) <meta> b) <head> c) <title> d) <body>
- 3) The visible content of a webpage is placed inside:
a) <head> b) <body> c) <html> d) <title>
- 4) Which tag is used to create a paragraph?
a) <h1> b) <p> c) d)
- 5) The tag is used to create:
a) A numbered list b) A table
c) A bullet list d) A form
- 6) Which tag is used to insert an image?
a) <video> b)
c) <audio> d) <media>
- 7) Identify the correct HTML document structure:
a) <body> <head> <html> b) <html> <body> <head>
c) <html> <head> <body> d) <head> <html> <body>
- 8) Answer by True or False
- HTML is the foundation of web development.
 - Writing correct HTML code does not require attention to detail.
 - Collaboration is important in web development projects.
- 9) Explain the importance of HTML tags in web development.
- 10) Create a simple HTML page with a heading and a paragraph.
- 11) Create a webpage that includes an image and a link to another website.
- 12) Explain how can creativity improve the design of a web page?
- 13) Describe how teamwork can help in developing web projects.

UNIT 6: CASCADING STYLE SHEET



Key unit competence: Apply styles in web pages for better visual presentation and usability

At the end of this unit, you will be able to:

Knowledge and understanding

- Describe how CSS controls the appearance and layout of HTML elements.
- Identify different text formatting properties
- Explain how combining text styling and layout styling contributes to visually appealing web pages.

Skills

- Interpret the structure and syntax of CSS rules
- Design a webpage section using proper font, spacing, and background properties.
- Implement borders, padding, and layout adjustments effectively.
- Distinguish between internal and external and inline CSS

Attitudes and values

- Value the role of good layout and spacing
- Demonstrate responsibility in following web design standards and best practices.
- Show creativity when selecting colors, fonts, and backgrounds for a webpage.

6.0 Introduction

The purpose of this unit is to help you understand the basic concepts of styling in web development using CSS. It will enable you to apply text formatting techniques such as font styling, text appearance, spacing, layout, and visual effects to improve the presentation of web content. By learning this unit, you will get competences allowing you to design and control the layout of web pages using properties like width, height, margin, padding, border, and box-sizing, as well as how to enhance elements using background styles such as background color, images, and positioning. To ultimately help you promote better organization and reusability of code.

Introductory activity

ISIMBI is creating a webpage for her school project on “Healthy Eating.” She wants the webpage to look attractive and organized. She changes the font style, size, and color of the headings to make them stand out. He also adjusts the paragraph spacing and alignment to make the text easy to read. To highlight important information, She uses bold, italics, and underlines. ISIMBI adds images of fruits and vegetables and creates a table to show the benefits of different foods. She also sets a background color and uploads a background image to make the webpage more visually appealing. Finally, ISIMBI experiments with internal, inline and external CSS to apply consistent styling across her webpage.

- 1) Give one example of layout styling applied to the paragraphs.
- 2) Mention one effect used by ISIMBI to highlight important information.
- 3) What CSS property did ISIMBI use to change the background color of the page?
- 4) Give one example of internal CSS ISIMBI applied.
- 5) Give one example of external CSS ISIMBI could use for consistent styling.
- 6) Identify one HTML element used to organize tabular data on the webpage.
- 7) Explain why using CSS is important for creating an attractive webpage.

6.1 Text formatting

Learning activity 6.1

A. Choose correct answer

- 1) Which CSS property is used to change the font type?
a) font-size b) font-family c) font-style d) text-align
- 2) Which property controls the size of text?
a) font-weight b) font-size c) text-transform d) color
- 3) Which value makes text bold?
a) italic b) underline c) bold d) capitalize
- 4) Which CSS property is used to change text color?
a) background-color b) font-color
c) color d) text-style
- 5) Which property is used to align text to the center?
a) text-style b) align c) text-align d) position

B. Answer with true or False

- 6) Font-weight is used to change text color.
- 7) Text-transform can change text to uppercase.
- 8) Margin is the space inside an element.
- 9) Padding is the space between content and border.
- 10) Letter-spacing controls space between words.

CSS stands for Cascading Style Sheets and is a language used to style and design webpages created with HTML.

Text formatting in CSS refers to the use of CSS properties to control how text appears on a webpage. It helps improve the readability, appearance, and presentation of content.

6.1.1 Font styling

Font styling in CSS refers to changing the appearance of text fonts on a webpage. It controls how text looks in terms of type, size, thickness, and style, making content more attractive and readable.

a. Font family

The font-family property in CSS is used to define the typeface (style of text) that will be displayed on a web page. It helps improve the appearance and readability of content by allowing web developers to choose specific fonts for different elements. More than one font can be specified as a fallback option so that if the first font is not available on the user's device, the browser will use the next available font in the list. It is recommended to end the list with a generic font family such as serif, sans-serif, monospace, cursive, or fantasy.

Syntax:

```
selector {  
  
    font-family: font-name, fallback-font, generic-family;  
  
}
```

Example:

```
p {  
    font-family: Arial, sans-serif;  
}
```

b. Font size

The font-size property in CSS is used to control the size of text displayed on a web page. It determines how large or small characters appear, helping to create a clear visual hierarchy between headings, subheadings, and body text. Appropriate font sizes improve readability and make web content easier to understand. Font sizes can be specified using different units such as pixels (px), percentages (%), em, rem, or points (pt), depending on the design requirements.

Syntax:

```
selector {  
    font-size: value;  
}
```

Example:

```
h1 {  
    font-size: 24px;  
}
```

c. Font weight

The font-weight property in CSS is used to control the thickness or boldness of text. It helps emphasize important words, headings, or sections of content by making the text appear lighter or heavier. The property can use keywords such as normal, bold, bolder, and lighter, or numeric values ranging from 100 (thin) to 900 (extra bold). Using different font weights improves the visual appearance and organization of a web page.

Syntax:

```
selector {  
    font-weight: value;  
}
```

Example:

```
p {  
    font-weight: bold;  
}
```

d. Font style

The font-style property in CSS is used to control the appearance of text by making it normal, italic, or oblique. It is commonly used to emphasize words, quotations, titles, or other important content. Italic text appears slightly slanted, helping it stand out from regular text. This property enhances the visual presentation of a web page and makes certain information more noticeable to readers.

Syntax:

```
selector {  
    font-style: value;  
}
```

Example:

```
p {  
    font-style: italic;  
}
```

e. Font variant

The font-variant property in CSS is used to control special text variations. One of its most common uses is displaying text in small capital letters (small-caps). When the value small-caps is applied, lowercase letters are converted into smaller versions of uppercase letters, while existing uppercase letters remain unchanged. This property is often used for headings, titles, labels, and decorative text to create a distinctive and professional appearance.

Syntax:

```
selector {  
    font-variant: value;  
}
```

Example:

```
p {  
    font-variant: small-caps;  
}
```

f. Font shorthand property

The font shorthand property in CSS is used to combine several font-related properties into a single declaration. Instead of writing separate properties such as font-style, font-weight, font-size, and font-family, they can all be written in one line. This makes CSS code shorter, cleaner, and easier to maintain. When using the shorthand property, font-size and font-family are required, while other properties such as font-style, font-weight, and font-variant are optional.

Syntax:

```
selector {  
    font: font-style font-weight font-size font-family;  
}
```

Example:

```
p {  
    font: italic bold 16px Arial, sans-serif;  
}
```

Example of font styling

```
<!DOCTYPE html>  
<html>
```

```

<head>
<style>
  body {
    font-family: Verdana, sans-serif;
  }
  h1 {
    font-size: 28px;
    font-weight: bold;
  }
  p {
    font-size: 16px;
    font-style: italic;
  }
</style>
</head>
<body>

<h1>ICT tools </h1>
<p>ICT Course is most important for us.</p>

</body>
</html>

```

Output from the above HTTM codes:

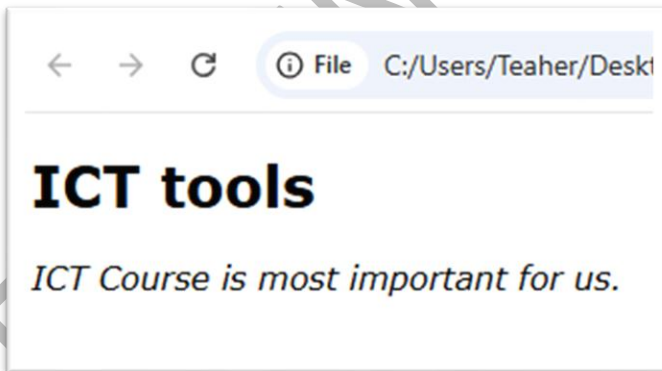


Figure 6. 1. Font styling example

Importance of font styling

- **Improves readability:** Proper font styling makes text easier to read and understand by using appropriate font types, sizes, and spacing,

allowing users to access information comfortably without straining their eyes.

- **Makes content visually appealing:** Well-chosen font styles enhance the overall appearance of a web page, making it more attractive, professional, and engaging for visitors.
- **Helps emphasize important text:** Font styling allows designers to highlight key information through the use of bold, italic, larger text sizes, or other formatting techniques, helping users quickly identify important content.
- **Enhances user experience:** Effective font styling contributes to a positive user experience by making web pages organized, easy to navigate, and pleasant to read across different devices and screen sizes.

Property	Purpose
font-family	Sets font type
font-size	Sets text size
font-weight	Controls boldness
font-style	Italic or normal
font-variant	Small caps

Table 6. 1. Properties and purposes of font styling

6.1.2 Text appearance

Text appearance in CSS refers to how text looks visually on a webpage. It includes properties that control color, decoration, alignment, and transformation of text to make content more attractive and readable

a. Text color

The color property in CSS is used to change the color of text displayed on a web page. It helps improve the appearance of content, attract users' attention, and make information easier to read. Different colors can be used to distinguish headings, paragraphs, links, and other elements. Colors can be

specified using color names such as blue, red, and green, hexadecimal values such as #0000FF, RGB values, or HSL values.

Syntax:

```
selector {  
    color: value;  
}
```

Example:

```
p {  
    color: blue;  
}
```

b. Text alignment

The text-align property in CSS is used to control the horizontal alignment of text within an element. It determines how text is positioned between the left and right margins of its container. Proper text alignment improves the organization, readability, and appearance of web page content. Common alignment values include left, right, center, and justify, each serving different presentation purposes.

Syntax:

```
selector {  
    text-align: value;  
}
```

Example:

```
p {  
    text-align: center;  
}
```

c. Text decoration

The text-decoration property in CSS is used to add decorative lines to text, such as underlines, over lines, or line-through effects. It is commonly used to emphasize text, indicate hyperlinks, or show that content has been removed or completed. This property helps improve the visual presentation of web pages and makes certain information more noticeable to users.

Syntax:

```
selector {  
    text-decoration: value;  
}
```

Example:

```
h1 {  
    text-decoration: underline;  
}
```

d. Text transformation

The text-transform property in CSS is used to control the capitalization of text without changing the original content in the HTML document. It allows web developers to display text in uppercase, lowercase, or capitalized form, helping to maintain a consistent appearance throughout a web page. This property is commonly used for headings, navigation menus, buttons, and other elements where a specific text style is required.

Syntax:

```
selector {  
    text-transform: value;  
}
```

Example:

```
p {  
    text-transform: uppercase;  
}
```

e. Text indentation

The text-indent property in CSS is used to add space at the beginning of the first line of a block of text, such as a paragraph. It helps improve the appearance and readability of written content by clearly indicating the start of a new paragraph. Text indentation is commonly used in articles, reports, books, and other documents where paragraphs need to be visually separated without adding extra blank lines.

Syntax:

```
selector {  
    text-indent: value;  
}
```

Example:

```
p {  
    text-indent: 30px;  
}
```

f. Text shadow

The text-shadow property in CSS is used to add shadow effects to text. It enhances the visual appearance of text by creating depth and making headings, titles, or important content stand out. Text shadows can be customized by adjusting the horizontal position, vertical position, blur radius, and shadow color. This property is commonly used in website banners, logos, headings, and decorative text to create attractive visual effects.

Syntax:

```
selector {  
    text-shadow: horizontal-offset  
vertical-offset blur-radius color;  
}
```

Example of text appearance:

```
<!DOCTYPE html>  
<html>  
<head>  
  <style>  
    h1 {  
      color: darkblue;  
      text-align: center;  
      text-decoration: underline;  
    }  
    p {  
      text-transform: capitalize;
```

Example:

```
h2 {  
    text-shadow: 2px 2px 5px gray;  
}  
  
    text-indent: 20px;  
  }  
</style>  
</head>  
<body>  
<h1>ICT Tool</h1>  
<p> ICT Course is most important for  
us.</p>  
</body>  
</html>
```

The output is:

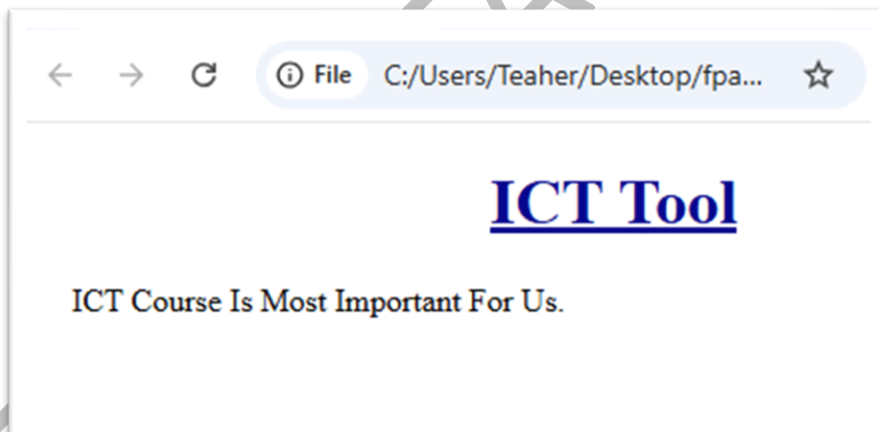


Figure 6. 2. Text appearance

Importance of text appearance

- **Makes text attractive and readable:** Proper text appearance enhances the visual appeal of a web page while ensuring that users can read and understand the content easily without difficulty or eye strain.
- **Helps organize content clearly:** Using different text styles, sizes, colors, and alignments helps structure information in a logical manner,

making it easier for users to identify headings, subheadings, and main content.

- **Highlights important information:** Text formatting techniques such as bold text, color changes, underlining, and text shadows draw attention to key points and important messages that users should notice quickly.
- **Improves user experience:** Well-formatted and visually appealing text makes web pages more engaging, professional, and easy to navigate, encouraging users to spend more time interacting with the content

Property	Purpose
Color	Changes text color
text-align	Aligns text
text-decoration	Adds lines
text-transform	Changes case
text-indent	Indents paragraph
text-shadow	Adds shadow

Table 6. 2. Properties and purposes of text appearance

6.1.3 Spacing and layout

Spacing and layout in CSS refer to how text and elements are arranged and spaced on a webpage. These properties help improve readability, organization, and visual structure of content.

a. Line spacing

It controls the space between lines of text.

```
p {  
  line-height: 1.5;  
}
```

b. Letter spacing

It controls the space between characters (letters).

```
h1 {  
  letter-spacing: 2px;  
}
```

c. Word spacing

It controls the space between words.

```
p {  
  word-spacing: 5px;  
}
```

d. Text alignment

It controls how text is positioned horizontally.

```
p {  
  text-align: justify;  
}
```

e. Margin

It controls the outer space around an element.

```
p {  
  margin: 20px;  
}
```

f. Padding

It controls the inner space inside an element (between border and content).

```
p {  
  padding: 10px;  
}
```

g. Width and height

It use to define the size of elements.

```
div {  
  width: 300px;
```

```
height: 150px;
}
```

Example of spacing & layout

```
<!DOCTYPE html>
<html>
<head>
<style>
  p {
    line-height: 1.6;
    letter-spacing: 1px;
    word-spacing: 4px;
    text-align: justify;
    margin: 20px;
    padding: 10px;
    border: 1px solid black;
  }
</style>
</head>
<body>

<p>
  Spacing and layout in CSS are essential for organizing content and making webpages
  clean, readable, and visually appealing.
</p>

</body>
</html>
```

Output:

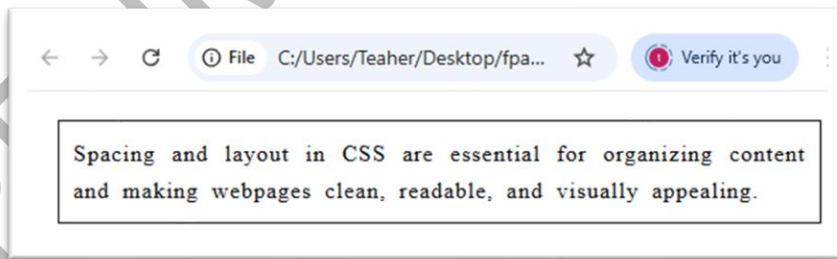


Figure 6. 3. Spacing and layout properties

This paragraph demonstrates spacing and layout properties in CSS.

Importance of spacing & layout

Spacing and layout play an important role in web design by improving readability, making content well-organized and easy to follow, enhancing the overall visual appearance of a web page, and helping to structure information clearly so that users can quickly locate and understand different sections of content.

Property	Purpose
line-height	Space between lines
letter-spacing	Space between letters
word-spacing	Space between words
text-align	Aligns text
margin	Outer space
padding	Inner space
width/height	Element size

Table 6. 3. Properties and purposes of paragraph spacing & layout

6.1.4 Effects

Effects in CSS refer to special styling techniques used to enhance the visual appearance of elements on a webpage. These effects make web pages more attractive, interactive, and engaging for users.

a. Text shadow

It adds a shadow effect to text.

```
h1 {  
  text-shadow: 2px 2px 5px gray;  
}
```

- 2px → horizontal shadow
- 2px → vertical shadow

- 5px → blur effect

b. Box shadow

It adds a shadow around an element (like a box).

```
div {  
  box-shadow: 4px 4px 10px black;  
}
```

c. Hover effects

It changes style when the user moves the mouse over an element.

```
button: hover {  
  background-color: green;  
  color: white;  
}
```

d. Opacity

It controls the transparency of an element.

```
img {  
  opacity: 0.5;  
}
```

e. Transform effects

It is used to move, rotate, scale, or skew elements.

```
div {  
  transform: rotate(20deg);  
}
```

Other examples:

```
transform: scale(1.2); /* zoom */  
transform: translate X(50px); /* move */
```

f. Transition effects

It adds smooth changes between styles.

```
div {  
  transition: all 0.5s;  
}
```

```
div:hover {  
  background-color: blue;  
}
```

Example of CSS effects

```
<!DOCTYPE html>  
<html>  
<head>  
  <style>  
    h1 {  
      text-shadow: 2px 2px 5px gray;  
    }  
    div {  
      width: 200px;  
      height: 100px;  
      background-color: lightgreen;  
      box-shadow: 3px 3px 8px blue;  
      transition: 0.5s;  
    }  
    div:hover {  
      background-color: green;  
      transform: scale(1.1);  
    }  
  </style>  
</head>  
<body>  
<h1>ICT is very important tool</h1>  
<div></div>  
</body>  
</html>
```



Figure 6. 4. CSS effects

Importance of CSS effects

- **Makes webpages more attractive:** CSS effects such as animations, shadows, transitions, and hover effects enhance the visual appearance of a webpage, making it more appealing, modern, and engaging to users who visit the site.
- **Improves user interaction:** CSS effects provide visual feedback when users interact with webpage elements such as buttons, links, menus, and images, helping users understand that their actions have been recognized by the website.
- **Highlights important elements:** Effects such as color changes, animations, glowing borders, and hover styles can be used to draw attention to important content, notifications, buttons, or calls to action that users need to notice quickly.
- **Enhances user experience:** Well-designed CSS effects make webpages more interactive, intuitive, and enjoyable to use, creating a smoother browsing experience that encourages users to explore and engage with the website's content.

Summary table

Effect	Purpose
text-shadow	Adds shadow to text

box-shadow	Adds shadow to elements
Hover	Changes style on mouse over
opacity	Controls transparency
transform	Moves or changes shape
transition	Smooth animation

Table 6. 4. Effects and purposes of CSS effects

Application Activity 6.1

- 1) Create a simple HTML page containing one heading and two paragraphs.
- 2) Apply different font styles by setting the font family, adjusting the font size, making the text bold, and displaying some text in italic.
- 3) Change the text color, underline the heading, and transform the text using either uppercase or capitalize.
- 4) Improve the layout by adjusting the line spacing (line-height), setting the text alignment (text-align) to either center or justify, and adding margin and padding to the paragraphs.
- 5) Apply a text shadow (text-shadow) effect to the heading to make it more visually appealing.
- 6) Save your HTML file and open it in a web browser. Observe and describe how the different CSS styles affect the appearance, readability, and overall presentation of the webpage.

6.2 Layout styling

Learning activity 6.2

A. Choose correct answer

- 1) Which property controls the thickness of a border?
 - a) border-style
 - b) border-width
 - c) border-color
 - d) padding
- 2) Which property adds space inside an element, between the content and the border?
 - a) margin
 - b) padding
 - c) border
 - d) width

- 3) What is the default behavior of a background image?
a) no-repeat b) repeat c) cover d) center
- 4) Which CSS property controls the position of a background image?
a) background-repeat b) background-size
c) background-position d) background-color
- 5) Which property allows you to set all background properties in one line?
a) background b) background-image
c) background-color d) background-style

B. Answer with true or false

- 6) Padding is outside the border of an element.
- 7) Margin is used to control the space outside an element.
- 8) Background-size: cover ensures the image fills the entire element.
- 9) Borders and padding are part of the CSS Box Model.

Layout styling in CSS refers to the use of CSS properties to control the structure, size, spacing, and arrangement of elements on a webpage. It helps organize content so that the page looks neat, well-structured, and visually appealing.

6.2.1 Borders and padding elements

Borders and padding are important CSS properties used to control the appearance and spacing of elements on a webpage. They are part of the CSS Box Model, which defines how elements are displayed.

a. Border

A border is a line that surrounds an HTML element, such as a paragraph, image, table, or division. Borders are used to separate content from other elements, highlight important sections, and improve the overall appearance and structure of a web page. By applying borders, web developers can make elements more visible and organized, creating a cleaner and more attractive layout.

Basic syntax:

```
div {  
    border: 2px solid black;  
}
```

(i) Border Properties

CSS provides several properties for controlling the appearance of borders.

- **Border-width:** The border-width property specifies how thick the border should be. The width can be defined using units such as pixels as px, ems as em, or predefined values such as thin, medium, and thick.
- **Border-style:** The border-style property specifies the appearance or pattern of the border line. Without a border style, the border will not be displayed even if a width and color are specified.
- **border-color:** The border-color property specifies the color of the border. Colors can be defined using color names, hexadecimal values, RGB values, or HSL values.

(ii) Common border styles

- **Solid border:** It consists of one continuous line. It is the most commonly used border style because it provides a clear and professional appearance.
- **Dashed border:** Is made up of a series of short line segments separated by spaces. It is often used to indicate editable areas, placeholders, or special sections.
- **Dotted border:** It consists of a series of dots. It is commonly used for decorative purposes or to create a lighter visual effect than a solid border.
- **Double border:** It displays two parallel lines around an element. It is often used to emphasize important content or create a decorative effect.

b. Padding

Padding is the space inside an HTML element, located between the element's content and its border. It creates empty space around the content, preventing text, images, or other elements from touching the border directly. Padding improves the readability and appearance of content by making it look less

crowded and more organized. It is commonly used in buttons, paragraphs, forms, and containers to provide a comfortable amount of space around the content.

When padding is increased, the content moves farther away from the border, making the element appear larger and more visually appealing.

Syntax:

```
selector {  
    padding: value;  
}
```

Example:

```
div {  
    padding: 10px;  
}
```

Padding properties

CSS allows padding to be applied to specific sides of an element:

- padding-top – Adds space at the top of the content.
- padding-right – Adds space on the right side of the content.
- padding-bottom – Adds space at the bottom of the content.
- padding-left – Adds space on the left side of the content.

Example:

```
div {  
    padding-top: 10px;  
    padding-right: 15px;  
    padding-bottom: 10px;  
    padding-left: 15px;  
}
```

c. Difference between border and padding

Feature	Border	Padding
---------	--------	---------

Meaning	Outer line around element	Space inside element
Position	Outside content	Between content and border
Purpose	Defines boundary	Adds inner spacing

Table 6. 5. Difference between border and padding

Example of border and padding

```
<!DOCTYPE html>
<html>
<head>
<style>
  div {
    border: 2px solid black;
    padding: 15px;
    width: 300px;
  }
</style>
</head>
<body>
<div>
  ICT is a very essential subject.
</div>
</body>
</html>
```

Output: This box has a border and padding.

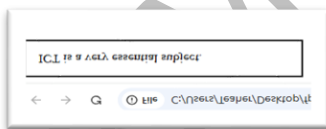


Figure 6. 5. Border and padding

6.2.2 Background of elements

The background of elements in CSS refers to how the area behind the content of an element is styled. It can include colors, images, and positioning, making webpages more attractive and visually engaging.

a. Background color

The background-color property is used to set the color that appears behind the content of an HTML element. It helps improve the visual appearance of a web page, highlight important sections, and make content easier to read. The background color fills the entire area of the element, including its content and padding. Different colors can be applied to different elements such as headings, paragraphs, tables, and divisions (div).

Example:

```
div {  
    background-color: lightblue;  
}
```

In this example, the background of the <div> element will appear in a light blue color.

You can specify colors using different formats:

Color names: red, blue, green, yellow

HEX values: #ff0000, #0000ff, #00ff00

RGB values: rgb(255, 0, 0), rgb(0, 0, 255)

Example using HEX value:

```
div {  
    background-color: #ffcc00;  
}
```

Example using RGB value:

```
div {  
    background-color: rgb(173, 216, 230);  
}
```

b. Background image

The background-image property is used to add an image as the background of an HTML element. Instead of displaying a solid color, the specified image is shown behind the element's content. Background images can be used to improve the visual appearance of a web page, create attractive designs, and

provide additional context or branding. The image is placed behind the text and other content within the element.

Example:

```
div {  
  background-image: url("image.jpg");  
}
```

In this example, the image named image.jpg will be displayed as the background of the <div> element.

c. Background repeat

The background-repeat property is used to control how a background image is repeated within an element. When a background image is smaller than the element, the browser repeats the image to fill the available space. This property allows you to specify whether the image should repeat horizontally, vertically, both directions, or not repeat at all.

Example: div {

```
  background-repeat: no-repeat;  
}
```

In this example, the background image will be displayed only once and will not be repeated.

Values of background-repeat

- Repeat (default value): The image repeats both horizontally and vertically until the entire element is covered.

```
div {  
  background-repeat: repeat;  
}
```

- No-repeat: The image is displayed only once.

```
div {
```

background-repeat: no-repeat;

}

- Repeat-x: The image repeats only horizontally (from left to right).

div {

background-repeat: repeat-x;

}

- Repeat-y: The image repeats only vertically (from top to bottom).

div {

background-repeat: repeat-y;

}

d. Background size

The background-size property is used to control the size of a background image. It determines how the image is displayed within an element and whether it should be stretched, enlarged, or reduced to fit the available space. This property is especially useful when creating responsive web pages because it helps background images display properly on different screen sizes.

Example: *div {*

background-size: cover;

}

In this example, the background image will be resized to cover the entire <div> element.

Common values of background-size

- Cover: The image is resized to completely cover the element. If the image and element have different proportions, some parts of the image may be cropped.

div {

```
background-size: cover;  
}
```

- **Contain:** The image is resized so that the entire image fits inside the element. No part of the image is cropped, but empty spaces may appear if the image and element have different proportions.

```
div {  
  background-size: contain;  
}
```

e. Background position

The background-position property is used to specify the starting position of a background image within an HTML element. By default, a background image is placed in the top-left corner of an element. This property allows you to move the image to different positions such as the center, top, bottom, left, or right. It is particularly useful when using large background images and only a specific part of the image needs to be visible.

Example:

```
div {  
  background-position: center;  
}
```

In this example, the background image is positioned at the center of the <div> element.

Common Values of background-position

- **Center:** Places the image in the center of the element.

```
div {  
  background-position: center;  
}
```

- Top: Places the image at the top of the element.

```
div {
  background-position: top;
}
```

- bottom: Places the image at the bottom of the element.

```
div {
  background-position: bottom;
}
```

- left: Places the image on the left side of the element.

```
div {
  background-position: left;
}
```

- right: Places the image on the right side of the element.

```
div {
  background-position: right;
}
```

f. Background shorthand property

The background property is a shorthand property that combines several background properties into a single declaration. Instead of writing separate lines for background-color, background-image, background-repeat, background-position, and other background-related properties, you can write them all in one line. This makes the CSS code shorter, cleaner, and easier to manage.

```
div {
  background: lightblue url("image.jpg") no-repeat center;
}
```

Example of background styling

```
<!DOCTYPE html>
<html>
<head>
<style>
  div {
    width: 300px;
```

```

    height: 150px;
    background-color: lightgreen;
    background-image: url("image.jpg");
    background-repeat: no-repeat;
    background-size: cover;
    background-position: center;
}
</style>
</head>
<body>

<div>
    Background styling example
</div>

</body>
</html>

```

Output:

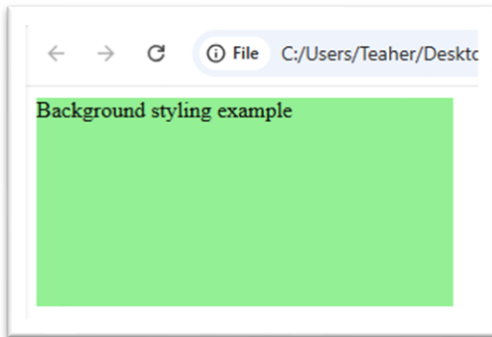


Figure 6. 6. Background styling

Application activity 6.2

- 1) Create a <div> element and give it a border, padding, and margin.
- 2) Apply a background color to one element.
- 3) Add a background image to another element and adjust its size and position.

6.3 Styles with internal CSS

Learning activity 6.3

Choose a correct answer:

- 1) The <style> tag is placed in the:
a) <body> b) <footer> c) <head> d) <header>
- 2) Internal CSS applies to:
a) Only the page where it is written b) All pages in the website
c) External style sheets d) JavaScript files
- 3) Which of the following is a disadvantage of internal CSS?
a) Easy to test styles b) Cannot reuse styles on multiple pages
c) Quick to implement d) Styles are in one file
- 4) The syntax .highlight { background-color: yellow; } is an example of:
a) ID selector b) Class selector
c) Element selector d) Inline CSS

Answer with True or False

- 5) Internal CSS requires a separate CSS file to work.
- 6) Internal CSS allows styling multiple elements on a single page.
- 7) Using internal CSS for large websites is recommended.

Fill in the blanks with the following words: current, CSS styles, dot

- 8) The <style> tag is used to define _____.
- 9) Styles defined in internal CSS apply only to the _____ HTML page.
- 10) A class selector starts with the _____ symbol in CSS.

Internal CSS refers to writing CSS rules inside an HTML document using the <style> tag, which is placed within the <head> section of the HTML page. This method allows you to style a single HTML page without using an external file.

a. Structure of internal CSS

```
<!DOCTYPE html>
<html>
<head>
<style>
/* CSS rules go here */
body {
  background-color: lightblue;
}
h1 {
  color: darkblue;
  text-align: center;
```

```

}
p {
  font-size: 16px;
  font-style: italic;
}
</style>
</head>
<body>
<h1>Welcome to ICT Class</h1>
<p>This is an example of internal CSS styling.</p>
</body>
</html>

```

Output:

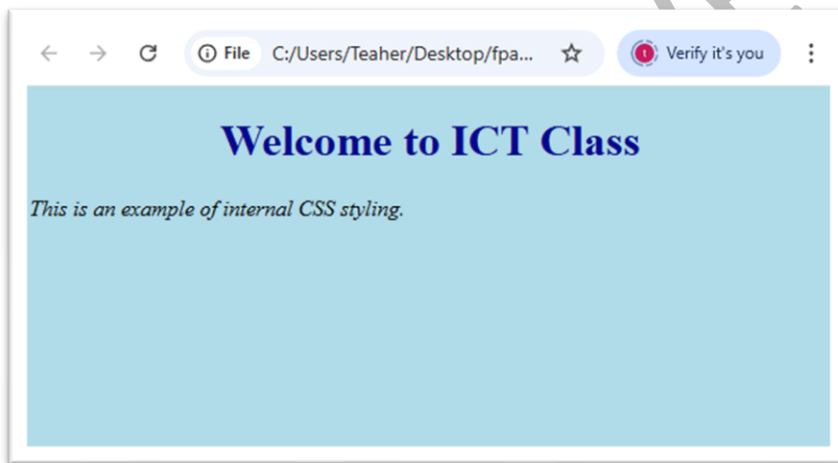


Figure 6. 7. Internal CSS styling

b. How it works

The `<style>` tag contains all the CSS rules for a web page. These rules define how HTML elements should appear, including their colors, fonts, sizes, spacing, borders, and backgrounds. The browser reads the CSS rules inside the `<style>` tag and applies them to the corresponding HTML elements when the page is loaded.

The `<style>` tag is usually placed inside the `<head>` section of an HTML document. This allows the browser to load and apply the styles before displaying the page content, resulting in a better user experience.

Rules written inside the <style> tag apply only to that specific HTML page. They cannot automatically affect other web pages. If the same styles are needed on multiple pages, an external CSS file should be used instead.

CSS works by matching selectors with HTML elements. A selector identifies the element or group of elements that should receive the specified styles.

You can style elements using different types of selectors:

Element selectors (p, h1, div) apply styles to all elements of a particular type.

Class selectors (.className) apply styles to all elements that share the same class name. ID selectors (#idName) apply styles to a single element with a unique ID. Each CSS rule consists of a selector followed by a set of declarations enclosed in curly braces { }. A declaration contains a property and a value, separated by a colon (:).

Example:

```
<style>
```

```
p {
```

```
  color: blue;
```

```
  font-size: 18px;
```

```
}
```

```
</style>
```

Example of internal CSS with multiple elements

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
  body {
```

```
    background-color: #f0f8ff;
```

```
    font-family: Arial, sans-serif;
```

```
  }
```

```
  h1 {
```

```
    color: #003366;
```

```

    text-align: center;
    text-decoration: underline;
}

p {
    color: #333333;
    font-size: 16px;
    line-height: 1.5;
}

.highlight {
    background-color: yellow;
    font-weight: bold;
}
</style>
</head>
<body>

<h1>Introduction to Internal CSS</h1>
<p>This paragraph is styled using internal CSS.</p>
<p class="highlight">This paragraph is highlighted with a yellow background.</p>

</body>
</html>

```

Output:

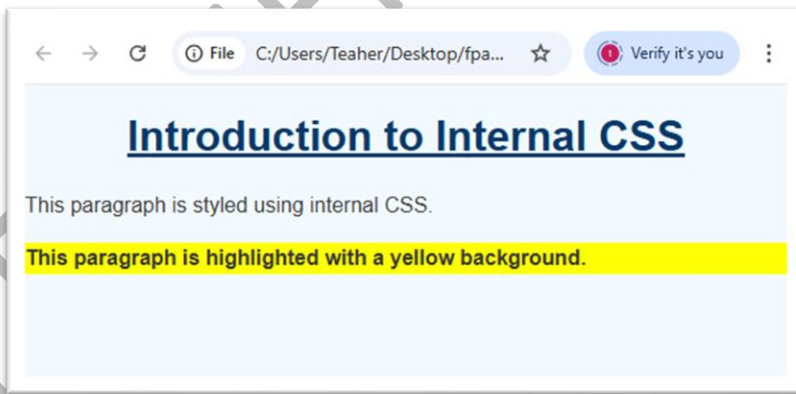


Figure 6. 8. Internal CSS with multiple elements

c. Advantages and disadvantages of internal CSS

Advantages

Disadvantages

- Easy to implement for single-page websites.
- Styles are contained in one HTML file, no external file needed.
- Quick testing and experimentation of styles.
- Styles cannot be reused on other pages.
- Makes HTML code larger and harder to maintain for big websites.
- Not suitable for websites with multiple pages.

Application activity 6.3

- 1) Create an HTML file and apply internal CSS to style elements as follows:
 - a) Add a <h1> heading with the text “My Webpage”.
 - b) Set the text color to dark blue.
 - c) Center align the heading.
 - d) Underline the heading.
 - e) Add two paragraphs below the heading: First paragraph: font size 16px, italic, text color gray and second paragraph: font size 18px, bold, background color light yellow (use a class).
 - f) Apply a background color to the <body> element.
 - g) Save the file and open it in a browser. Observe how internal CSS affects the page.

6.4 Styles with external CSS

Learning activity 6.4

Choose a correct answer:

- 1) Which file extension is used for external CSS files?
 - a) .html
 - b) .js
 - c) .css
 - d) .txt
- 2) The href attribute in the <link> tag specifies:
 - a) The type of stylesheet
 - b) The path to the CSS file
 - c) The title of the HTML page
 - d) The style of headings

- 3) Which of the following is an advantage of external CSS?
- a) Styles can be reused on multiple pages
 - b) Styles apply only to one page
 - c) Makes HTML files longer
 - d) Requires inline styles
- 4) If the external CSS file is mislinked, what happens?
- a) The page still displays the styles
 - b) The page appears unstyled
 - c) The page will not load
 - d) Only headings are styled

Answer with true or false

- 5) External CSS is stored inside the <style> tag of an HTML page.
- 6) External CSS allows styling multiple pages using a single file.
- 7) Using external CSS makes website maintenance easier.

Fill in the blanks with the following words: relationship, .css, head

- 8) External CSS rules are written in a separate _____ file.
- 9) The HTML <link> tag is placed inside the _____ section.
- 10) The rel attribute in <link> specifies the _____ of the file.

External CSS refers to writing CSS rules in a separate .css file that is linked to one or more HTML pages. This method allows you to apply the same styles across multiple web pages, making it easier to maintain and update the website.

An external CSS file is linked to an HTML page using the <link> tag. The href attribute specifies the CSS file location, while rel="stylesheet" identifies it as a style sheet. Any changes made in the CSS file automatically apply to all linked web pages.

a. Syntax of external CSS

```
/* style.css */
body {
    background-color: lightblue;
    font-family: Arial, sans-serif;
}
h1 {
    color: darkblue;
    text-align: center;
}
p {
```

```
font-size: 16px;
font-style: italic;
}
```

b. Example of external CSS

style.css

```
body {
  background-color: #0f8ff;
  font-family: Verdana, sans-serif;
}
h1 {
  color: #003366;
  text-align: center;
  text-decoration: underline;
}
p {
  color: #333333;
  font-size: 16px;
  line-height: 1.5;
}
```

index.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Example of external CSS.</title>
  <link rel="stylesheet" type="text/css" href="style.css">
</head>
<body>
<h1>Example of external CSS.</h1>
<p>This paragraph uses styles from an external CSS file.</p>
<p>Lorem ipsum dolor sit amet consectetur adipisicing elit.<br> Dolorum alias,
doloremque quos natus accusantium excepturi vel et<br> nam quisquam impedit amet
unde repudiandae atque dolor perferendis modi,<br> delectus maiores qui nam
quisquam impedit amet unde repudiandae.</p>
</body>
</html>
```

c. Advantages and disadvantages of external CSS

Advantages

Disadvantages

- Styles can be reused on multiple pages, promoting consistency.
- Makes HTML files cleaner, separating content from presentation.
- Easier to update or maintain large websites.
- Requires an extra file to be loaded, which may slightly increase page load time.
- If the CSS file is missing or mislinked, styles will not apply.

Feature	Description
CSS File	Stored separately with .css extension
Link	<code><link rel="stylesheet" href="filename.css"></code>
Scope	Can style multiple pages
Advantages	Reusable, cleaner HTML, easier maintenance
Disadvantages	Needs extra file, page may look unstyled if file is missing

Table 6. 6. Features and descriptions of external CSS

Application activity 6.4

Create an HTML file and a separate CSS file on external CSS

1) Add the following styles:

- 1) Set the background color of the body.
- 2) Style `<h1>` with a text color, underline, and center alignment.
- 3) Style `<p>` with a different text color, font size, and line spacing.
- 4) Create a class for highlighted text with a background color and bold font.

2) Create a file named index.html.

3) Include the `<link>` tag in the `<head>` section to link style.css.

4) Add the following elements in the `<body>`:

- a) <h1> heading with any text
- b) Two paragraphs, one using the highlighted class

6.5 Styles with inline CSS

Learning activity 6.5

Choose the correct answer:

1) Which of the following is correct Inline CSS syntax?

- a) <p color="red">Text</p>
- b) <p style="color: red;">Text</p>
- c) <p css="color:red;">Text</p>
- d) <p style-red>Text</p>

2)Inline CSS is written:

- a) In a separate CSS file
- b) Inside the HTML tag using the style attribute
- c) Inside JavaScript
- d) In the head only

3)Which attribute is used for Inline CSS?

- a) class
- b) id
- c) style
- d) css

Practical question

4) Write HTML code that display word “Welcome to Web Design”, changes the font size to 24px and aligns the text to the center use the text.

5) Create HTML webpage using inline CSS. The webpage should contain a blue heading, a paragraph with a font size of 20px, gray background color, and a centered heading.

6) Study the code below and answer the questions.

```
<p style="color: blue;">Welcome to CSS</p>
```

- a) Which attribute is used for Inline CSS?
- b) Which property is used?
- c) What is the value of the property?

Inline CSS is a method of applying CSS directly inside an HTML tag using the style attribute.

a. Syntax of Inline CSS

`<tag style="property: value;">Content</tag>`

Example1:

```
<html>
<head>
<title>Inline CSS Example</title>
</head>
<body style="width: 500px;height: 100px ;border-radius:700px,100px,700px ;box-
shadow: 70px;text-align: center;">
<h1 style="color: blue; text-align:center;">
Welcome to CSS
</h1>
<p style="font-size:18px; background-color:lightgray;">
Inline CSS is used inside HTML tags.
</p>
<p style="background-color: cadetblue; color: black;">
  <b>
    Welcome to our website.<br>
    This paragraph shows how inline CSS can be used to style text.<br>
    The background color is cadet blue and the text color is black.<br>
    Inline CSS applies styles directly inside the HTML tag.<br>
    It is easy to use for small webpages and simple designs.<br>
    Learning HTML and CSS helps you create attractive web pages.
  </b>
</p>
</body>
</html>
```

Output:

Welcome to CSS

Inline CSS is used inside HTML tags.

Welcome to our website.
This paragraph shows how inline CSS can be used to style text.
The background color is cadet blue and the text color is black.
Inline CSS applies styles directly inside the HTML tag.
It is easy to use for small webpages and simple designs.
Learning HTML and CSS helps you create attractive web pages.

Figure 6.9 Styling paragraph by using inline css

Example2: <!DOCTYPE html>

```
<html>
<head>
<title>Student Registration Form</title>
</head>
<body style="background-color:lightblue;">
<h1 style="text-align:center; color:darkblue;">
Student Registration Form
</h1>
<form style="background-color:white;
width:300px;
margin:auto;
padding:20px;
border:2px solid black;">
<label style="font-size:18px;">Full Name:</label><br>
<input type="text"
style="width:250px;
padding:8px;
border:1px solid gray;"><br><br>
<label style="font-size:18px;">Email:</label><br>
<input type="email"
style="width:250px;
padding:8px;
border:1px solid gray;"><br><br>
<label style="font-size:18px;">Password:</label><br>
<input type="password"
style="width:250px;
padding:8px;
border:1px solid gray;"><br><br>
<input type="submit"
value="Register"
style="background-color:green;
```

```

        color:white;
        padding:10px;
        border:none;
        font-size:16px;">
</form>
</body>
</html>

```

Figure 6.10 Designing student registration form by using inline css

b. Advantages of inline css

Inline CSS is easy to use because the CSS code is written directly inside the HTML tag, making it suitable for beginners who are learning web design. It allows developers to quickly apply styles to individual elements without creating a separate CSS file. For small webpages or simple projects, inline CSS can save time since only a few styles may be needed. Additionally, all styling is contained within the HTML file itself, eliminating the need for a separate CSS document and making simple webpages easier to manage.

Application activity 6.5

A. Multiple choice question

1) What does this code create?

```
<input type="submit" style="background-color: green; color: white;">
```

- | | |
|-----------------------------------|--------------------|
| a) A green button with white text | b) A red input box |
| c) A table | d) A hyperlink |

2) Which Inline CSS property is used to center a form on a page?

- a) text-align
- b) margin:auto
- c) align:center-text
- d) position:center

3) Which Inline CSS property changes button color?

- a) text-color
- b) background-color
- c) button-color
- d) font-color

B. Practical questions

4) Write HTML tags to style form elements using Inline CSS and apply the following styles:

a) Input fields:

Width: 250px, font size: 16px, and padding: 8px.

b) Submit button:

Background color: green, text color: white, font size: 18px, and no border.

c) Heading: The heading is center-aligned and has a red color.

5)a) Create an HTML form containing: Full Name input box, email input box, password input box and submit button

b) Apply an Inline CSS whereby the form background color is white, the form width is 300px and centered on the page, add border around the form (2px solid black) and add padding inside the form (10px).

6.6 End of unit assessment

6.6.1 Project activity

Project title: Design and Development of a School Club Website Using Cascading Style Sheets (CSS)

Project description

Your school administration would like to promote different school clubs through a website. As a web designer, your task is to create a webpage for one school club of your choice (ICT Club, Debate Club, Environment Club, Sports Club, Music Club, Science Club, etc.). The webpage should provide information about

the club and use CSS to make the page visually appealing, organized, and easy to read.

Choose one school club and collect information about it. Your webpage should contain: club name, club logo or image, introduction to the club, vision and mission of the club, club activities, benefits of joining the club, contact information and footer section.

1) Create an HTML file named: index.html.

Use appropriate HTML elements to organize your content. Your page should include headings, paragraphs, lists, images, sections or div containers. Ensure that all information is arranged logically.

2) Create one section of the webpage using internal CSS and use external CSS by creating a separate CSS file named "style.css". Move your CSS rules into this file and link it to the HTML page using: `<link rel="stylesheet" href="style.css">`

- a) Use CSS to format text by applying: Font family, Font size, Font weight (bold), Font style (italic), Text color, Text alignment, Text decoration and Text transformation. Make headings different from normal paragraphs.
- b) Use CSS properties to improve the layout of your webpage. Apply Margin, Padding, Width, Height, Text spacing, Line spacing and Letter spacing. Ensure that the webpage is neat and easy to read.
- c) Use CSS to style webpage sections by applying: Borders, Border colors, Border styles, Background colors and Background images. Different sections should be visually distinct.
- d) Apply at least two of the following CSS effects: Text shadow, Box shadow, Hover effect, Opacity, Transform effect or Transition effect. These effects should improve the appearance and user interaction of the webpage.

6.6.2 End of unit assessment

Choose the correct answer.

- 1) What does CSS stand for?
 - a) Computer Style Sheets
 - b) Cascading Style Sheets
 - c) Creative Style System
 - d) Colorful Style Sheets
- 2) Which CSS property is used to change text color?
 - a) font-color
 - b) text-color
 - c) color
 - d) background
- 3) Which of the following is used for external CSS?
 - a) <style>
 - b) <script>
 - c) <link>
 - d) <css>
- 4) Which CSS property is used to align text?
 - a) text-style
 - b) text-align
 - c) align-text
 - d) position
- 5) Where is internal CSS written?
 - a) Inside <body>
 - b) Inside <head>
 - c) Outside HTML
 - d) Inside <footer>

Answer with True or False

- 6) CSS is used to design the structure of a webpage.
- 7) External CSS can be used on multiple pages.
- 8) Margin controls inner spacing.
- 9) Text-shadow adds shadow to text.
- 10) The <link> tag is used to connect external CSS.
- 11) Font-size controls text color.
- 12) Background-size cover fills the entire element.
- 13) Internal CSS is reusable across many pages.
- 14) Hover effect works when the mouse moves over an element.

Answer questions

- 15) State two roles of CSS in web design.
- 16) Explain how you would respect users when designing web pages with CSS styles.
- 17) Explain the difference between internal CSS and external CSS.
- 18) How can creativity improve the design of a webpage using CSS?
- 19) State two best practices when applying CSS in web development.
- 20) Create an HTML webpage and apply CSS styles as follows:
 - a. Add a heading and two paragraphs
 - b. Change font type, size, and color
 - c. Align text and adjust spacing
 - d. Add margin and padding
 - e. Apply a background color

f. Add one CSS effect (shadow or hover)



EXPERIMENTAL VERSION

UNIT 7: DATABASE DESIGN



Key unit competence: Design a normalized relational model using entity-relationship diagrams and appropriate keys

At the end of this unit you will be able to:

Knowledge and understanding:

- Explain different database access levels
- Explain the difference between primary keys and foreign keys.
- Explain different types of relationships
- Describe normalization and its importance in database design.

Skills

- Create tables with appropriate data types and constraints
- Establish relationships between tables
- Design Entity Relationship diagrams to represent database structures.
- Normalize data structures up to third Normal Form.

Attitudes and values

- Appreciate the importance of well-structured databases
- Demonstrate accuracy and consistency when designing a database.
- Develop critical thinking skills when analyzing data requirements and designing solutions.

7.0 Introduction

In this unit, you will learn how databases are used to store, organize, and manage information efficiently. You will explore important database concepts such as primary keys, foreign keys, and different types of relationships between data. You will learn how to design Entity Relationship Diagrams, create relational database tables using SQL, and organize data through normalization. In short, you will get the skills enabling you to design and manage a simple relational database system for real-life situations such as managing school records.

Introductory activity

SENTORE is a student who keeps records of his classmates using a computer. He stores their names, ages, and marks in a table. Each student has a unique student ID. SENTORE also connects this table to another table that stores subjects and marks. His teacher asks him to organize the data properly and draw a simple diagram to show how the data is related.

- 1) What is SENTORE using to store information?
- 2) Give one example of data stored in the table.
- 3) What is the purpose of the student ID?
- 4) What is the name of the structure used to store data in rows and columns?
- 5) What is the relationship between students and marks?

7.1 Database concepts

Learning activity 7.1

A. Choose the correct answer

- 1) A database is:
 - a) A collection of programs
 - b) An organized collection of related data
 - c) A type of hardware
 - d) A web browser
- 2) Who has full control over a database?
 - a) Guest user
 - b) Standard user
 - c) Database administrator
 - d) Student
- 3) Which access level allows a user to modify existing data?
 - a) Read
 - b) Write
 - c) Update
 - d) Delete

- 4) Physical independence means:
- | | |
|---|---------------------------|
| a) Data is stored physically | b) Users can change data |
| c) Changes in storage do not affect users | d) Data is always visible |
- 5) In a relational database model, data is stored in:
- | | | | |
|----------|-----------|-------------|-----------|
| a) Files | b) Tables | c) Programs | d) Images |
|----------|-----------|-------------|-----------|

B. Answer with True or False

- 6) A primary key can have duplicate values.
7) A guest user can delete data.
8) Tables in a relational model contain rows and columns.

C. Answer the following questions

- 9) Differentiate a primary key and foreign key
10) State one importance of access levels.

A database is an organized collection of related data that is stored and managed in a computer system so that it can be easily accessed, updated, and used.

Examples: A school database stores student names, marks, and classes.
Hospital database stores patient records.

7.1.1 Database access levels and users

Database access levels refer to the different permissions or rights given to people when they use a database. Not everyone is allowed to do the same actions, some can only view data, while others can add, edit, or delete it.

If all users were given the same access level, it could lead to serious problems in the database. Unauthorized users might accidentally or intentionally modify, delete, or misuse important data, resulting in data loss, errors, security breaches, and reduced data integrity. Different access levels help protect information by ensuring that users can only perform tasks that are relevant to their roles and responsibilities

a. Types of database users

Database administrator: He is someone that has full control of the database, he can create, update, delete, and manage data and control user access and security.

Example: School IT manager managing student records

- **Standard user:** A standard user can view and modify specific data within the database but has limited permissions. For example, a teacher may be allowed to enter and update students' marks but cannot change the database structure or manage user accounts.
- **Guest/User :** A guest user has the lowest level of access in a database. This user can only view information stored in the database and cannot add, edit, or delete any data. Read-only access helps protect data from accidental or unauthorized changes.

For example a student checking examination results from the school database.

b. Access levels

Access levels are the specific rights granted to users to determine what actions they can perform in a database. Different users are assigned different permissions based on their roles and responsibilities to ensure data security and proper management.

c. Different users are given different levels of access

- **Read (view only):** Allows a user to view and retrieve information stored in the database without making any changes. This permission is useful for users who only need to access information for reference purposes.
- **Write (add):** Allows a user to enter new records into the database.

For example, a teacher can add students' marks, or a receptionist can register new patients in a hospital database.

- **Update (edit):** Allows a user to modify or correct existing data in the database. This permission is important when information changes or errors need to be corrected.
- **Delete:** Allows a user to permanently remove records from the database. Because deleting data can result in the loss of important information, this

permission is usually restricted to authorized users such as database administrators.

d. Importance of access levels

- **Protects data from unauthorized changes:** Access levels ensure that only authorized users can modify or delete data. For example, a student can view examination results but cannot change marks stored in the database.
- **Ensures data security:** By limiting access to sensitive information, access levels help prevent unauthorized users from viewing confidential data such as financial records, personal information, or examination results.
- **Prevents data loss or errors:** Restricting permissions reduces the risk of accidental deletion, modification, or corruption of important data. Only users with the appropriate rights can make changes to records.
- **Helps manage different users efficiently:** Access levels allow each user to perform only the tasks related to their role. This improves organization, accountability, and the overall efficiency of database operations.

In a school database Admin manages the whole system, Teacher enters and updates marks and Student views results only

7.1.2 Physical independence

Physical independence is a concept in database management that refers to the ability to change the physical storage of data without affecting how the users interact with the database. In simpler terms, it means that users and applications don't need to know how the data is physically stored.

a. Why it is important

- **Flexibility:** The database storage can be optimized without changing applications.
- **Security:** Users do not need to know the physical details of data storage.
- **Maintenance:** Database administrators can reorganize or improve storage structures without affecting programs or users.

b. Databases separate the logical view of data from the physical storage:

- **Logical level (User view):** The logical level describes how users see and interact with data in the database. At this level, users view data in the form of tables, rows, columns, forms, and queries without needing to know how the data is physically stored. The logical level focuses on the structure and organization of the data that users work with.

For example, a teacher may see a Student Marks table containing student names, subjects, and marks, without knowing where or how that information is stored on the computer.

- **Physical level (Storage view):** The physical level describes how data is actually stored and organized inside the computer system. It includes storage structures such as files, indexes, storage blocks, and disk locations that are used to store and retrieve data efficiently. Database administrators manage this level to improve performance and storage efficiency.

For example, although a teacher sees a single Student Marks table, the database may store the information across several files and use indexes to speed up data retrieval.

c. Data independence

Data independence is the ability to change one level of a database system without affecting another level. Physical independence ensures that changes at the physical level like moving data to a new server, or changing indexing method do not affect the logical view.

- **Physical data independence:** This ensures that changes made at the physical level, such as moving data to a new storage device, changing file structures, or creating indexes, do not affect the logical view seen by users. Users continue to access the data in the same way even though the storage method has changed.

- **Logical data independence:** This ensures that changes made at the logical level, such as adding a new field, creating a new table, or modifying relationships between tables, do not affect the user applications and views that do not depend on those changes. This allows the database structure to evolve while minimizing the impact on users and application programs.

7.1.3 Relational database model

A Relational Database Model is a database model that organizes data into tables called relations. Each table consists of rows and columns, making it easier to store, organize, retrieve, and manage related information efficiently. The relational model is based on the idea that data can be grouped into separate tables and linked together through common fields.

In a relational database, each table stores information about a specific subject. For example, a school database may contain a Students table, a Teachers table, and a Marks table. Each table contains records (rows) and attributes (columns) that describe the data being stored.

The relational database model uses relationships to connect data stored in different tables. These relationships help reduce data duplication and improve data consistency. Instead of storing the same information repeatedly, related data can be linked using keys.

a. Main components of a Relational Database Model

- **Table:** A table is a collection of related data organized into rows and columns. Each table represents a specific entity such as students, teachers, or courses. Table is also called relation .

For example: Student table

StudentID	Name	Class
101	Alice	S4
102	John	S4

Table 7. 1 Student table

- **Row:** A row represents a single record in a table. Each row contains information about one item or entity. Row is also called Record or Tuple.
For example: The row (101, Alice, S4) represents one student record.
- **Column:** A column represents a particular characteristic or property of the data stored in a table. Column is also called attribute or field. For example, In the Students table, StudentID, Name, and Class are columns.
- **Relationship:** A relationship is a connection between two or more tables. Relationships allow information stored in different tables to be linked together.
For example, a StudentID in the Students table can be linked to a StudentID in the Marks table to show each student's marks.

b. Advantages of the Relational Database Model

- **Data is organized in a simple tabular format:** Data is stored in tables made up of rows and columns, making it easy for users to understand, enter, and manage information. The tabular structure resembles spreadsheets, which are familiar to many users.
- **Reduces duplication of data:** Information is stored only once and linked through relationships between tables. This minimizes unnecessary repetition of data, saves storage space, and makes database maintenance easier.
- **Enhances data security through access controls:** Different users can be assigned different access rights according to their roles.
- **Improves data consistency and accuracy:** Since data is stored in a structured manner and duplication is reduced, changes made to a record are reflected wherever that data is used. This helps maintain accurate and up-to-date information throughout the database.
- **Makes data retrieval easier through queries:** Users can quickly search, filter, sort, and retrieve specific information using database queries.

For example, a school can easily retrieve all students in a particular class or identify students who scored above a certain mark.

Supports relationships between different tables: Related information can be stored in separate tables and connected using keys. This allows users to combine data from multiple tables without repeating information.

7.1.4 Primary key

A primary key is a special column or set of columns in a table that uniquely identifies each row in that table. It ensures that no two rows have the same value in the primary key columns, which helps maintain data integrity.

Example of primary key

Students table

StudentID	Name	Age
101	Kalisa	16
102	Uwase	17
103	Rukundo	16

Table 7. 2 Primary key example

Here, StudentID is the primary key because it uniquely identifies each student. You cannot have two students with StudentID = 101.

a. Key features of a primary key

- **Uniqueness:** Every value in the primary key column must be unique. For example two students cannot have the same StudentID.
- **Non-null:** A primary key column cannot have a NULL value because it must always identify a record.
- **Immutable :** The value of a primary key should rarely or never change.
- **Single or composite:** A primary key can consist of one column as simple primary key or multiple columns as composite primary key to uniquely identify a row.

b. Importance of primary key

A primary key is very important in a database because it helps maintain the integrity and organization of data. Its importance can be explained as follows:

- **Prevents duplicate records in a table:** A primary key ensures that each record in a table is unique. Since no two records can have the same primary key value, it prevents the accidental entry of duplicate data.
- **Provides a reference point for linking other tables through foreign keys:** A primary key allows different tables in a database to be connected. Other tables can use the primary key as a foreign key to create relationships between related data.
- **Ensures data consistency and reliability in the database:** By uniquely identifying each record, a primary key helps ensure that data remains accurate and consistent. It allows users to retrieve, update, or delete the correct record without confusion. This improves the reliability of the database and reduces errors in data management.

7.1.5 Foreign key

A foreign key is a column or set of columns in one table that is used to link that table to another table. It refers to the primary key in another table by creating a relationship between the two tables. It connects related tables, maintains data consistency, prevents invalid or orphan records and supports relational database structure.

Example of foreign key

Students table

StudentID (PK)	Name
101	Kalisa
102	Uwase

Table 7. 3. Foreign key example

Marks table

MarkID	StudentID (FK)	Score
1	101	85
2	102	90

Table 7. 4. Marks table

StudentID in the Marks table is a foreign key and it refers to StudentID in the Students table (primary key).

You cannot add a record in the Marks table with a StudentID that does not exist in the Students table. For example: StudentID = 999 is not allowed because it does not exist in Students table. Where PK stands for Primary Key and FK stands for Foreign Key

a. Key features of a foreign key

- It must match a value in the primary key of another table: A foreign key can only contain values that already exist in the primary key of the related table, ensuring that relationships between tables are valid.
- Helps connect related data in different tables: A foreign key establishes a relationship between tables, making it possible to link and retrieve related information from multiple tables.
- Prevents invalid data from being entered by ensuring that every foreign key value corresponds to an existing record in the related table.

b. Difference between primary key and foreign key

Feature	Primary key	Foreign key
Purpose	Identifies a record	Links to another table
Uniqueness	Must be unique	Can have duplicates
Null values	Not allowed	Can allow null (sometimes)
Location	In its own table	In another table

Table 7.5. Difference between primary key and foreign key

Application activity 7.1

- 1) Define physical independence.
- 2) Identify the database users in the following examples:
 - a) School IT manager controlling student records
 - b) Teacher entering marks
 - c) Student viewing results
- 3) Why are database access levels important?
- 4) Draw a table showing a primary key and foreign key

7.2 Types of relationship

Learning activity 7.2

A. Choose the correct answer

- 1) A one-to-one relationship means:
 - a) One record relates to many
 - b) One record relates to one
 - c) Many records relate to one
 - d) No relationship
- 2) In a one-to-many relationship:
 - a) One record links to many records
 - b) Many records link to one record only
 - c) Both a and b
 - d) None
- 3) Many-to-many relationships are implemented using:
 - a) Primary key
 - b) Foreign key
 - c) Junction table
 - d) Index

B. Answer with True or False

- 4) A student can take many courses.
- 5) A one-to-one relationship is the most common.
- 6) A junction table is used in many-to-many relationships.

C. Answer the following questions

- 7) Give one real-life example of a one-to-many relationship and explain it.
- 8) Give one example of many-to-many relationship.

In a database, a relationship shows how data in one table is connected to data in another table. Relationships help organize data and make it easier to retrieve related information.

7.2.1 One to one relationship

A one-to-one (1:1) relationship occurs when one record in a table is related to exactly one record in another table, and vice versa. Each record in table A matches only one record in table B and each record in table B matches only one record in table A.

Example: Student and School Identity Card

Each student is issued one school identity card, and each school identity card belongs to one student.

Student Table

Student Name	class
Kalisa	S4
Uwase	S5

Table 7. 6. Student table

School Identity Card Table

Card Number	Student Name	Date Issued
ID101	Kalisa	2024-01-15
ID102	Uwase	2024-01-15

Table 7. 5. School Identity Card Table

From the tables above, Kalisa has one school identity card (ID101) and Uwase has one school identity card (ID102). Similarly, card ID101 belongs only to Kalisa, and card ID102 belongs only to Uwase. This is a one-to-one relationship because each student is associated with exactly one identity card, and each identity card is associated with exactly one student.

7.2.2 One to many relationship

A one-to-many (1:N) relationship occurs when one record in a table is related to many records in another table, but each record in the second table is related to only one record in the first table. This means: One to Many or Many to One.

Example 1: Department and Employees

One department can have many employees, but each employee belongs to one department.

DepartmentID	DepartmentName
1	ICT
2	Finance

Table 7. 6. Department table

EmployeeID	name	department ID
101	Kalisa	1
102	Uwase	1
103	Mutesi	2

Table 7. 7. Employee table

DepartmentID in Employee table is a foreign key, one department can have many employees, and each employee belongs to one department.

A one-to-many (1:N) relationship is implemented by adding a foreign key in the table on the "many" side. The foreign key stores the value of the primary key from the table on the "one" side.

This occurs when one record in one table is related to many records in another table.

7.2.3 Many to many relationship

Many-to-many (M:N) relationship occurs when many records in one table are related to many records in another table. Students and Subjects, a student can take many subjects, and a subject can have many students. This relationship is usually handled using a third table (junction table).

A many-to-many (M:N) relationship occurs when one record in Table A can relate to many records in Table B, and one record in Table B can relate to many records in Table A .

This type of relationship cannot be implemented directly, so we use a junction (bridge) table to break it into two one-to-many relationships.

Junction Table in a Database also called a link table, bridge table, or associative table, is a table used to implement a many-to-many (M:N) relationship between two tables.

In a many-to-many relationship, one record in the first table can be related to many records in the second table, and one record in the second table can also be related to many records in the first table. A junction table is created to store these relationships.

Example 1: Students and Courses

A student can take many courses, and a course can have many students.

Student table

StudentID (PK)	Name
101	Kalisa
102	Uwase

Table 7.9. *Student table*

Course table

CourseID	CourseName
C1	Math
C2	ICT

Table 7. 10. *Customer table*

Enrollment table (Junction table)

StudentID	CourseID
101	C1
101	C2
102	C2

Table 7. 11. *Enrollment table*

- A student can appear multiple times (taking different courses)
- A course can appear multiple times (taken by different students)
- Enrollment table links both tables

Application activity 7.2

- 1) What is a composite primary key?
- 2) a) What is a junction table?
b) What is another name for a junction table?
- 3) A student can take many courses, and a course can have many students. Draw the three tables needed to implement this relationship and identify the junction table.
- 4) Create an example of a one-to-many relationship from a school environment. Draw the two tables and show how they are related.

5) Draw your own tables for one-to-one relationship and many-to-many relationship

7.3 Entity relationship diagram

Learning activity 7.3

A. Choose the correct answer

- 1) An entity is:
a) A relationship b) A real-world object
c) A column d) A row
- 2) Attributes are:
a) Tables b) Properties of an entity
c) Relationships d) Keys
- 3) A relationship shows:
a) Data types b) Connection between entities
c) Rows d) Columns

B. Answer with True or False

- 3) An entity must always have attributes.
- 4) A weak entity can exist alone.
- 5) ERD uses diagrams to represent data.

C. Answer the following questions

- 6) List and explain the element of an entity.
- 7) What is a relationship?

7.3.1 Entities

An entity is a real-world object, person, place, event, or thing about which data is stored in a database. In simple terms, an entity is anything you want to keep information about. An entity is sometimes referred to as a data object because it represents something for which information is collected and maintained.

Examples of entities: Student, Teacher, Course and Book

In a school system, Student is an entity and Subject is also an entity.

a. Entity with attributes

Every entity has a set of attributes, which are the characteristics, properties, or details that describe the entity. Attributes provide specific information about an entity and help distinguish one instance of the entity from another.

For example, if Student is an entity, its attributes may include Student ID, Name, Age, Gender, and Class. These attributes store important details about each student in the database. Without attributes, an entity would only represent a general object without any useful information.

Example of an entity: Student table

StudentID	Name	Age	Gender	Class
01	SEMANA Jean	18	Male	Senior 4
02	UWASE Divine	19	Female	Senior 5
03	KEZA Brienne	17	Female	Senior 3

Table 7. 12. Example an entity

Student is an entity and StudentID, Name, Age, Gender and Class are attributes.

b. Types of entities

In database design, entities are classified according to how they exist and how they are identified. Some entities can exist independently and have their own unique identifiers, while others depend on another entity for their existence. Understanding the different types of entities helps in designing accurate and efficient databases and establishing proper relationships between data items.

- **Strong entity:** It exists on its own and has a primary key (e.g., Student)

Employee_ID	Employee_Name	Department
E001	James MUGISHA	Finance
E002	Alice UWIMANA	ICT

Table 7.13: An example of strong entity

- **Weak entity:** It depends on another entity and it cannot exist alone (e.g., Dependent (depends on Employee))

Employee_ID	Employee_Name	Relationship
E001	Grace MUGISHA	Spouse
E001	David MUGISHA	Child
E002	Eric UWIMANA	Child

Table 7.14: An example of weak entity

7.3.2 Attributes

An attribute is a property or characteristic of an entity. It describes more details about the entity.

Example of attributes

Consider the entity Student. Some attributes of a student are StudentID, StudentName, Gender, DateOfBirth, and Class.

StudentID	StudentName	Gender	DateOfBirth	Class
S001	Alice	Female	15/03/2008	S4A
S002	John	Male	22/07/2007	S4B
S003	Diane	Female	10/11/2008	S4A
S004	Eric	Male	05/01/2007	S4C

Table 7. 8. Example of attributes

Types of attributes

Attributes describe the properties or characteristics of an entity in a database. They help define and provide more details about the data stored in a system. Attributes can be classified into different types based on their structure and the kind of values they hold.

- Simple attribute: It cannot be divided further. It represents a single, basic piece of data. Example: Age, gender or employee_id.
- Composite attribute: It can be divided into smaller parts for better organization and clarity.

Example: Name can be divided into First Name, Middle Name, and Last Name. For instance, "Jean Claude Dushimimana" can be split into its components.

- Single-valued attribute: Has only one value for each entity. Example: Date_of_Birth.

StudentID	Name	Date_of_Birth
101	Alice	2000-05-10
102	Bob	1999-08-22
103	Charlie	2001-01-15

Table 7.16. Single-valued attribute

- Multi-valued attribute: It can have more than one value for a single entity. Example: Phone numbers.

StudentID	Name	Phone Numbers
101	Alice	078xxxxxxx, 072xxxxxxx
102	Bob	079xxxxxxx
103	Clara	078xxxxxxx, 073xxxxxxx, 072xxxxxxx

Table 7. 17. Multi-valued attribute

- Key attribute: It uniquely identifies each entity in a dataset and ensures no duplicate records.

Example: StudentID.

StudentID	Name	Age
101	Alice	20

102	Bob	22
103	Clara	19

Table 7.18. Key attribute

7.3.3 Relationship

A relationship is a connection between two or more entities in a database. It shows how data in one entity is related to data in another entity. In simple terms a relationship explains how entities are connected.

Example of relationship: Student and Course, a relationship exists between the Student and Course entities where a student takes a course, commonly expressed as: Student $\rightarrow$ takes $\rightarrow$ Course.

StudentID	Name
101	Alice
102	Bob
103	Clara

CourseID	CourseName	StudentID	CourseID
C01	Mathematics	101	C01
C01	Mathematics	101	C02
C02	Database	102	C02
C03	Programming	103	C03

From the tables above: Alice (101) takes Mathematics (C01) and Database (C02); Bob (102) takes Database (C02); Clara (103) takes Programming (C03). This shows a many-to-many relationship because one student can take multiple courses and one course can be taken by many students. The Enrollment table links Student and Course using StudentID and CourseID.

7.3.4 Representation of E-R-D

The representation of an Entity-Relationship Diagram (ERD) refers to the symbols and shapes used to show entities, attributes, and relationships in a database design. ERD uses diagrams (shapes) to visually show how data is organized and connected.

a) Symbols used in ERD

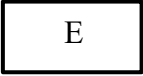
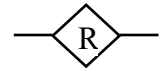
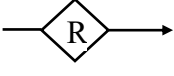
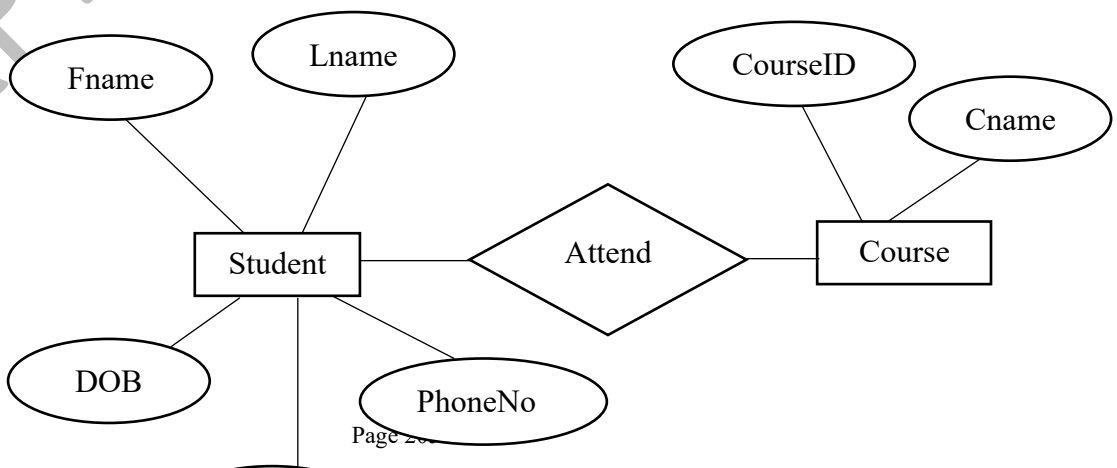
Symbols	Symbol name
	Entity
	Attribute
	Primary key
	Many_to_many relationship
	Many_to_one relationship

Table 7.19. Symbols used in ERD

Example of ERD for Student and Course

The ERD below is made respectively by the following two entities and their corresponding attributes(fields): Student(RegNo, Fname, Lname, DOB, RegNo) and Course(CourseID, Cname).



b) Importance of ERD representation

- Helps visualize database structure
- Makes design easy to understand
- Helps in converting into tables (relational model)
- Reduces design errors

Application activity 7.3

- 1) A company stores information about its customers and orders in a database. The Customer entity has the attributes CustomerID, FName, LName, Age, Address and PhoneNo, while the Order entity has the attributes OrderID and Date. One customer can place many orders, but each order is placed by only one customer. Draw an ERD showing the entities, their attributes and the relationship between them.
- 2) State the key attribute in the Student table and explain why it is considered a key attribute.

StudentID	StudentName	Gender	DateOfBirth	Class
S001	Alice	Female	15/03/2008	S4A
S002	John	Male	22/07/2007	S4B
S003	Diane	Female	10/11/2008	S4A
S004	Eric	Male	05/01/2007	S4C

7.4 Relational model

Learning activity 7.4

A. Choose the correct answer

- 1) A table consists of:
a) Rows and columns b) Keys only
c) Programs d) Files
- 2) A column represents:
a) A record b) A field c) A table d) A database
- 3) A row represents:
a) A column b) A record c) A key d) A relationship

B. Answer with True or False

- 4) A primary key can be NULL.
- 5) Foreign keys connect tables.
- 6) Data types define the type of data stored.

C. Answer the following questions

- 7) What do you understand by a table?
- 8) Distinguish a row from a column.
- 9) Explain the relational model and state two of its main components.

7.4.1 Tables

A table is a structure used in a database to store data in an organized way using rows and columns. It represents a specific entity.

7.4.2 Columns

A column is a vertical part of a table that stores a specific type of data and represents an attribute (field) of an entity. It serves as a category of information in a table, where each column has a unique name, stores only one type of data, and is assigned a specific data type such as text, number, date, or currency to ensure that data is stored consistently and accurately. Columns organize data into clear categories, ensure consistency (same type of data in a column) and help in searching and sorting data easily

Example of columns: Students table

StudentID	Name	Age
-----------	------	-----

Table 7. 9. Columns in database

StudentID, Name, Age are columns and each column stores a particular type of data. StudentID column stores ID numbers, Name column stores student names, Age column stores student ages.

a) Data types in columns

A data type is a classification that specifies the kind of values that can be stored in a column. It defines the format and nature of the data, such as text, numbers, dates, or logical values. Every column in a table is assigned a data type to ensure that only appropriate data is entered and stored.

For example, a StudentName column may use the Text data type because it stores letters and words, while an Age column may use the Number data type because it stores numeric values. By assigning data types to columns, databases maintain data accuracy, consistency, and efficient storage.

Columns are defined with data types such as:

- Integer (INT): Numbers
- Text (VARCHAR): Words or letters
- Date: Dates

7.4.3 Rows

A row is a horizontal part of a table that represents a single record or entry and contains all the information about one item in the table. Each row stores data for one record, has values for the same set of columns as other rows, and is usually identified by a primary key. Rows are important because they store the actual data in the table, allow easy retrieval and updating of records, and help organize information clearly and systematically. A row is like a record in a register book, where each line contains the complete details of one person.

Example of rows in Students table

StudentID	Name	Age	Class
101	Kalisa	16	S4
102	Uwase	17	S5

Table 7. 101. Rows examples

Each line (101, Kalisa, 16, S4) is a row and represents one student.

7.4.4 Primary keys

A primary key (PK) is a special column (or a combination of columns) in a table that uniquely identifies each row (record) in that table. It ensures that no two rows have the same value in that column. Primary keys are important because it prevent duplicate records, allow quick searching and sorting, ensure data integrity and enable relationships between tables when it is used as foreign keys in other tables.

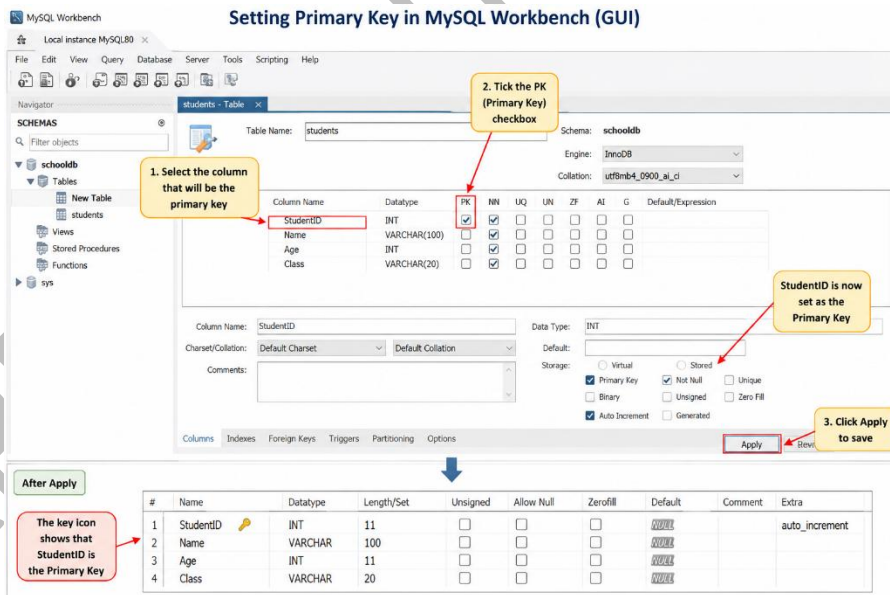


figure 7.2 Setting Primary key in MySql

Key features of primary key

- Unique: Each value in the primary key column must be different.
- Non-null: A primary key column cannot have empty or NULL values.
- Single or composite: Can be one column (single primary key) or multiple columns (composite primary key).
- Identifies records: Used to reference and retrieve specific data efficiently.

Example of primary key in Students table:

StudentID (PK)	Name	Age	Class
101	Kalisa	16	S4
102	Uwase	17	S5
103	Rukundo	16	S4

Table 7. 11. Example of primary key

- StudentID is the primary key
- Each student has a unique StudentID
- No two students can have the same StudentID

7.4.5 Foreign keys

A foreign key (FK) is a column (or a set of columns) in one table that refers to the primary key in another table. It is used to establish and enforce a link (relationship) between the two tables. It is used to create relationships between tables (one-to-many, many-to-many, ensure data consistency: you cannot assign a ClassID to a student that doesn't exist in the Classes table and help in joining tables to retrieve combined data.

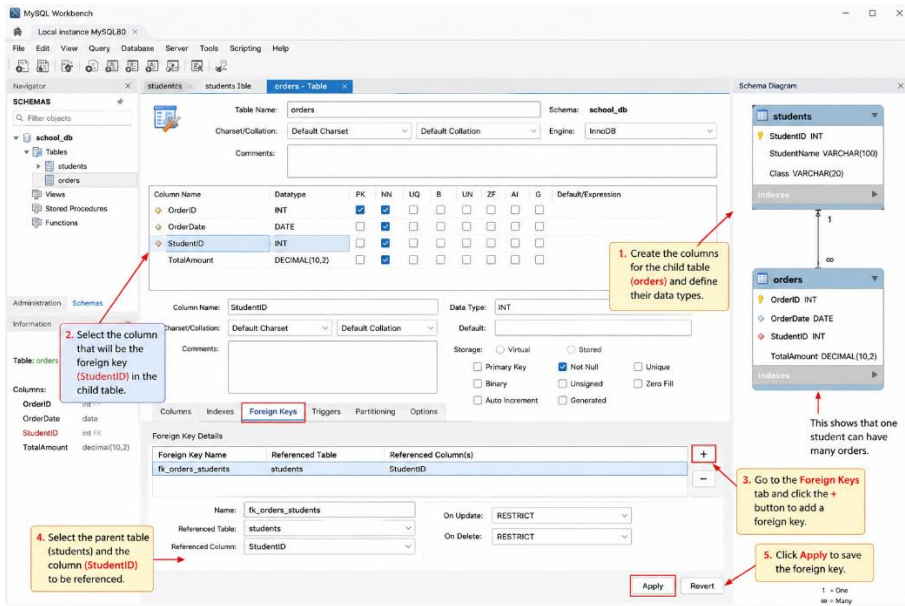


figure 7.3 Setting foreign key in MySql

Key features of a foreign key

- References another table: Points to the primary key in another (or the same) table.
- Maintains referential integrity: Ensures that the value in the foreign key column exists in the referenced table.
- Can have duplicates: Multiple rows in the referencing table can point to the same primary key.
- Can be null: A foreign key can be empty if optional, depending on the database design.

Example of foreign key in Students table

StudentID (PK)	Name	Age	ClassID (FK)
101	Kalisa	16	1
102	Uwase	17	2
103	Rukundo	16	1

Table 7. 123. Example of foreign key

ClassID (PK)	ClassName
1	S4
2	S5

Table 7. 13. Example of foreign key

ClassID in Students table is the foreign key, it refers to ClassID in Classes table (primary key) and it links each student to the class they belong to.

7.4.6 Data types

A data type defines the kind of data that can be stored in a column of a table. It helps the database understand what operations can be performed, ensures data integrity, and optimizes storage.

Why data types are important:

- Ensures data integrity: Prevents wrong types of data from being entered (e.g., text in a numeric column)
- Optimizes storage: Only allocates memory needed for that type
- Supports validation and operation: You can sum numeric columns, search text columns, or filter by dates

In short, data types tell the database what kind of data to expect, which helps in storing, retrieving, and processing data efficiently.

a. Numeric data types

Numeric data types are used to store numbers. They are suitable for calculations such as addition, subtraction, multiplication, and division.

- INT / INTEGER: Whole numbers (e.g., Age INT: 16, 25, 42)
- FLOAT / DOUBLE / DECIMAL: Numbers with decimals (e.g., Price DECIMAL(6,2): 125.50)

b. Character/ text data types

Character or text data types are used to store letters, words, symbols, and sentences. They are suitable for storing names, addresses, and descriptions.

- CHAR(n): Fixed-length string (e.g., Gender CHAR(1): 'M' or 'F')
- VARCHAR(n): Variable-length string (e.g., Name VARCHAR(50): "Alice Johnson")
- TEXT: Long text or paragraphs

c. Date and time data types

Date and time data types are used to store dates, times, or both. They help in tracking events and managing time-related information.

- DATE: Only date (YYYY-MM-DD), e.g., DOB DATE : 2007-03-15
- TIME: Only time (HH:MM: SS)
- DATETIME / TIMESTAMP: Both date and time

d. Boolean/logical data type

Boolean or logical data type is used to store true or false values. For example IsActive Boolean returns TRUE or FALSE

Example table using different data types

StudentID (INT)	Name (VARCHAR)	Age (INT)	DOB (DATE)	GPA (DECIMAL)	IsActive (BOOLEAN)
101	Kalisa	16	2007-03-15	3.75	TRUE
102	Uwase	17	2006-08-20	3.60	TRUE

103	Rukundo	16	2007-06-05	3.90	FALSE
-----	---------	----	------------	------	-------

Table 7. 14. Different data types

Application activity 7.4

- 1) Differentiate column and row in a database table
- 2) Create a table with at least 4 columns, with different data types and then identify primary key and foreign key.
- 3) Given Student table:

StudentID	Name	Age
1011	Kalisa	16
102	Uwera	17

- a) Identify the rows and columns in the table
- b) Identify the primary key in a Students table.
- c) Identify the foreign key
- 4) Why should database records be entered carefully?
- 5) Why is honesty important when managing data?
- 6) Why should you avoid duplicate records in a database?

7.5 Keys in relational model

Learning activity 7.5

A. Choose the correct answer

- 1) A primary key:
 - a) Can be duplicated
 - b) Cannot be NULL
 - c) Is optional
 - d) Stores images

B. Answer the following questions:

- 2) What is a constraint?
- 3) Explain the use of a table in a relational database
- 4) Explain a many-to-many relationship?
- 5) Why is accuracy important when creating database tables?

7.5.1 Tables creation

Creating tables in a relational database involves defining the structure of the table, specifying the columns, their data types, and applying constraints that enforce rules on the data. This ensures data integrity, consistency, and proper relationships between tables.

Steps for Creating a Table in MySQL Workbench:

- Open MySQL Workbench.
- Connect to the MySQL server.
- Expand the required schema.
- Right-click Tables and select Create Table.
- Enter the table name.
- Add column names and data types.
- Select PK for the primary key column.
- Click Apply.
- Review the generated SQL.
- Click Apply and Finish.

When designing a database, careful planning of columns, data types, and constraints helps improve performance, data integrity, and ease of maintenance. The GUI tools in MySQL Workbench simplify the design process and reduce errors.

It is important to define the table name, column names, data types, and constraints. The table name identifies the table, while column names specify the information stored in each field. Data types determine the kind of data that can be entered into each column, and constraints such as Primary Key (PK) and Not Null (NN) help maintain data accuracy and integrity.

a. Primary key constraint

A primary key uniquely identifies each row in a table, it cannot contain NULL values, usually applied to ID columns.

In the example below, StudentID is the primary key because each student has a unique ID.

StudentID (PK)	Name
101	Alice
102	Brian
103	Diane

Steps to run SQL queries:

1. Open Xampp program from start menu
2. Click on start button Apache, MySQL, FileZilla, Mercury and Tomcat
3. Once MySQL has started, click on Admin or go in your browser URL

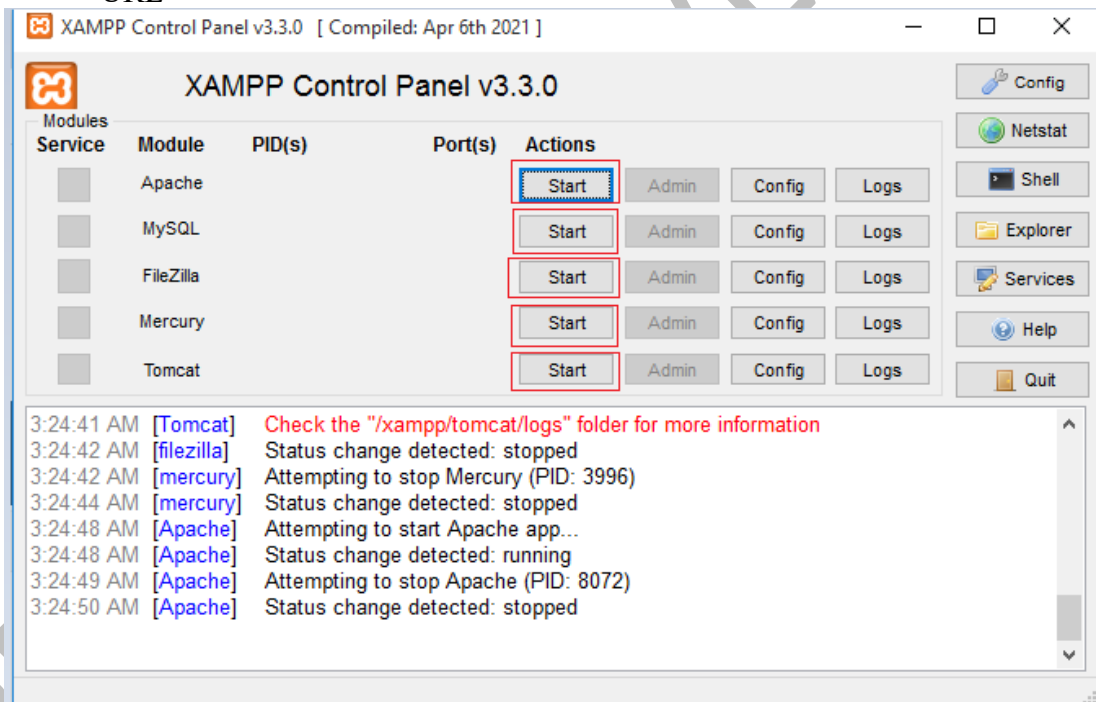


Figure 7. 4. Xampp control panel

4. Type <http://localhost/phpmyadmin/> then confirm with Enter button

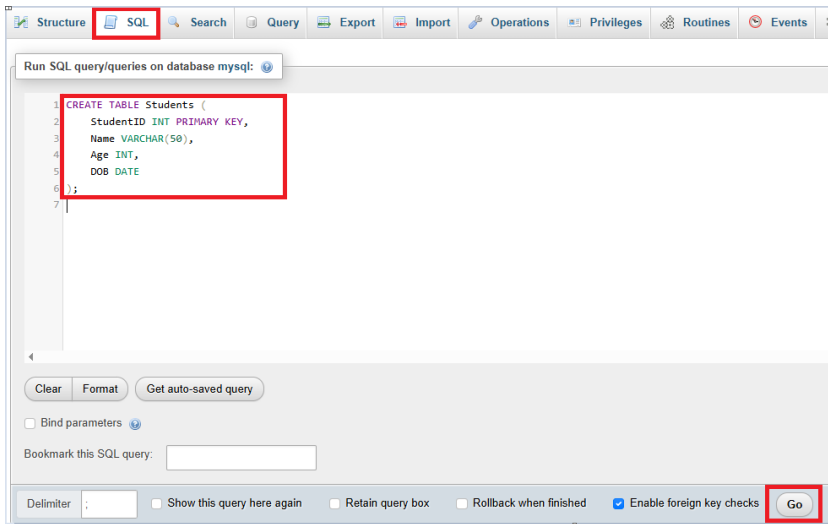


Figure 7. 5. SQL queries window

5. On the opened window, click on SQL menu
6. Write SQL queries in the queries zone
7. Then click on Go button to run your query.

Here, StudentID is the primary key ensuring each student has a unique identifier

b. Foreign key constraint

- A foreign key links one table to another.
- It ensures referential integrity: the value in one table must match a value in the related table's primary key.

Example: Consider two tables Departments and Employees.

For the Department table the Primary Key (PK) is DepartmentID

DepartmentID (PK)	DepartmentName
D01	Finance
D02	Human Resources
D03	ICT

Employee table:

EmployeeID (PK)	EmployeeName	DepartmentID (FK)
E001	Alice	D01
E002	John	D03
E003	Mary	D01
E004	Peter	D02

In the Employees table, the **DepartmentID** column is a foreign key because it refers to the DepartmentID column in the Departments table.

The foreign key constraint ensures that every DepartmentID entered in the Employees table must already exist in the Departments table. For instance, a DepartmentID such as D05 cannot be assigned to an employee unless D05 exists in the Departments table. This maintains consistency and accuracy of data between the two tables.

Summary of key points:

Feature	Purpose
Primary Key	Uniquely identifies each row, cannot be NULL
Foreign Key	Links tables, ensures referential integrity
Data Types	Define kind of data (INT, VARCHAR, DATE, etc.)
Constraints	Rules for data validation (NOT NULL, UNIQUE, PRIMARY/FOREIGN KEY, etc.)

Table 7. 15. Summary of key points

7.5.2 Relation model representation

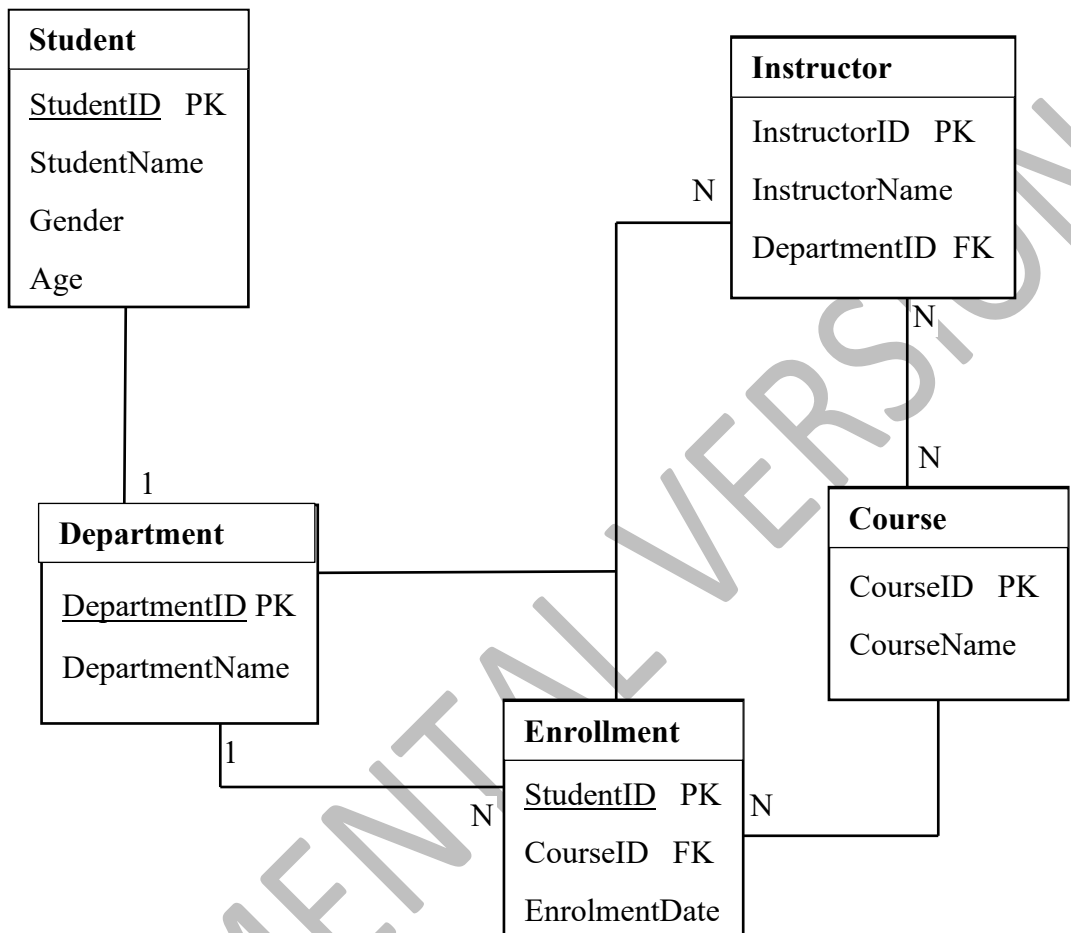
The relational model is a way of structuring a database using tables (relations), columns (attributes), and rows (tuples).

Entity-Relationship Diagram to relational model representation is the process of converting a visual ERD into actual tables in a relational

database. This ensures that the database design is properly implemented and maintains data integrity.

a) Example of a Real ERD

A School Management System consists of five entities: Student, Course, Department, Instructor, and Enrollment. The Student entity stores student details, the Course entity stores course information, and the Department entity stores departmental information. The Instructor entity is linked to the Department entity through DepartmentID, creating a one-to-many (1:N) relationship, where one department can have many instructors. The Enrollment entity connects Students and Courses by storing StudentID, CourseID, and EnrollmentDate, resolving the many-to-many (M:N) relationship between students and courses. Thus, a student can enroll in many courses, and a course can have many students.



b) Steps to convert ERD to a Relational Model

Step 1: Convert entities to tables

Each entity in the ERD becomes a table. The attributes become columns, and the primary key uniquely identifies each record.

Example of ERD Entity:

Entity: Student, attributes: StudentID as PK, Name, Age

Step 2: Convert relationships

c. One-to-one relationship

The primary key of one table is used as a foreign key in the related table and can also merge the two tables if appropriate.

Example:

Entities: Person and Passport, passport table can have PersonID as a foreign key referencing PersonID in Person table.

d. One-to-Many relationship

The primary key of the one-side table becomes a foreign key in the many-side table.

Example:

Entities: Student (1) $\rightarrow$ Course (N)

A student can enroll in many courses, and a course can have many students.

e. Many-to-Many relationship

Create a junction table (also called associative table) to break the many-to-many relationship into two one-to-many relationships and junction table contains foreign keys referencing both related tables.

Example: Entities Student (M) $\leftrightarrow$ Course (N)

Step 3: Handle attributes

- Simple attributes: columns in the table
- Composite attributes: split into multiple columns
- Multivalued attributes: create a separate table

Example:

- Entity: Student,
- Attribute: PhoneNumbers (multivalued). A student may have several phone numbers.

Step 4: Apply data types and constraints

- Assign appropriate data types such as INT, VARCHAR, DATE, or DECIMAL.
- Set primary keys for unique identification.
- Create foreign keys to establish relationships.
- Use NOT NULL where data is mandatory.
- Use UNIQUE where duplicate values are not allowed.
- Save and apply the table design.

Summary table: ERD relational model

ERD component	Relational model representation
Entity	Table
Attribute	Column
Primary Key	Primary key column
One-to-One	Foreign key (or merge tables)
One-to-Many	Foreign key in “many” table
Many-to-Many	Junction table with composite primary key
Multivalued Attr.	Separate table

Table 7. 16. Summary of ERD relational model

Application activity 7.5

- 1) Write SQL code to create an Employee (EmpID, Fname, Lname, Age, Address) and Salary (SalaryID, EmpID, SalAmount, Salmonth, PayDate) tables with all necessary constraints.

7.6 Normalization

Learning activity 7.6

A. Choose the correct answer

- 1) Normalization is used to:
 - a) Increase duplication
 - b) Reduce redundancy
 - c) Delete data
 - d) Create programs
- 2) Define normalization.
- 3) State two roles of normalization.

7.6.1 Normalization definition

Normalization is the process of organizing data in a database to reduce redundancy, improve data integrity, minimize the use of null values, and avoid update anomalies. It helps make database structures more efficient and easier to maintain.

Normalization is achieved through a series of rules known as normal forms, including First Normal Form (1NF), Second Normal Form (2NF) and Third Normal Form (3NF).

Example of a table before normalization

StudentID	Name	Course1	Course2
101	Kalisa	Math	ICT

Table 7. 17. Example before normalization

Repeating columns (Course1, Course2), difficult to manage if more courses are added.

The Student table after normalization

StudentID	Name
101	Kalisa

Table 7. 18. Student table after normalization

Courses table generated by normalization:

CourseID	CourseName
1	Math
2	ICT

Table 7. 19. Course table after normalization

Enrollment table also generated by normalization of the original table

StudentID	CourseID
101	1
101	2

Table 7. 20. Enrollment table after normalization

No duplication, data is well organized and easy to update and manage

7.6.2 Normalization role

Normalization plays a very important role in designing efficient and reliable databases. It ensures that data is organized properly, accurate, and easy to manage.

- Reduces data redundancy: Normalization removes duplicate data stored in multiple places and each piece of information is stored only once. Instead of repeating a student's name in many rows, it is stored once and referenced where needed.
- Improves data consistency: When data is not duplicated, it is easier to keep it consistent and updating data in one place automatically reflects everywhere it is used.

Example: If a student's name changes, you update it once, not in many tables.

- Maintains data integrity: Ensures that relationships between tables are correct and valid and prevents errors such as entering invalid or unrelated data.

Example: A student cannot be assigned to a course that does not exist.

- Simplifies data management: Organized tables make it easier to insert, update, and delete data and reduces confusion when working with large databases.
- Improves database efficiency: Saves storage space by avoiding repeated data and makes queries (searching data) faster and more efficient.
- Enhances flexibility: It becomes easier to modify or expand the database structure and new data can be added without affecting existing data.

Application activity 7.6

1) Normalize the following Library table with repeated data:

BookID	Title	Author1	Author2	Author3
B001	Database Basics	John Smith	Mary Jones	
B002	Web Design	Alice Brown		
B003	Programming	David Lee	James Kim	Sarah W

7.7 Types of normal forms

Learning activity 7.7

A. Choose the correct answer

- 1) 1NF requires:
 - a) Multiple values in a cell
 - b) Atomic values
 - c) Duplicate rows
 - d) No keys
- 2) 2NF removes:
 - a) Redundancy
 - b) Partial dependency
 - c) Tables
 - d) Keys
- 3) 3NF removes:
 - a) Primary keys
 - b) Transitive dependency
 - c) Columns
 - d) Rows

B. Answer with True or False

- 4) 1NF allows repeating groups.
- 5) 2NF depends on 1NF.
- 6) 3NF removes transitive dependency.

7.7.1 First normal form

First Normal Form (1NF) is the first step in normalization. It ensures that a table is organized so that each column contains atomic (indivisible) values and there are no repeating groups or multiple values in a single field. A table is in First Normal Form (1NF) if:

- Each column contains only one value (atomic value)
- There are no repeating columns or groups
- Each record (row) can be uniquely identified (using a primary key)

Example before 1NF

StudentID	Name	Courses
101	Kalisa	Math, ICT
102	Uwase	Biology, Chem

Table 7. 21. Example before 1NF

The Courses column has multiple values, not atomic violates 1NF

The table after 1NF:

StudentID	Name	Course
101	Kalisa	Math
101	Kalisa	ICT
102	Uwase	Biology
102	Uwase	Chem

Table 7. 22. Example after 1NF

- Each cell contains only one value
- No repeating groups
- Table satisfies 1NF

Key rules of 1NF

- No multiple values in one column • Use separate rows for multiple
- No repeating columns (like values Course1, Course2)

First Normal Form (1NF) makes data simple and well-organized by ensuring that each field contains a single, atomic value. It improves data storage and retrieval by reducing unnecessary repetition and making information easier to manage. It also prepares the table for further normalization, such as Second Normal Form (2NF) and Third Normal Form (3NF).

7.7.2 Second normal form

Second Normal Form (2NF) is the next step after First Normal Form (1NF). It ensures that the table is not only in 1NF but also that all non-key attributes fully depend on the entire primary key. A table is in Second Normal Form (2NF) if:

- It is already in First Normal Form (1NF)
- All non-key attributes are fully dependent on the whole primary key, not just part of it. This mainly applies when a table has a composite primary key (a primary key made of more than one column).

Understanding partial dependency

Partial dependency occurs when a column depends on only part of a composite key, not the whole key. 2NF removes this problem.

Example before 2NF

StudentID	CourseID	StudentName	CourseName
101	C1	Kalisa	Math
102	C2	Uwase	ICT

Table 7. 23. Example before 2NF

Composite Primary Key: (StudentID + CourseID)

- StudentName depends only on StudentID
- CourseName depends only on CourseID
- This is partial dependency and it violates 2NF requirements.

Example after 2NF

Students table

StudentID	StudentName
101	Kalisa
102	Uwase

Table 7. 24. Example after 2NF (Students table)

Courses table

CourseID	CourseName
C1	Math
C2	ICT

Table 7. 25. Example after 2NF

Enrollment table

StudentID	CourseID
101	C1
102	C2

Table 7. 26. Example after 2NF (Enrollment table)

- Each non-key attribute depends on the whole key
- No partial dependency
- Table is in 2NF

The Second Normal Form (2NF) must first satisfy all the requirements of First Normal Form (1NF). It removes partial dependency, meaning that non-key attributes must depend on the whole primary key rather than only part of it. To achieve this, large tables may be split into smaller, related tables to reduce data redundancy and improve data organization.

Importance of 2NF:

- Reduces data redundancy
- Improves data consistency
- Makes the database more efficient and organized

7.7.3 Third normal form

Third Normal Form (3NF) is the next step after Second Normal Form (2NF). It ensures that the table has no transitive dependency, meaning that non-key attributes depend only on the primary key and not on other non-key attributes. A table is in Third Normal Form (3NF) if:

- It is already in Second Normal Form (2NF)
- There is no transitive dependency (no non-key attribute depends on another non-key attribute)

Understanding transitive dependency:

A transitive dependency occurs when a non-key attribute depends on another non-key attribute instead of the primary key.

In Third Normal Form (3NF), a **primary key should determine non-key attributes directly**, and a non-key attribute should not depend on another non-key attribute. Therefore, the dependency **Primary key → non-key → another non-key** is not allowed because it creates a **transitive dependency**.

Example before 3NF:

StudentID	StudentName	ClassID	ClassName
101	Kalisa	1	S4
102	Uwase	2	S5

Table 7. 27. Example before 3NF

- ClassName depends on ClassID, not directly on StudentID
- This is a transitive dependency: violates 3NF

Example after 3NF:

Students table

StudentID	StudentName	ClassID
101	Kalisa	1
102	Uwase	2

Table 7. 289. Example after 3NF (Students table)

Classes table

ClassID	ClassName
1	S4
2	S5

Table 7. 29. Example after 3NF (Classes table)

- All non-key attributes depend only on the primary key
- No transitive dependency
- Tables satisfy 3NF

Key Points of 3NF

- Must first satisfy 2NF
- Removes transitive dependencies
- Separates related data into different tables

Third Normal Form (3NF) eliminates unnecessary data duplication by removing transitive dependencies and organizing data into appropriate related tables. It improves data consistency and accuracy by ensuring that each piece of information is stored in the most suitable place. It also

makes the database more flexible, organized, and easier to maintain and update.

Application activity 7.7

1) Answer with True or False

- a) A primary key can contain duplicate values.
- b) A foreign key must match a value in another table.
- c) Physical independence allows changes without affecting users.
- d) A row represents a single record in a table.
- e) A table can exist without columns.

Practical question

2) Convert the following table into 1NF, 2NF, 3NF:

ProductID	ProductName	SupplierName	SupplierPhone
P01	Laptop	Tech Ltd	0781111111
P01	Laptop	Tech Ltd	0782222222
P02	Phone	Mobile Inc	0783333333
P03	Laptop	Tech Ltd	0781111111
P03	Laptop	Tech Ltd	0782222222

3) Explain differences between 1NF, 2NF, and 3NF.

7.8 End of unit assessment

7.8.1 Project activity

Project Title: Designing and Creating a School Database System

Guidance:

You are an ICT technician in a school. The school administration needs a database system to manage students, classes, subjects, assessments, and academic records efficiently. Using a database management tool such as XAMPP/phpMyAdmin, design and implement a relational database that meets the school's requirements.

- Open XAMPP and start Apache and MySQL services.
- Open a browser and type <http://localhost/phpmyadmin/>.
- Create a database called SchoolDB.

Questions on the project

- 1) Identify the entities required for the school database and create appropriate tables.
- 2) State the Primary Keys used and explain why they were selected.
- 3) State the Foreign Keys used and explain their purpose.
- 4) Draw and explain your ERD.
- 5) Explain how you ensured your database satisfies Third Normal Form (3NF).
- 6) Describe any challenges encountered and how you solved them.
- 7) Explain three advantages of using a relational database in a school environment.
- 8) Explain the types of relationships present in your database and provide examples.

7.8.2 Assessment questions

A. Multiple choice

- 1) Which of the following best describes a database?
 - a) A collection of programs
 - b) A collection of unrelated files
 - c) An organized collection of related data
 - d) A type of hardware
- 2) Which user has permission to control access and security in a database?
 - a) Guest user
 - b) Standard user
 - c) Database administrator
 - d) Student
- 3) In a relational database, data is stored in:
 - a) Files
 - b) Trees
 - c) Tables
 - d) Graphs

- 4) Which of the following ensures that each record is unique?
 - a) Foreign key
 - b) Data type
 - c) Primary key
 - d) Attribute
- 5) Which statement is TRUE about a foreign key?
 - a) It must always be unique
 - b) It links two tables
 - c) It cannot be null
 - d) It stores only numbers
- 6) Which type of relationship requires a junction table?
 - a) One-to-one
 - b) One-to-many
 - c) Many-to-many
 - d) None
- 7) What does an entity represent in a database?
 - a) A relationship
 - b) A real-world object
 - c) A data type
 - d) A constraint
- 8) Which data type is best for storing a person's name?
 - a) INT
 - b) BOOLEAN
 - c) VARCHAR
 - d) DATE

B. Fill in the blanks

- 9) A _____ is used to uniquely identify each record in a table.
- 10) A _____ connects two tables in a database.
- 11) A _____ is a property that describes an entity.
- 12) A _____ contains multiple tables in a relational database.
- 13) Match the items in Column A with Column B:

Column A	Column B
a) Entity	i. Links tables
b) Foreign key	ii. Real-world object
c) Row	iii. Single record
d) Column	iv. Attribute
e) Primary key	v. Unique identifier

C. Answer the following questions:

- 14) State two characteristics of a relational database model.
- 15) Explain why foreign keys are important in maintaining relationships.
- 16) Outline two challenges that may occur if access levels are not controlled
- 17) Consider the following table:

Employees table

EmpID	Name	DeptID
1	Alice	D1
2	Bob	D2

3	Carol	D1
---	-------	----

Departments table

DeptID	DeptName
D1	ICT
D2	Finance

- a) Identify the primary key in each table.
 - b) Identify the foreign key and explain its role.
 - c) State the type of relationship between Employees and Departments.
- 18) Using the phpMyAdmin graphical user interface (GUI), create a table named Teachers with the following fields: TeacherID (Primary Key), Name and Subject.
 - 19) Using the phpMyAdmin graphical user interface (GUI), create a table named Results with the following fields: ResultID (Primary Key), StudentID (Foreign Key referencing the Students table) and Marks.

REFERENCES

1. Gaskin, S. E., Vargas, G., & Geoghan, D. (2020). *GO! with Google Docs: Getting started*. Pearson Education.
2. Lambert, J., & Cox, J. (2022). *Microsoft Office 365 & Office 2021: Step by step*. Microsoft Press.
3. Mano, M. M., & Ciletti, M. D. (2018). *Digital design (6th ed.)*. Pearson.
4. Floyd, T. L. (2017). *Digital fundamentals (11th ed.)*. Pearson.
5. Forouzan, B. A. (2018). *Foundations of computer science (4th ed.)*. Cengage Learning.
6. Brassard, G., & Bratley, P. (2008). *Fundamentals of algorithmics*. Prentice Hall.
7. Downey, A. (2015). *Think Python: How to think like a computer scientist (2nd ed.)*. O'Reilly Media.
8. Sweigart, A. (2019). *Automate the boring stuff with Python (2nd ed.)*. No Starch Press.
9. Duckett, J. (2011). *HTML and CSS: Design and build websites*. Wiley.
10. Castro, E., & Hyslop, B. (2013). *HTML and CSS: Visual quickstart guide (8th ed.)*. Peachpit Press.
11. Meyer, E. A. (2018). *CSS: The definitive guide (4th ed.)*. O'Reilly Media.
12. Duckett, J. (2011). *HTML and CSS: Design and build websites*. Wiley.
13. Coronel, C., & Morris, S. (2019). *Database systems: Design, implementation, & management (13th ed.)*. Cengage Learning.
14. Connolly, T., & Begg, C. (2015). *Database systems: A practical approach to design, implementation and management (6th ed.)*. Pearson.